

Machine Learning for Systems and Control

5SC28

Lecture 1

dr. ir. Maarten Schoukens & prof. dr. ir. Roland Tóth

Control Systems Group
Department of Electrical Engineering
Eindhoven University of Technology

Academic Year: 2025-2026

Learning Outcomes

What is Machine Learning?

Machine Learning vs Systems & Control

Course Overview

Examination Overview

Who Are We?



Dr. ir. Maarten Schoukens (FLUX 5.080, m.schoukens@tue.nl)

Associate professor – Control Systems Group

Specialization: nonlinear system identification, machine learning for systems and control

Office hours: appointment by e-mail



Prof. dr. ir. Roland Tóth (FLUX 5.134, r.toth@thue.nl)

Professor – Control Systems Group

Specialization: data-driven modeling, LPV systems and control, applied machine learning

Office hours: appointment by e-mail



MSc. Gé van Otterdijk (g.j.e.v.otterdijk@tue.nl)

MSc. Máté Kiss

PhD Candidates – Control Systems Group

Specialization: machine learning for systems and control

Office hours: appointment by e-mail





What do you know?

What do you want to
learn?

What is Machine Learning?

How can computers learn to solve problems without being explicitly programmed? In other words, how can computers be made to do what needs to be done, without being told exactly how to do it?¹

[1] Koza, John R.; Bennett, Forrest H.; Andre, David; Keane, Martin A. (1996). *Automated Design of Both the Topology and Sizing of Analog Electrical Circuits Using Genetic Programming*. Artificial Intelligence in Design '96. Springer, Dordrecht. pp. 151–170

What is Machine Learning?

How can computers learn to solve problems without being explicitly programmed? In other words, how can computers be made to do what needs to be done, without being told exactly how to do it?¹

Objective

Policy Learning

Feature Selection

Pattern Recognition

Prediction / Simulation

Classification

[1] Koza, John R.; Bennett, Forrest H.; Andre, David; Keane, Martin A. (1996). *Automated Design of Both the Topology and Sizing of Analog Electrical Circuits Using Genetic Programming*. Artificial Intelligence in Design '96. Springer, Dordrecht. pp. 151–170

What is Machine Learning?

How can computers learn to solve problems without being explicitly programmed? In other words, how can computers be made to do what needs to be done, without being told exactly how to do it?¹

Objective

Policy Learning

Prediction / Simulation

[1] Koza, John R.; Bennett, Forrest H.; Andre, David; Keane, Martin A. (1996). *Automated Design of Both the Topology and Sizing of Analog Electrical Circuits Using Genetic Programming*. Artificial Intelligence in Design '96. Springer, Dordrecht. pp. 151–170

What is Machine Learning?

How can computers learn to solve problems without being explicitly programmed? In other words, how can computers be made to do what needs to be done, without being told exactly how to do it?¹

Objective

Model / Framework

Policy Learning

Prediction / Simulation

Neural Networks

Support Vector Machines

Gaussian Processes

Genetic Programming

[1] Koza, John R.; Bennett, Forrest H.; Andre, David; Keane, Martin A. (1996). *Automated Design of Both the Topology and Sizing of Analog Electrical Circuits Using Genetic Programming*. Artificial Intelligence in Design '96. Springer, Dordrecht. pp. 151–170

What is Machine Learning?

How can computers learn to solve problems without being explicitly programmed? In other words, how can computers be made to do what needs to be done, without being told exactly how to do it?¹

Objective

Policy Learning

Prediction / Simulation

Model / Framework

Neural Networks

Gaussian Processes

[1] Koza, John R.; Bennett, Forrest H.; Andre, David; Keane, Martin A. (1996). *Automated Design of Both the Topology and Sizing of Analog Electrical Circuits Using Genetic Programming*. Artificial Intelligence in Design '96. Springer, Dordrecht. pp. 151–170

What is Machine Learning?

How can computers learn to solve problems without being explicitly programmed? In other words, how can computers be made to do what needs to be done, without being told exactly how to do it?¹

Objective

Model / Framework

Learning Type

Policy Learning

Neural Networks

Supervised Learning

Prediction / Simulation

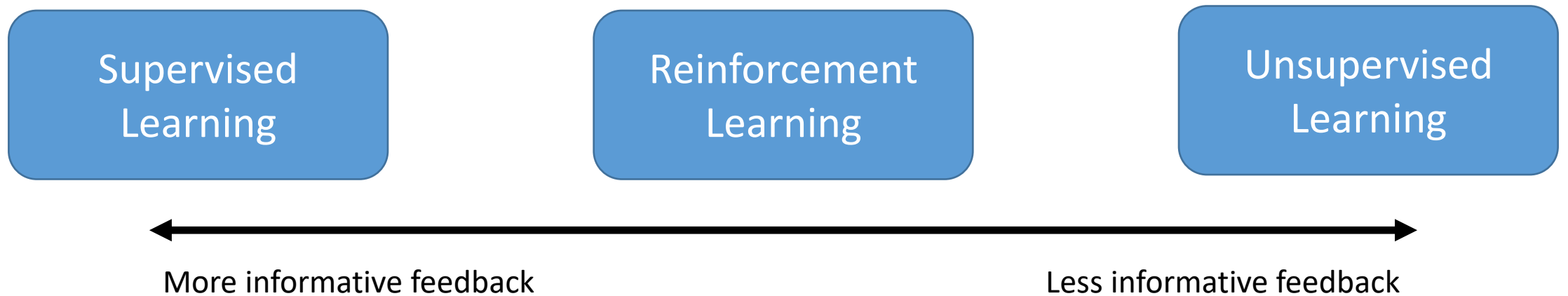
Gaussian Processes

Unsupervised Learning

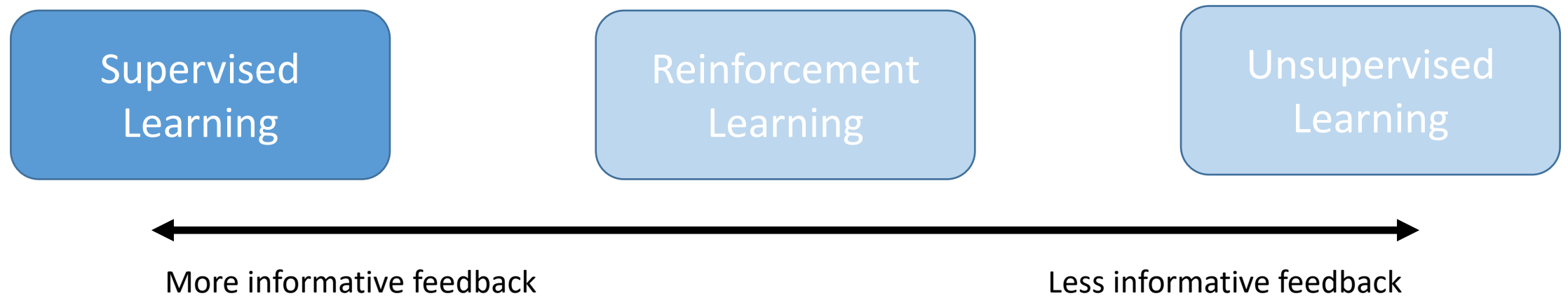
Reinforcement Learning

[1] Koza, John R.; Bennett, Forrest H.; Andre, David; Keane, Martin A. (1996). *Automated Design of Both the Topology and Sizing of Analog Electrical Circuits Using Genetic Programming*. Artificial Intelligence in Design '96. Springer, Dordrecht. pp. 151–170

Supervised, Reinforcement and Unsupervised Learning



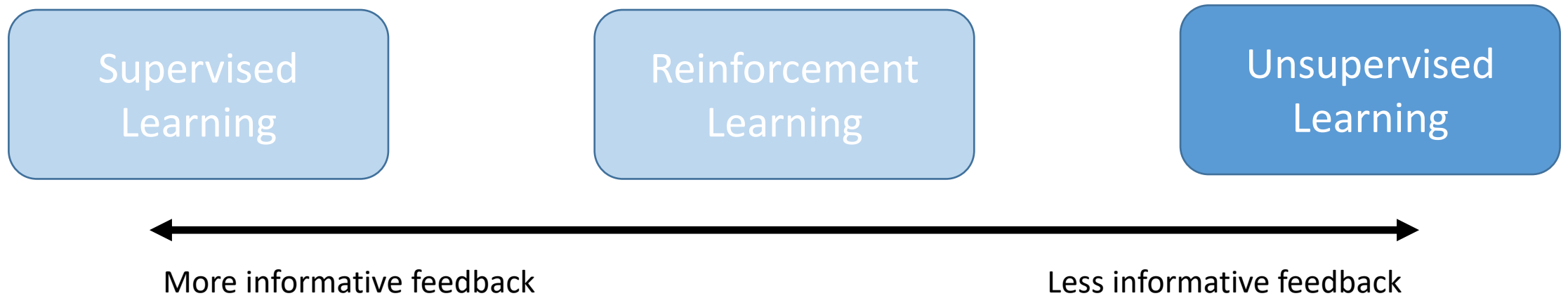
Supervised, Reinforcement and Unsupervised Learning



Inputs and outputs are known
Infer input-output relationship

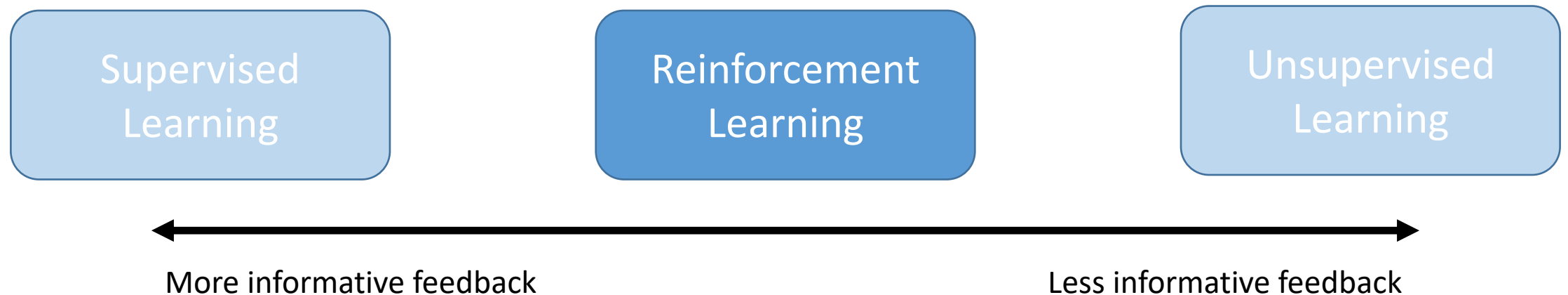
e.g. function estimation

Supervised, Reinforcement and Unsupervised Learning



Only the inputs are known
Find patterns and features from data
e.g. clustering

Supervised, Reinforcement and Unsupervised Learning



Correct outputs not available, only rewards
Find optimal policy / behavior / controller

What is Machine Learning?

How can computers learn to solve problems without being explicitly programmed? In other words, how can computers be made to do what needs to be done, without being told exactly how to do it?¹

Objective

Model / Framework

Learning Type

Policy Learning

Neural Networks

Supervised Learning

Prediction / Simulation

Gaussian Processes

Unsupervised Learning

Reinforcement Learning

[1] Koza, John R.; Bennett, Forrest H.; Andre, David; Keane, Martin A. (1996). *Automated Design of Both the Topology and Sizing of Analog Electrical Circuits Using Genetic Programming*. Artificial Intelligence in Design '96. Springer, Dordrecht. pp. 151–170

What is Machine Learning?

How can computers learn to solve problems without being explicitly programmed? In other words, how can computers be made to do what needs to be done, without being told exactly how to do it?¹

Objective

Policy Learning

Prediction / Simulation

Model / Framework

Neural Networks

Gaussian Processes

Learning Type

Supervised Learning

Reinforcement Learning

[1] Koza, John R.; Bennett, Forrest H.; Andre, David; Keane, Martin A. (1996). *Automated Design of Both the Topology and Sizing of Analog Electrical Circuits Using Genetic Programming*. Artificial Intelligence in Design '96. Springer, Dordrecht. pp. 151–170

Why Machine Learning?

Powerful Modelling Frameworks

Nonparametric Methods

Little User Interaction

Machine Learning vs Systems and Control

Many things in common but ...

Classical Systems and Control

Dynamic Systems (Engineering)

Theoretical Guarantees

Mainly Parametric Approaches

High User Interaction

Noise & Uncertainties

System Analysis

Machine Learning

Static & Dynamic Systems (CS)

Limited Theoretical Guarantees

Nonparametric Approaches

Low User Interaction

Large-Scale Problems

Example: AlphaStar



Source: <https://deepmind.com/blog/alphastar-mastering-real-time-strategy-game-starcraft-ii/>

Example: AlphaStar

Goal: Beat Starcraft II
Pro-Gamers



Source: <https://deepmind.com/blog/alphastar-mastering-real-time-strategy-game-starcraft-ii/>

Example: AlphaStar

Goal: Beat Starcraft II
Pro-Gamers

Challenges:

- High Decision Space
- Long-Term Dynamics
- Imperfect Information

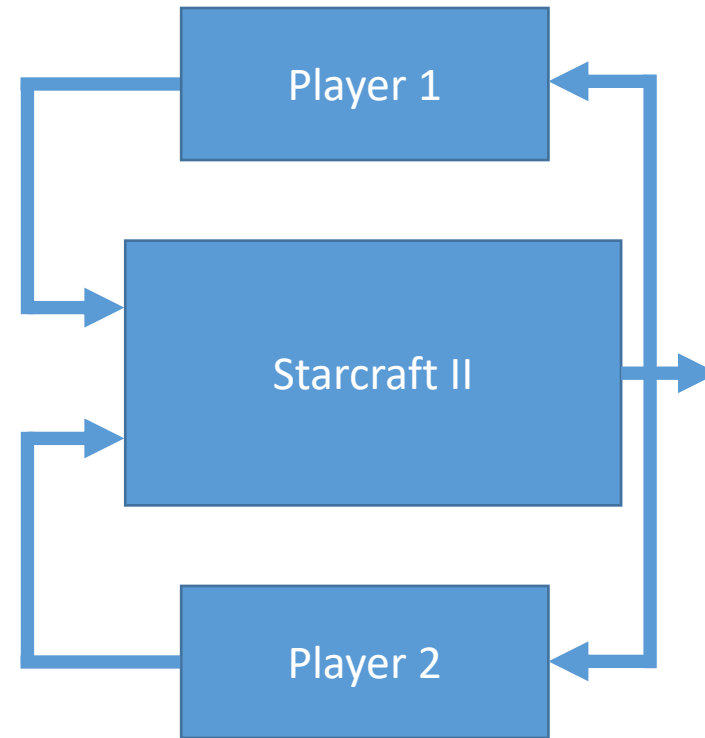


Source: <https://deepmind.com/blog/alphastar-mastering-real-time-strategy-game-starcraft-ii/>

Example: AlphaStar

Goal: Beat Starcraft II
Pro-Gamers

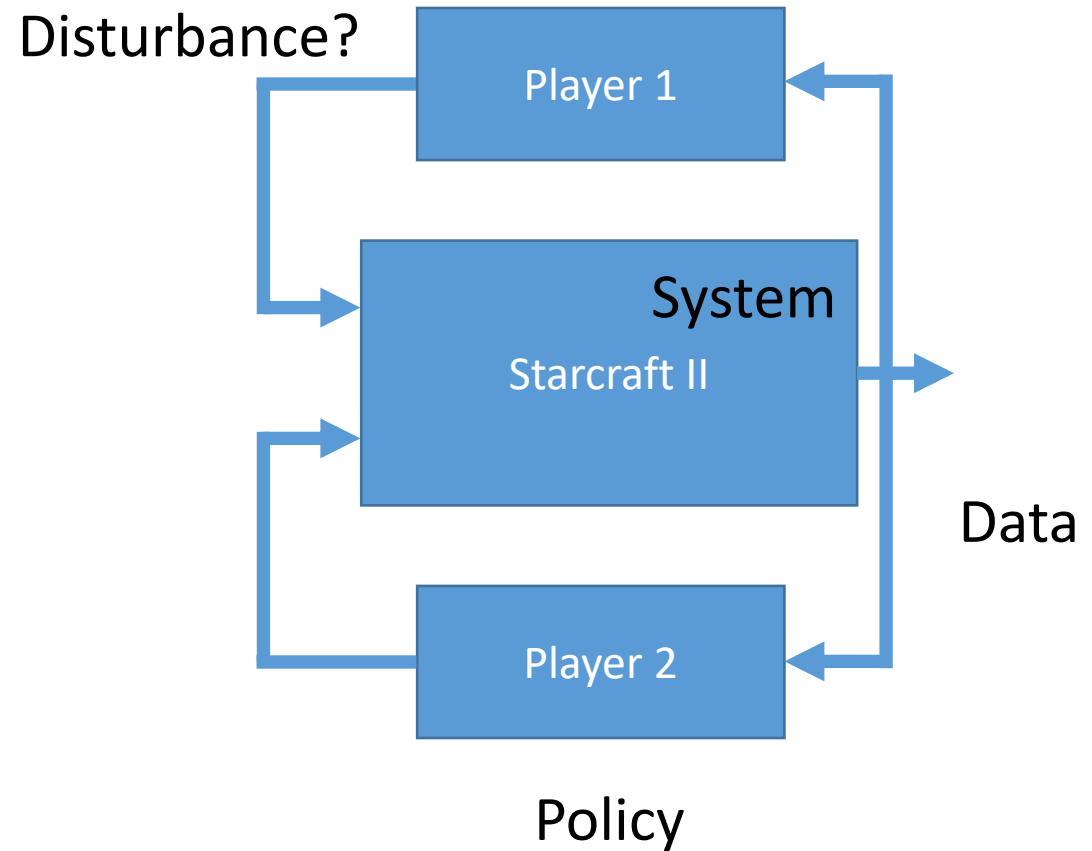
How: Learn a Policy
aka 'controller'



Example: AlphaStar

Goal: Beat Starcraft II
Pro-Gamers

How: Learn a Policy
aka 'controller'



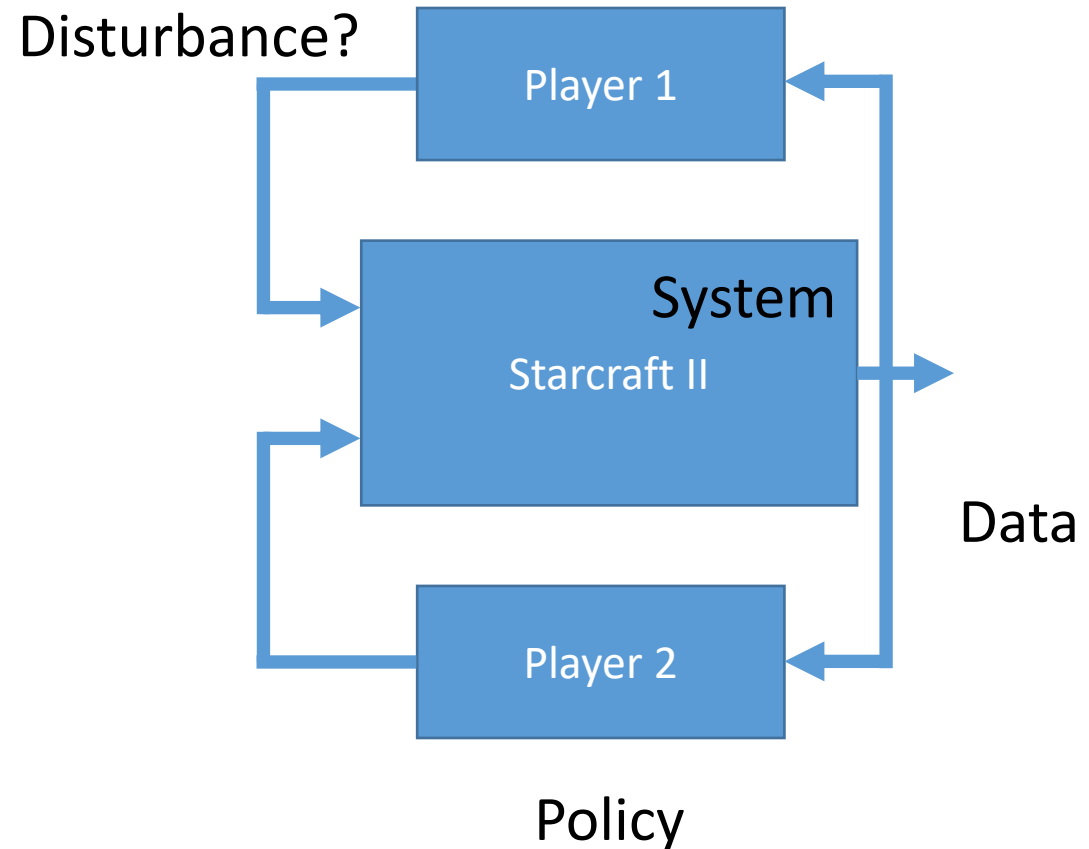
Example: AlphaStar

Goal: Beat Starcraft II
Pro-Gamers

How: Learn a Policy
aka 'controller'

Framework: Neural Net

Method:
Supervised / Reinforcement
Learning



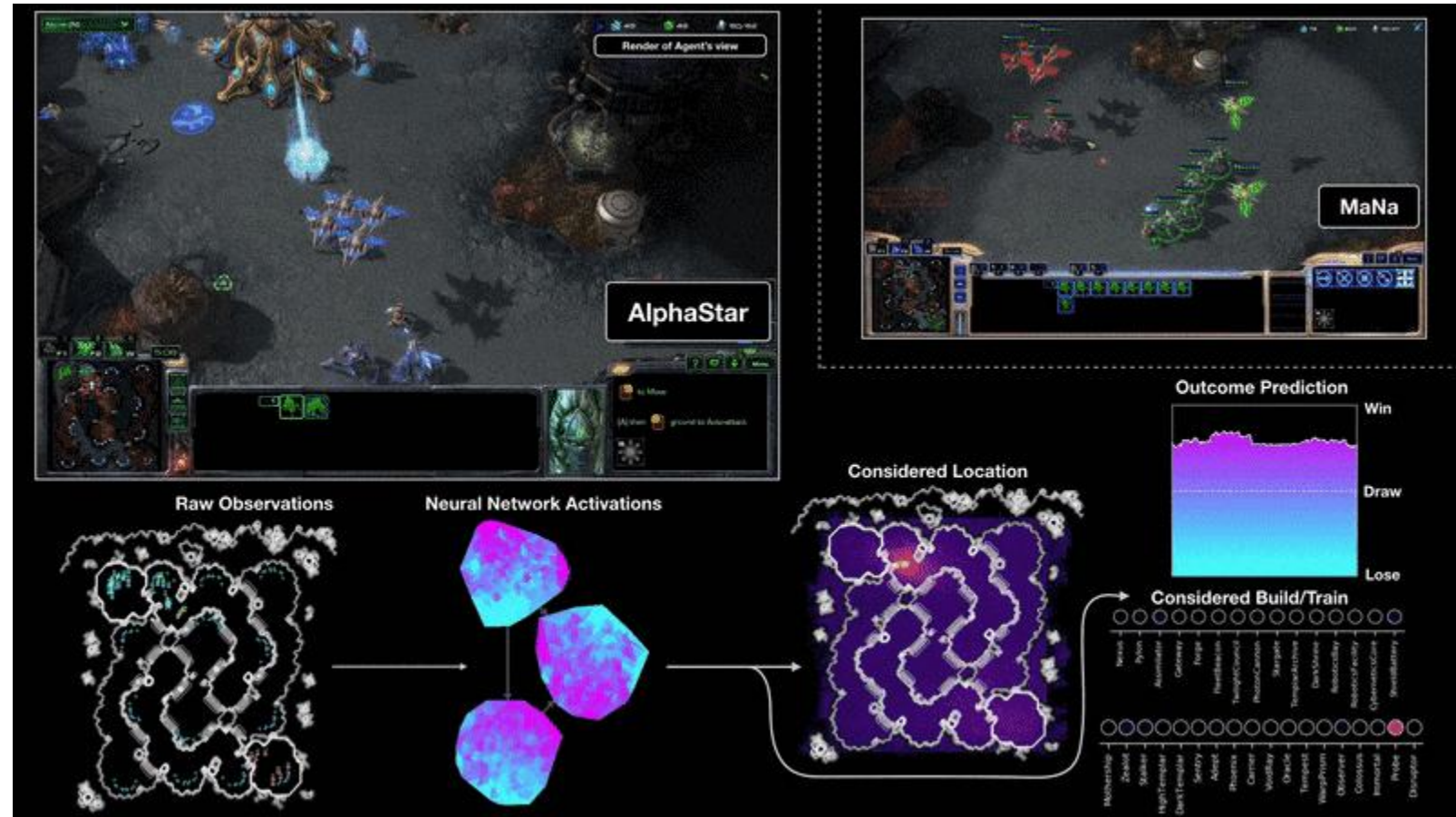
Example: AlphaStar

Goal: Beat Starcraft II Pro-Gamers

How: Learn a Policy aka 'controller'

Framework: Neural Net

Method:
Supervised / Reinforcement
Learning



Source: <https://deepmind.com/blog/alphastar-mastering-real-time-strategy-game-starcraft-ii/>

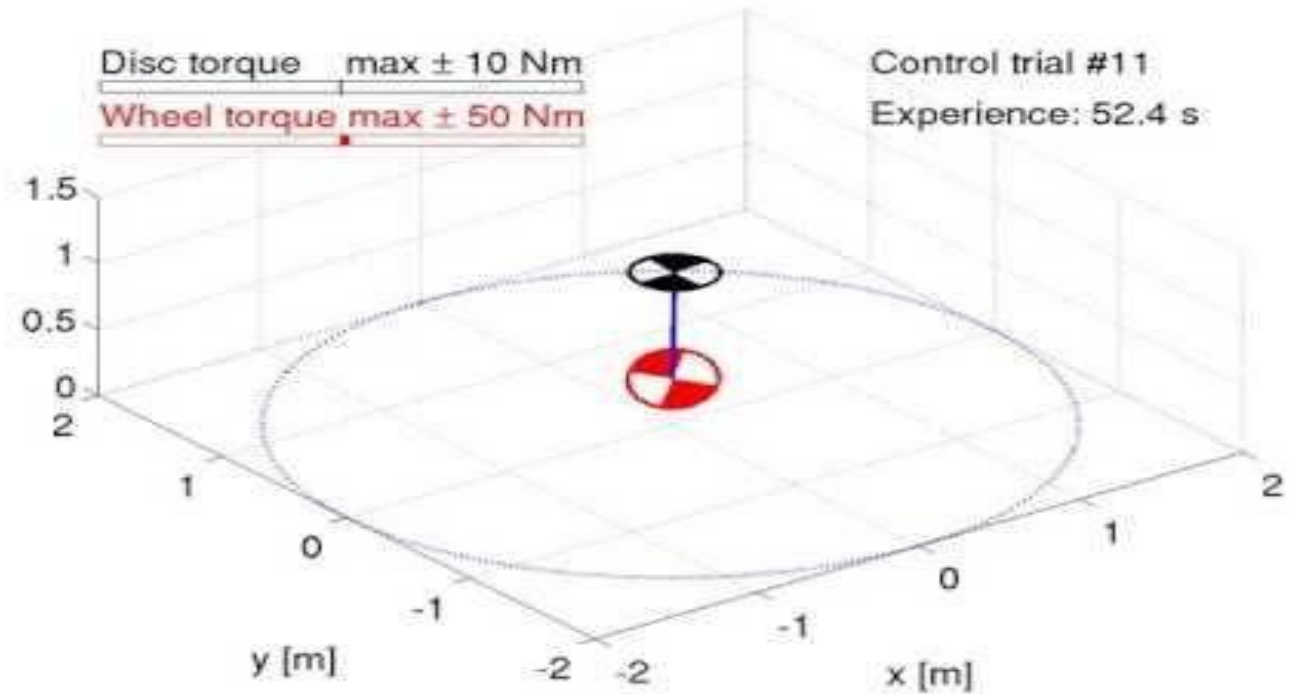
Other Examples

Cart-Pole Double-Pendulum Swing-Up

Learning to Unicycle

Throttle Valve Control

Robot Manipulator Control



Source: <http://mlg.eng.cam.ac.uk/pilco/>

Course Content

Data-driven modelling: neural networks and deep learning
 regularization, Gaussian processes

Data-driven control: basics of reinforcement learning
 actor-critic methods
 model internalization methods

Course Content

Data-driven modelling: neural networks and deep learning
 regularization, Gaussian processes

Data-driven control: basics of reinforcement learning
 actor-critic methods
 model internalization methods

Introductory course: provide and overview of the key concepts
 practical user experience

Course Information

Lectures: 9 x 2hrs

Exercises: 9 x 2hrs (Q&A) + preparation at home

Project: ~50 hrs

Exact schedule on Canvas (study guide)

Check study guide! (canvas)

Lectures on Monday and Wednesdays

Q&A after the lectures

Slots reserved for design assignment
No TA activity

Design assignment deadline
25/06/2023

Presentation + Defense
25/06-03/07

Resit
13/08-14/08

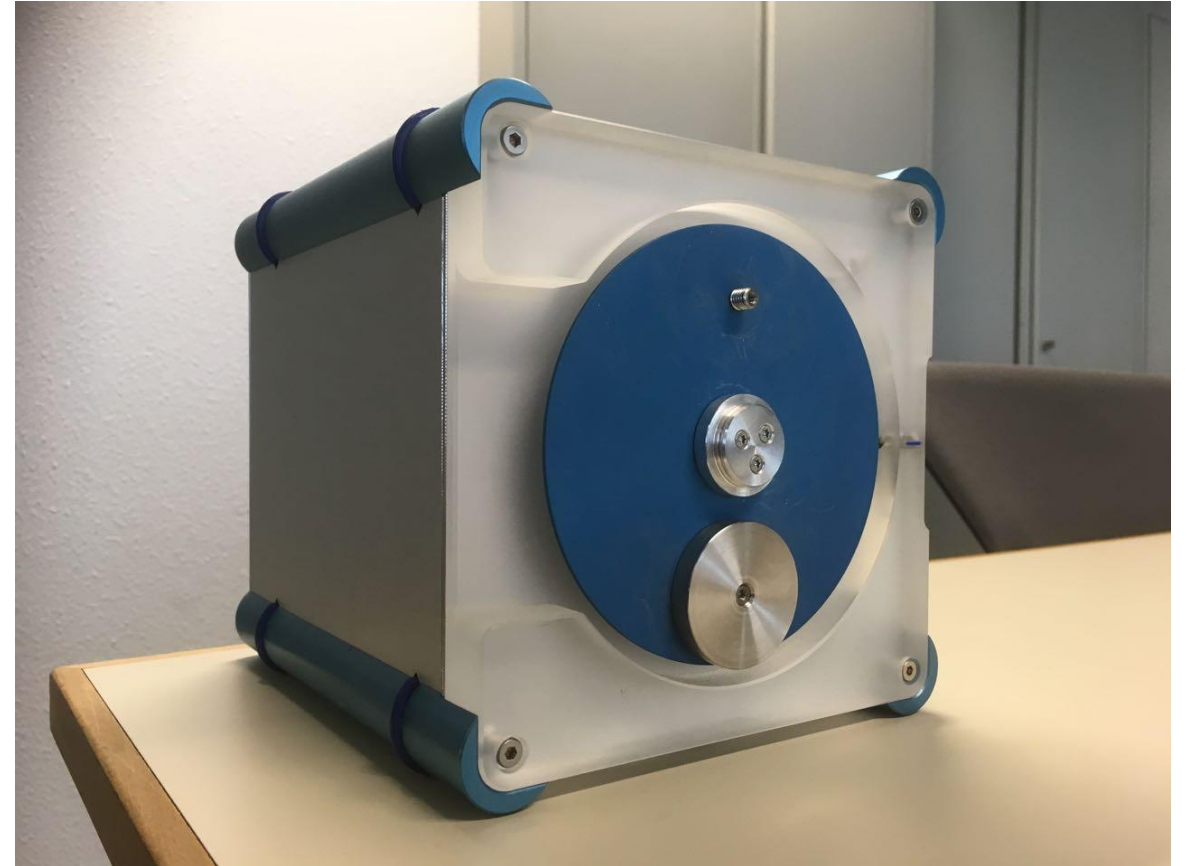
Week	Date	Timeslot 1 (2hrs)	Timeslot 2 (2hrs)
17	Mon 20/04	L1: Intro & Preliminaries	Python + Exercise Introduction
17	Wen 22/04	L2: ANN	Exercise Introduction + Design Assignment Q&A
18	Mon 27/04		
18	Wen 29/04	L3: Deep ANN	Exercise Introduction + Design Assignment Q&A
19	Mon 04/05		
19	Wen 06/05	L4: Gaussian Processes Fundamentals (R)	Exercise Introduction + Design Assignment Q&A
20	Mon 11/05	L5: Gaussian Processes Applications (R)	Exercise Introduction + Design Assignment Q&A
20	Wen 13/05	L6: Basics of Reinforcement Learning	Seminar + Exercises + Design As. Q&A
21	Mon 18/05	Reserved for Design Assignment	Reserved for Design Assignment
21	Wen 20/05	L7: RL with Function Approximation	Exercise Introduction + Design Assignment Q&A
22	Mon 25/05		
22	Wen 27/05	L8: Actor-Critic RL	Seminar + Exercises + Design As. Q&A
23	Mon 01/06	L9: Model Internalization RL (R)	Exercise Introduction + Design Assignment Q&A
23	Wen 03/06	Reserved for Design Assignment	Exercise + Design Assignment Q&A
24	Mon 08/06	Reserved for Design Assignment	Reserved for Design Assignment
24	Wen 10/06	Reserved for Design Assignment	Exercise + Design Assignment Q&A
25	Mon 15/06	Reserved for Design Assignment	Reserved for Design Assignment
25	Wen 17/06	Reserved for Design Assignment	Exercise + Design Assignment Q&A
26	Mon 22/06 - 23:59	Hand in Design Assignment report	
26-27	25/06 - 03/07	Oral defense period	(if needed due to student number, days can be added)
33	13/08 - 14/08	Resit oral defense	

Design Project

Reinforcement Learning for an Inverted Pendulum

1. Identify pendulum dynamics using GP *and* ANN
2. Obtain a single policy to swing up the pendulum *and* track a reference

Tools: Reinforcement Learning
Gaussian Processes
Neural Networks



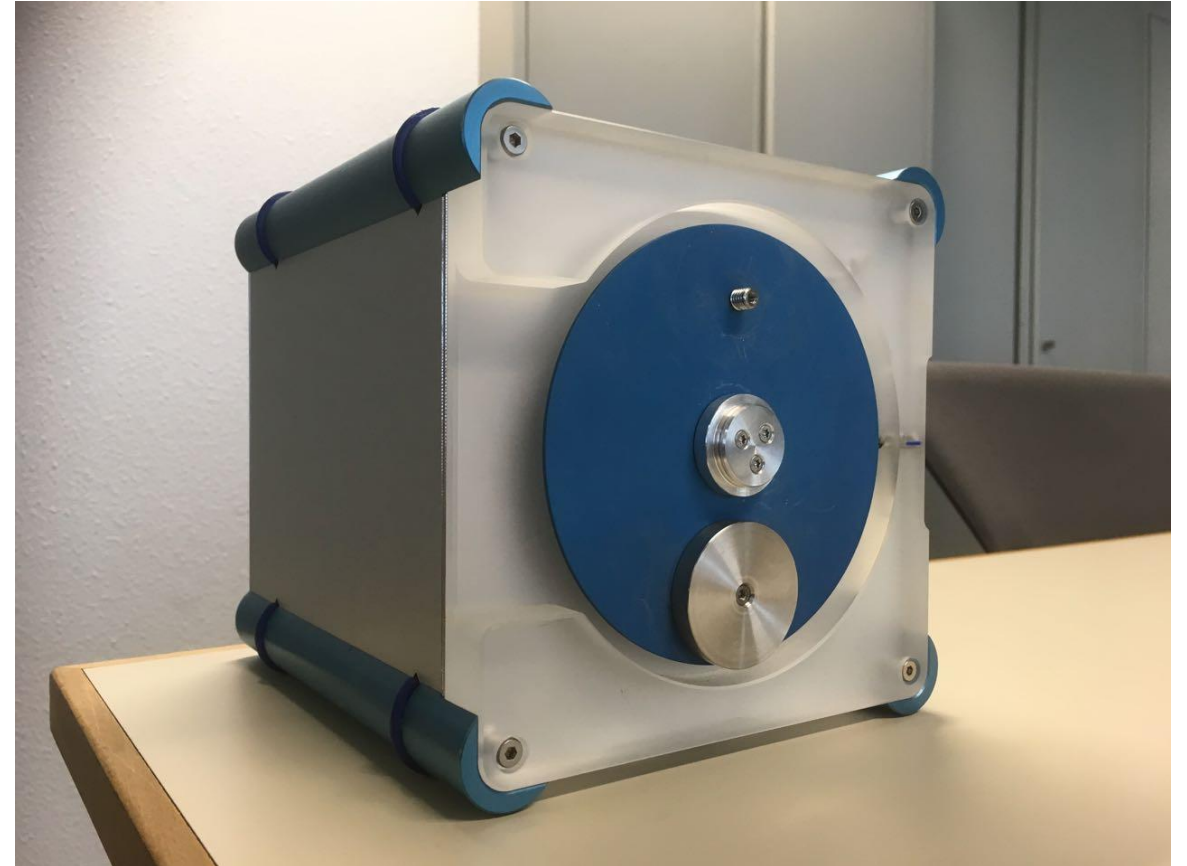
Design Project

Groups of 4 students

Registration for the groups through
Canvas

6-12 pages IEEE style written report.

Oral defense (25 June – 03 July 2025)



Grading

Oral defense of the Design Project + theoretical knowledge:

- a. 6-12 pages IEEE style project report (per group) + 1-page reflection reports (template will be provided)
- b. 10 min group presentation + 45-60 min discussion of the obtained result + course theory

motivated + systematic way of obtaining the solutions
the applied and general course knowledge
correctness of the final solution
clarity of the final report and presentation

Grading

Design Assignment Obtained Results:	$OR \in [0, 10]$
Design Assignment Design Process:	$DP \in [0, 10]$
Theory Discussion:	$ThD \in [0, 10]$
Relative Individual Contribution:	$RIC \in [-1, 1]$

Final Grade:

$$FP = 0.33 \times OR + 0.33 \times DP + 0.33 \times ThD + RIC$$

if $FP \geq 6$ then

$\text{Min}(\text{Round}(FP), 10)$

else

$\text{Min}(\text{Round}(FP), 5)$

Suggested Literature

Gaussian Processes

C.E. Rasmussen and C.K.I. Williams, *Gaussian Processes for Machine Learning*, the MIT Press, 2006. (<http://www.gaussianprocess.org/gpml/>)

Neural Networks

I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, the MIT Press, 2016. (<https://www.deeplearningbook.org/>)

Reinforcement Learning

R.S. Sutton and A.G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018. (<http://incompleteideas.net/book/the-book-2nd.html>)

Continued Development of the Course

We appreciate feedback!

Ask questions! (lectures, exercise Q&A, canvas discussions)

After the course: send remarks

Questionnaire at the end



Poll: Student Background

A: Systems & Control / EE - Control Systems

B: Artificial Intelligence Engineering Systems

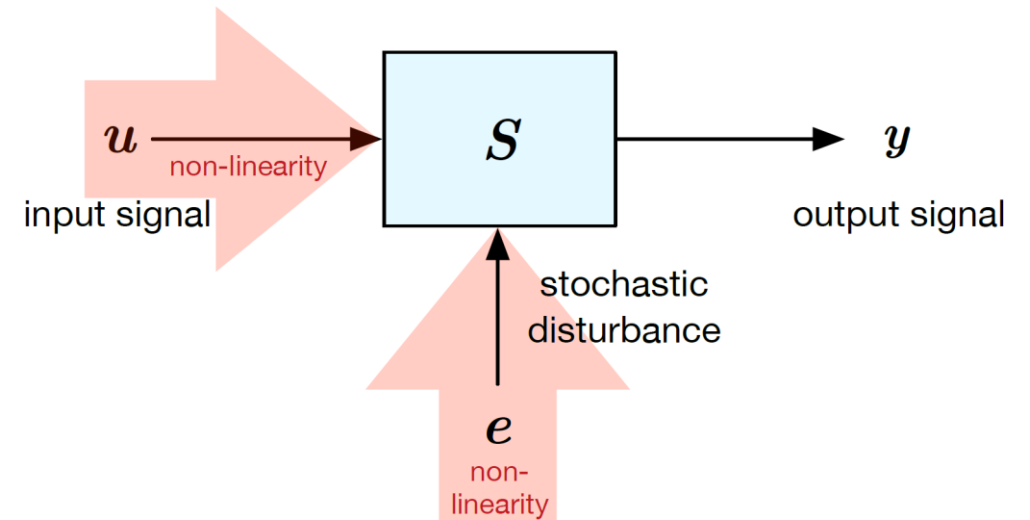
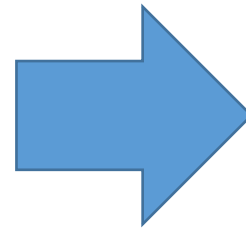
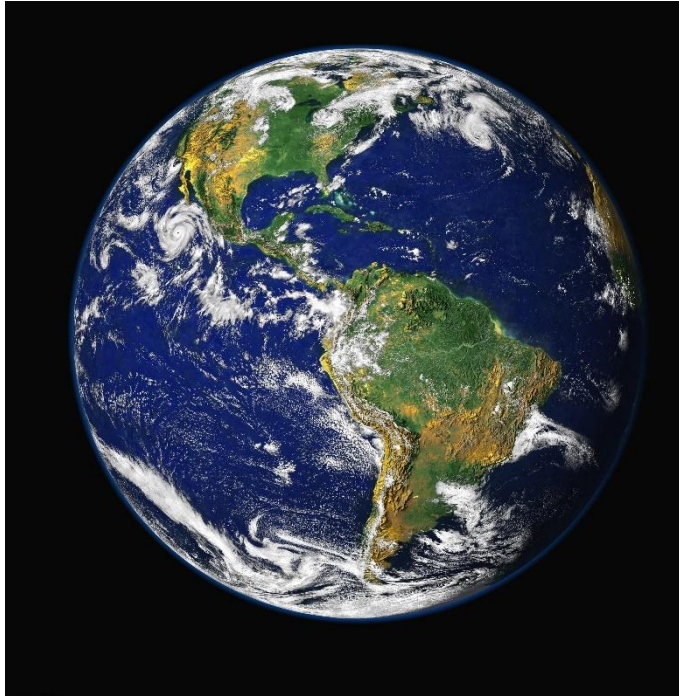
C: Electrical Engineering (not S&C / AIES)

D: Mechanical Engineering (not S&C / AIES)

E: Applied Physics

F: Other

From Data to Model



Learning Objectives

Why Data-Driven Modelling?

What Model Structures Are Used?

Estimation Framework

Contents

Data-Driven Modelling Motivation

Data-Generating Systems / Models of Dynamical Systems

Estimators

Data-Driven Modelling Process

Contents

Data-Driven Modelling Motivation

- Models in Engineering

- Data-Driven Modelling

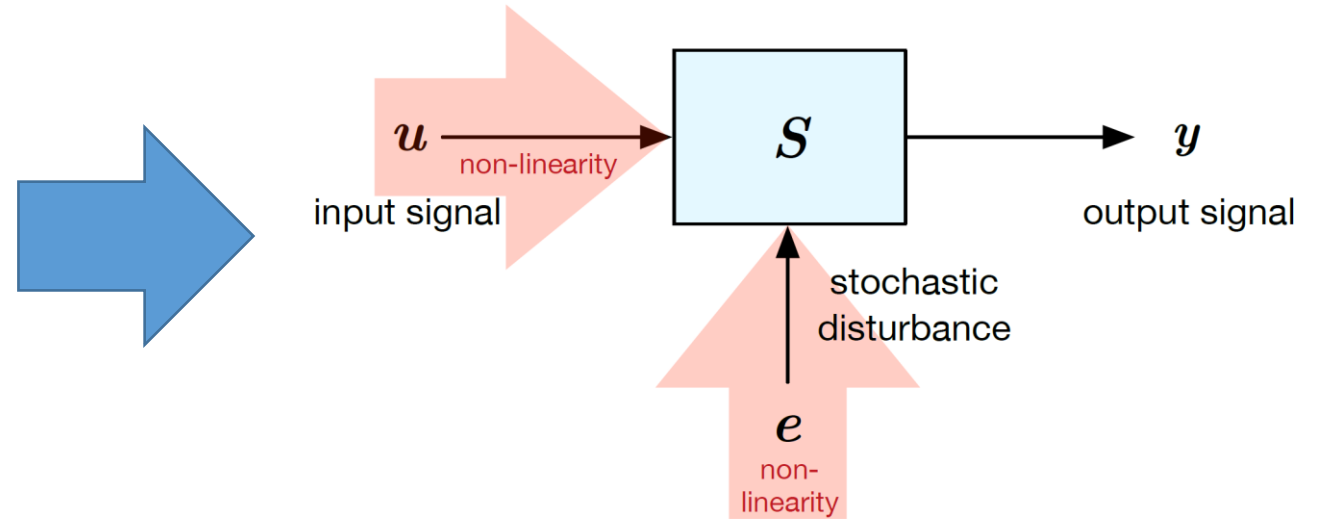
- Motivating Example

Data-Generating Systems / Models of Dynamical Systems

Estimators

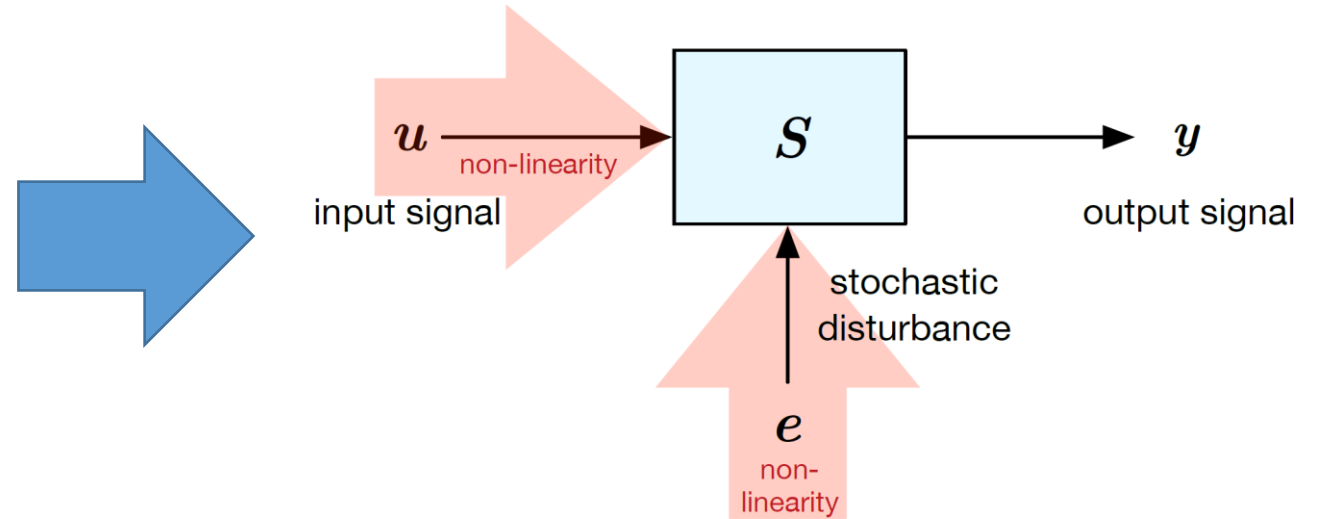
Data-Driven Modelling Process

Models



A model is an abstraction (approximation/simplification) of a part of reality, which is adequate for a given utilization purpose

Models



system (re)design, control design / optimization, prediction, simulation,
diagnosis / fault detection, monitoring / soft sensing

Models in Engineering

Models are Needed Everywhere

Essential to Understanding

Essential for Design Process

Abstraction of Knowledge / Information

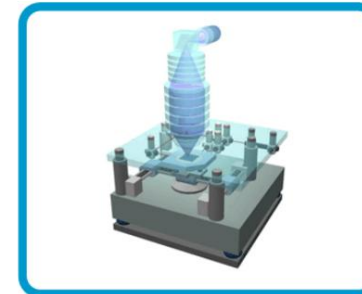
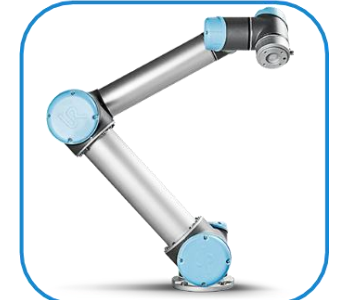
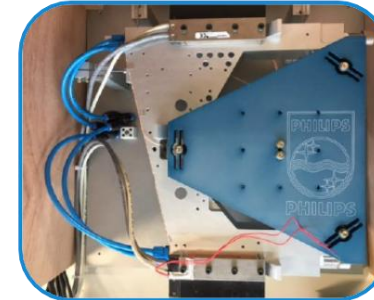
Utilization Perspective is Important

Dynamic vs Static Models

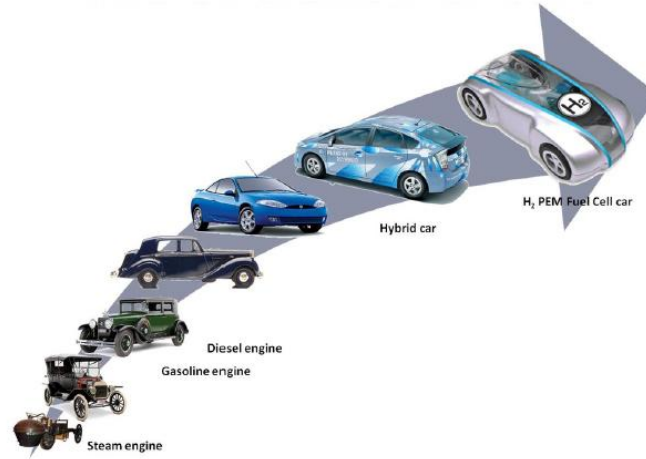
Temporal Aspects of the Dynamics

Deterministic vs Stochastic

Prediction / Simulation / Control



Growing Challenges



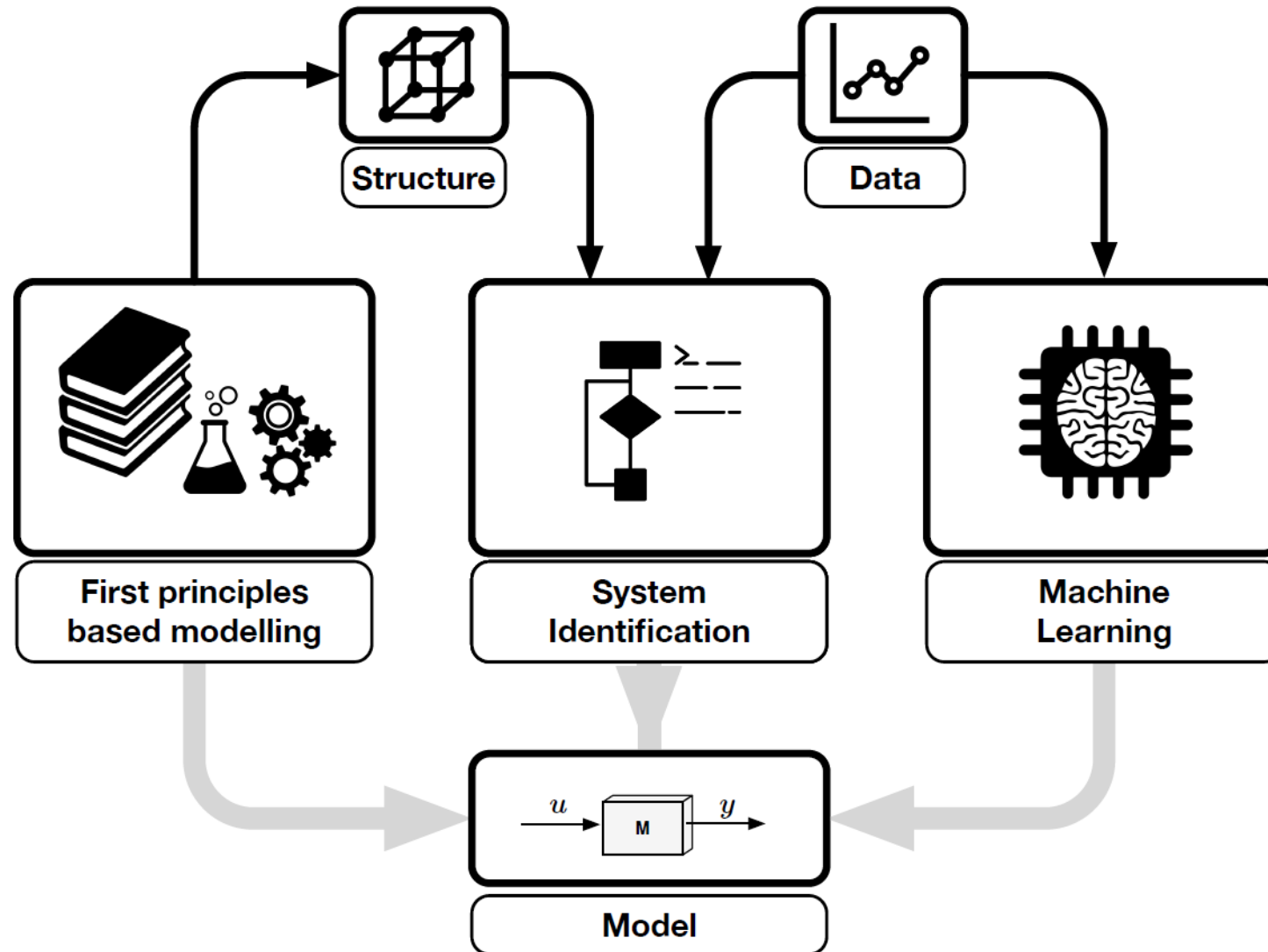
Complexity

Performance

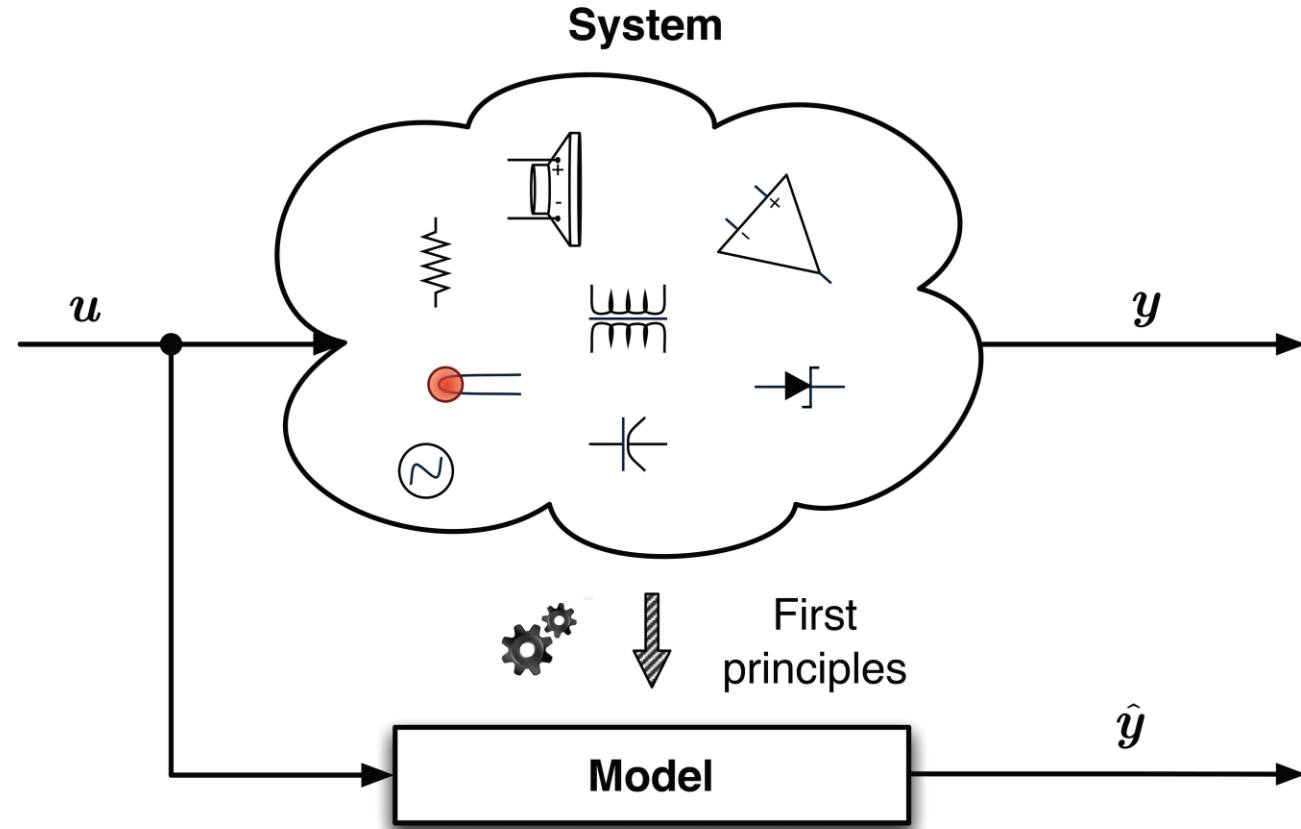
Interconnectivity



The Process



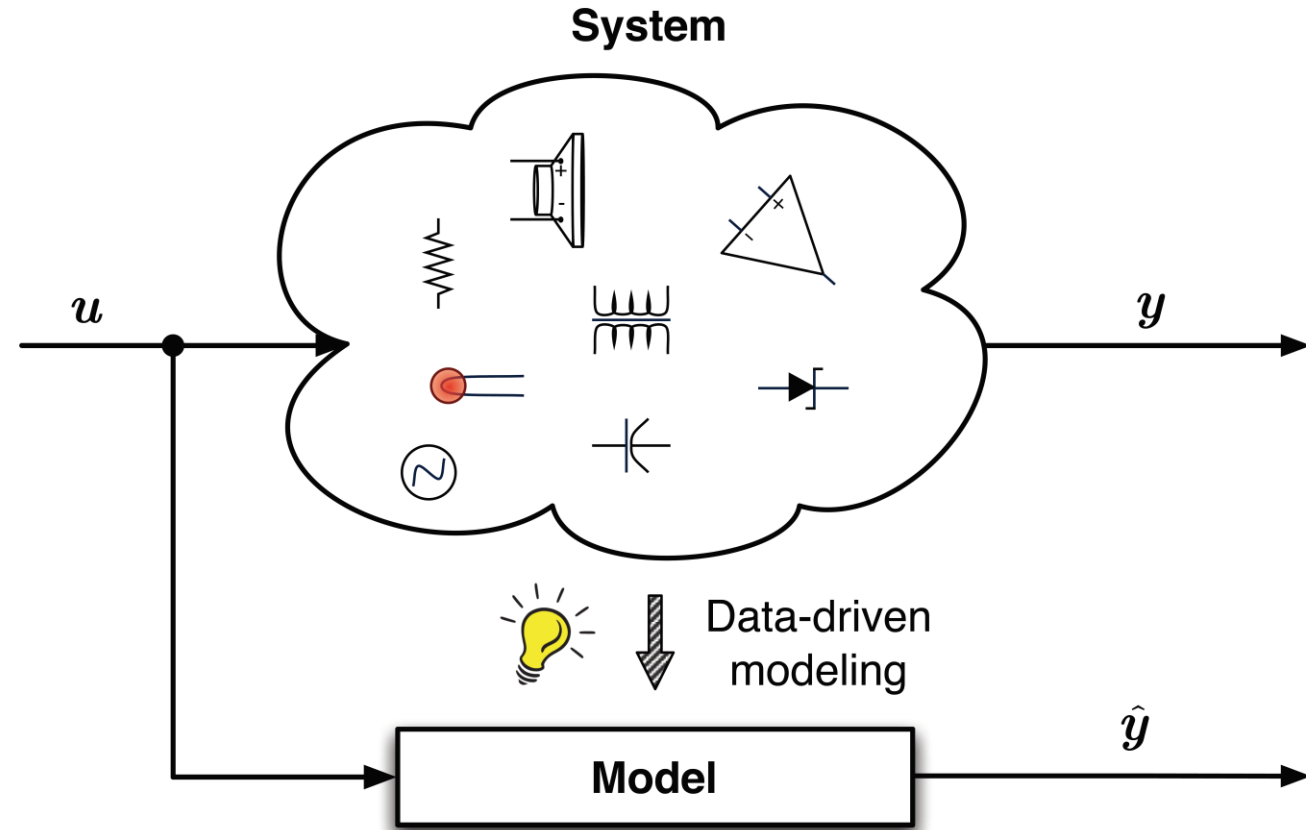
White-Box



First principles-based modeling (white box):

- a priori knowledge and first principles laws
- physical interpretation
- high complexity and cost, but strong reliability

Data-Driven



Data-driven modeling (identification, black & grey box):

From measured data to model

Exploration of the dynamical structure: loss of physical interpretation

Low complexity and cost: model what you observe

Data-Driven Methods in Systems & Control

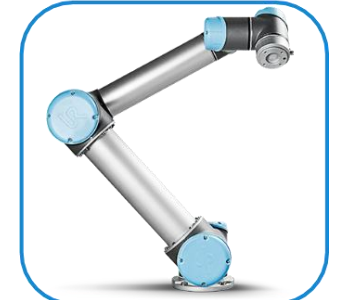
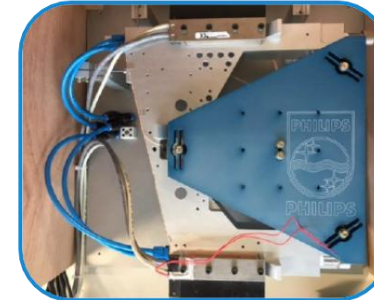
Data-Driven Modelling

Capturing Unknown Dynamics
Model Verification

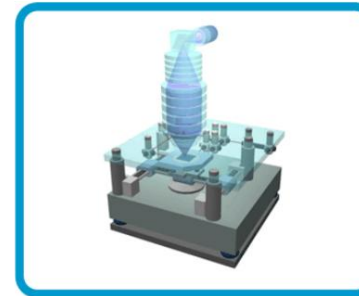


Model Maintenance and Adaptation

Extracting Control Relevant Behavior



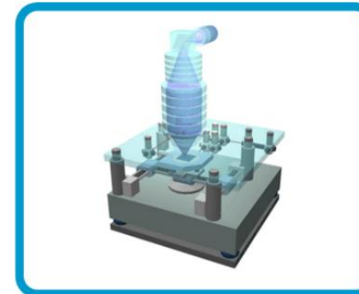
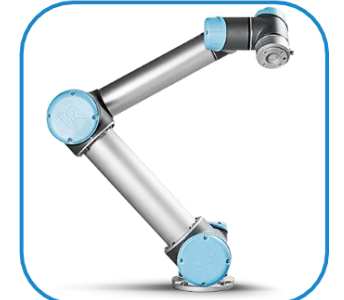
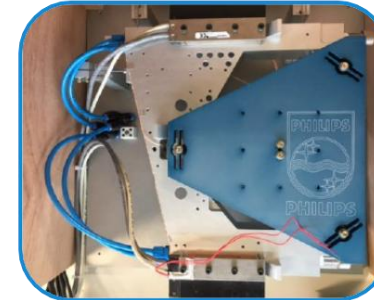
Model Free Control Design



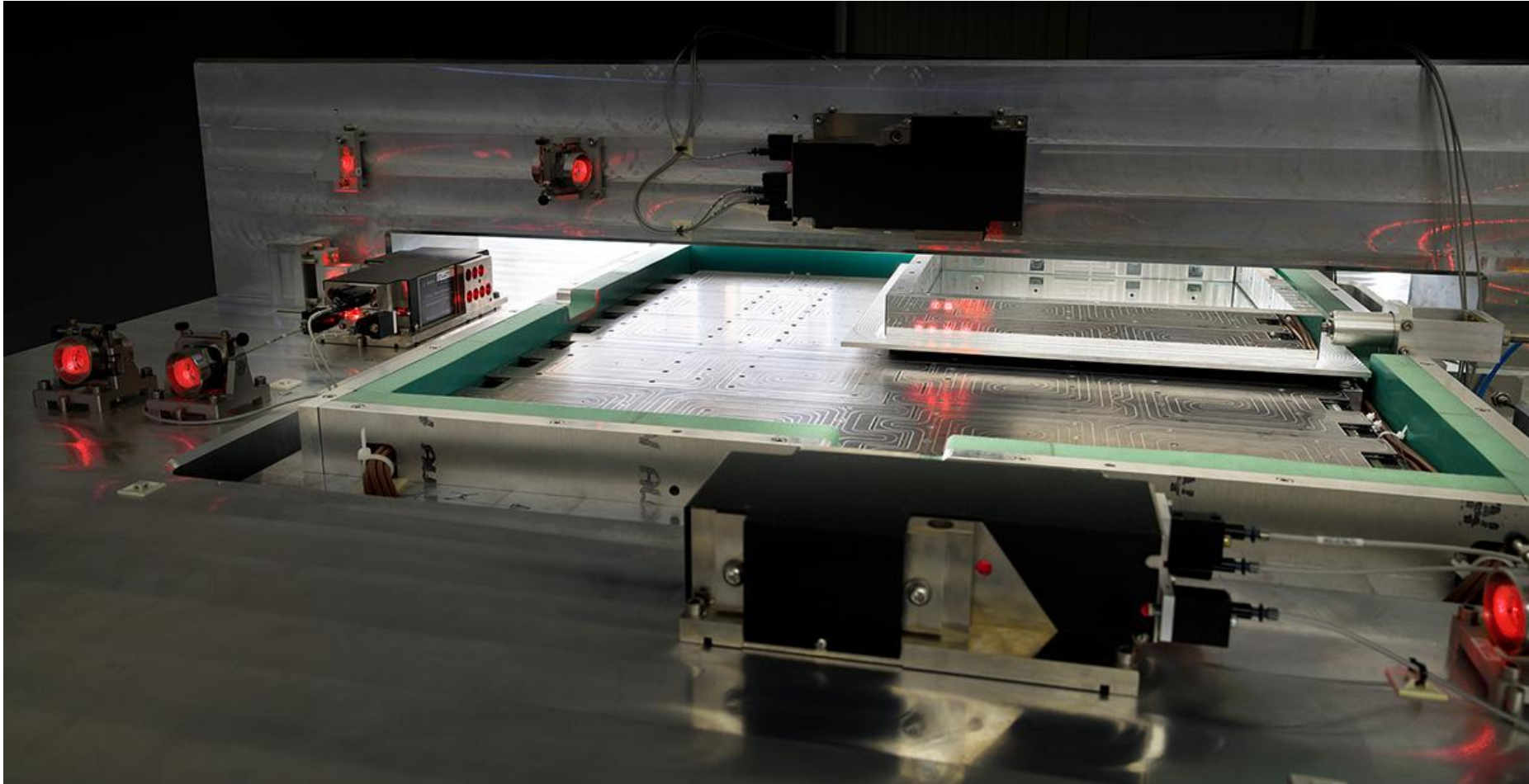
Data-Driven Methods in Systems & Control

How to scale with the surge of complexity & performance needs?

Can machine learning offer the same guarantees as the classical frameworks?

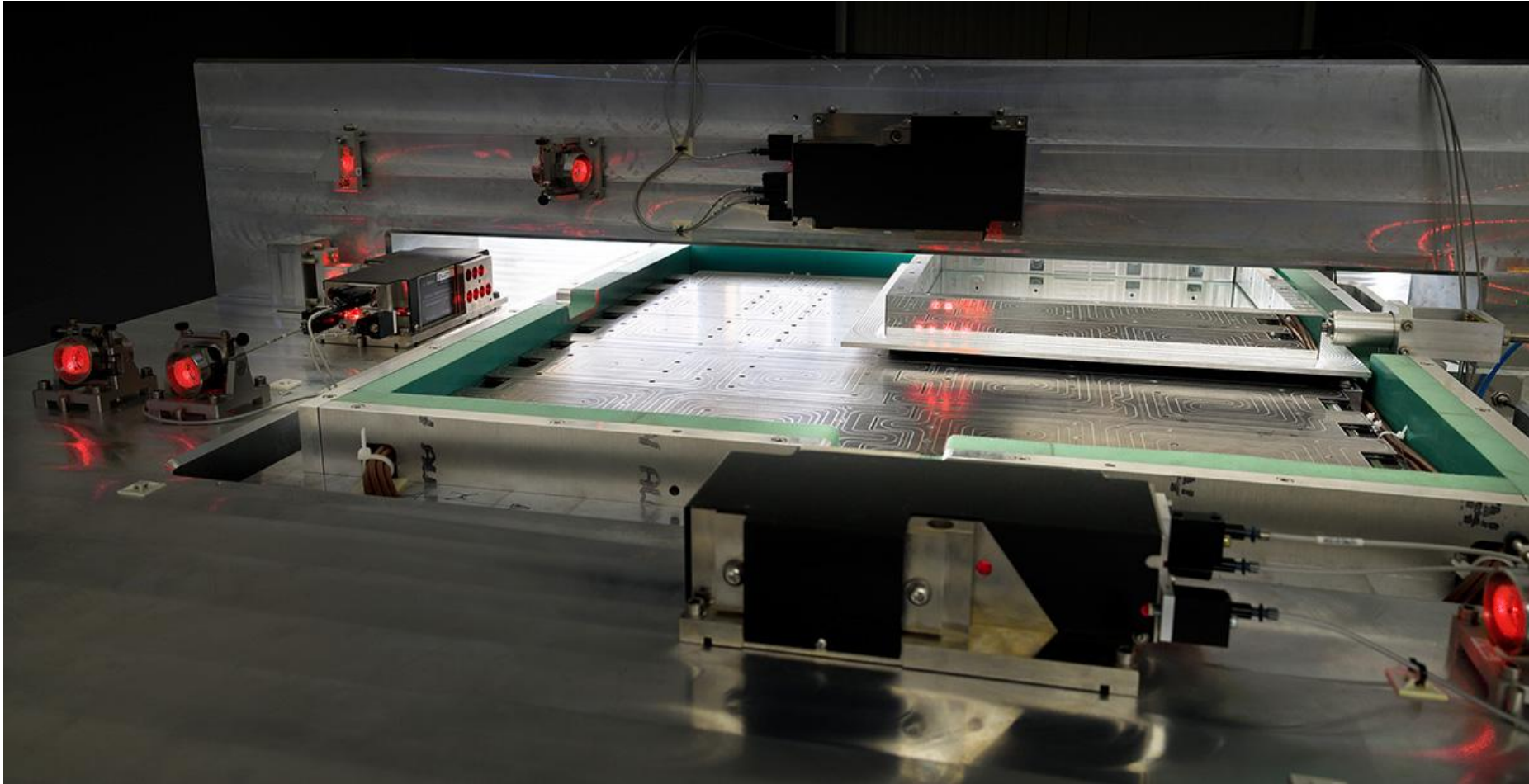


6-DOF High-Precision Motion System



Developed by EPE and CS

6-DOF High-Precision Motion System



Achieve nm-accurate motion of a magnetically levitated stage with high-velocity and acceleration

6-DOF High-Precision Motion System



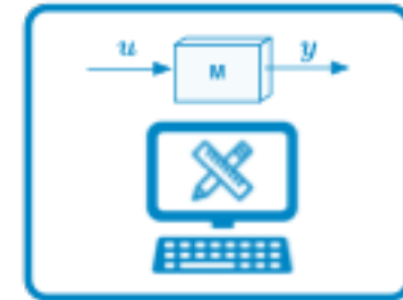
**First principles
based modelling**

High-fidelity ME & EM model
(Fourier series + FEM)



**System
Identification**

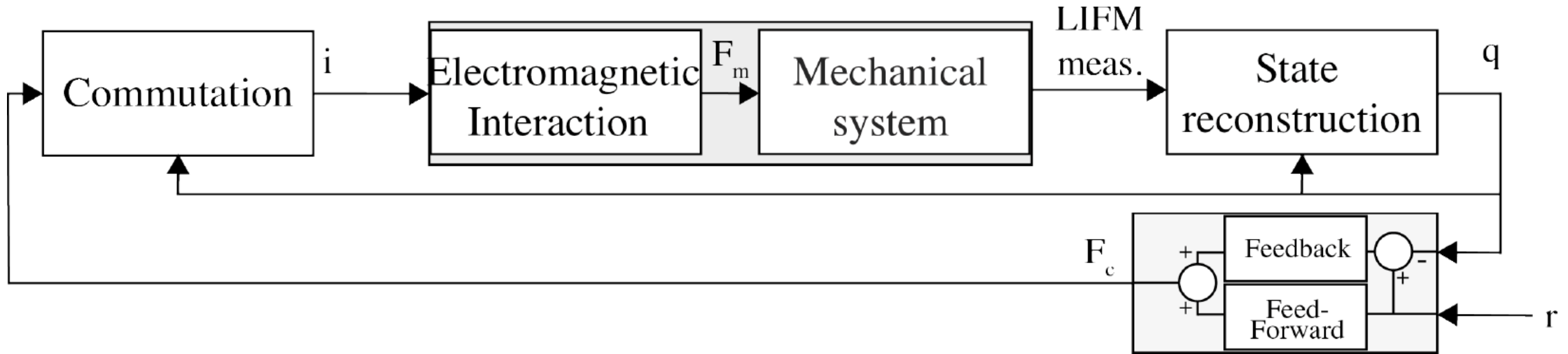
System identification
(FRF modelling)



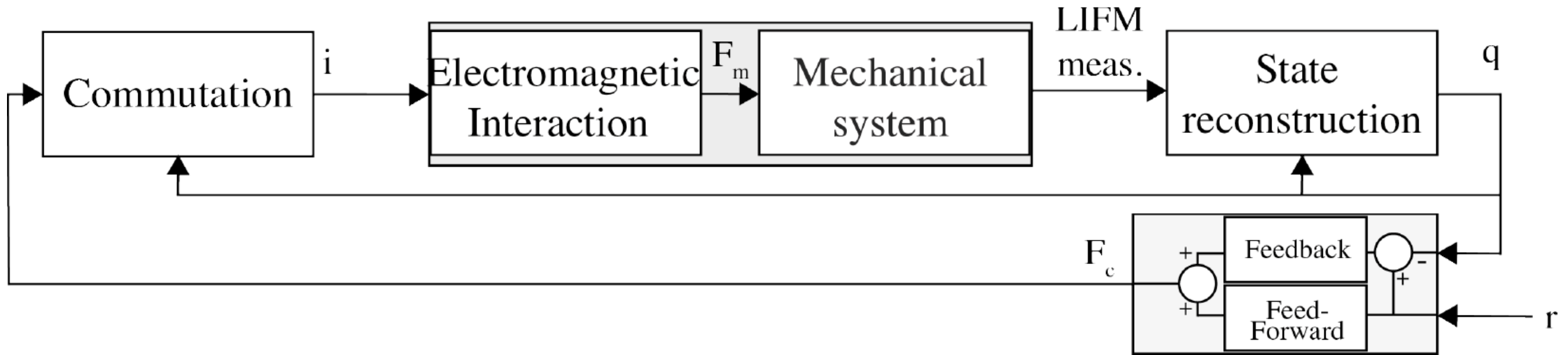
**Controller
Design**

Sophisticated
control design

6-DOF High-Precision Motion System



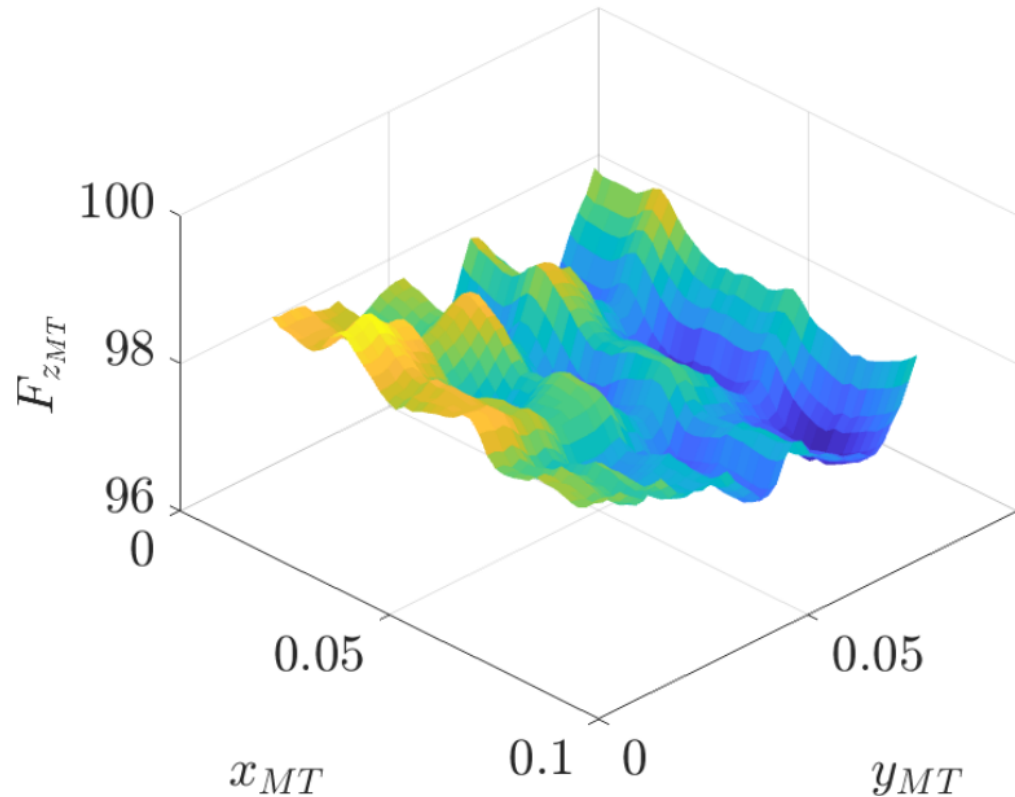
6-DOF High-Precision Motion System



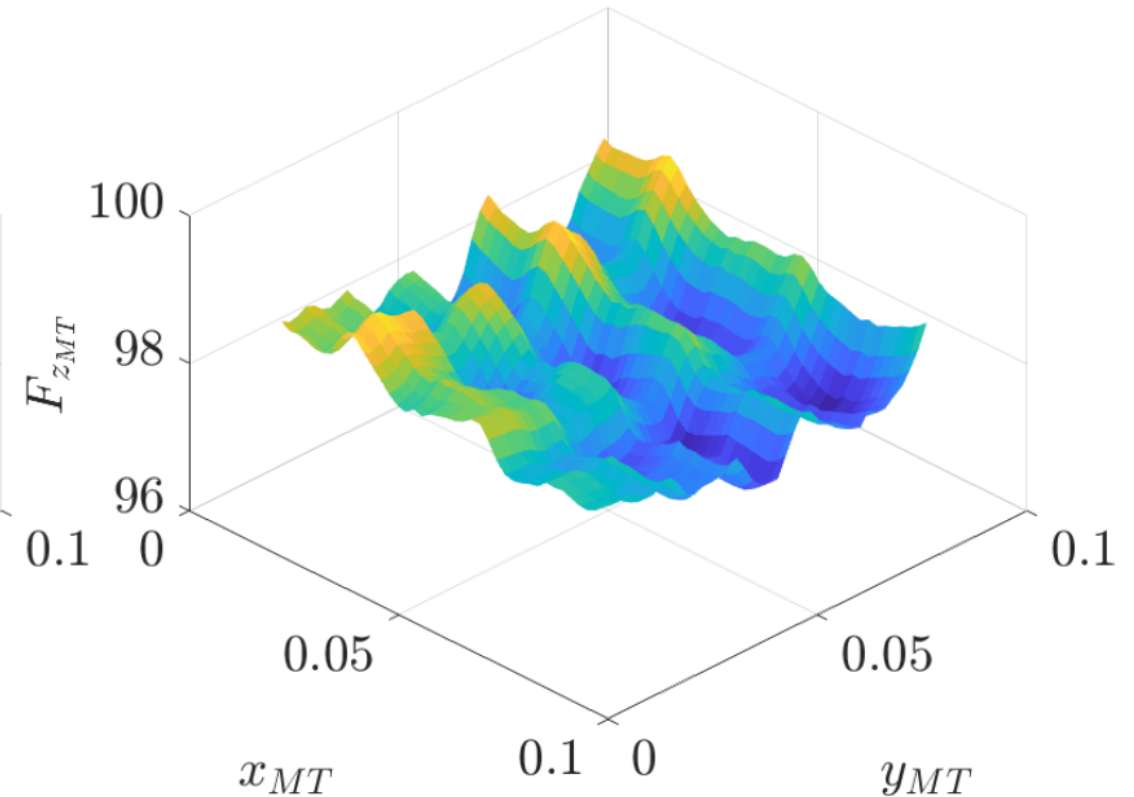
Unforeseen disturbance effects

6-DOF High-Precision Motion System

Experiment 1



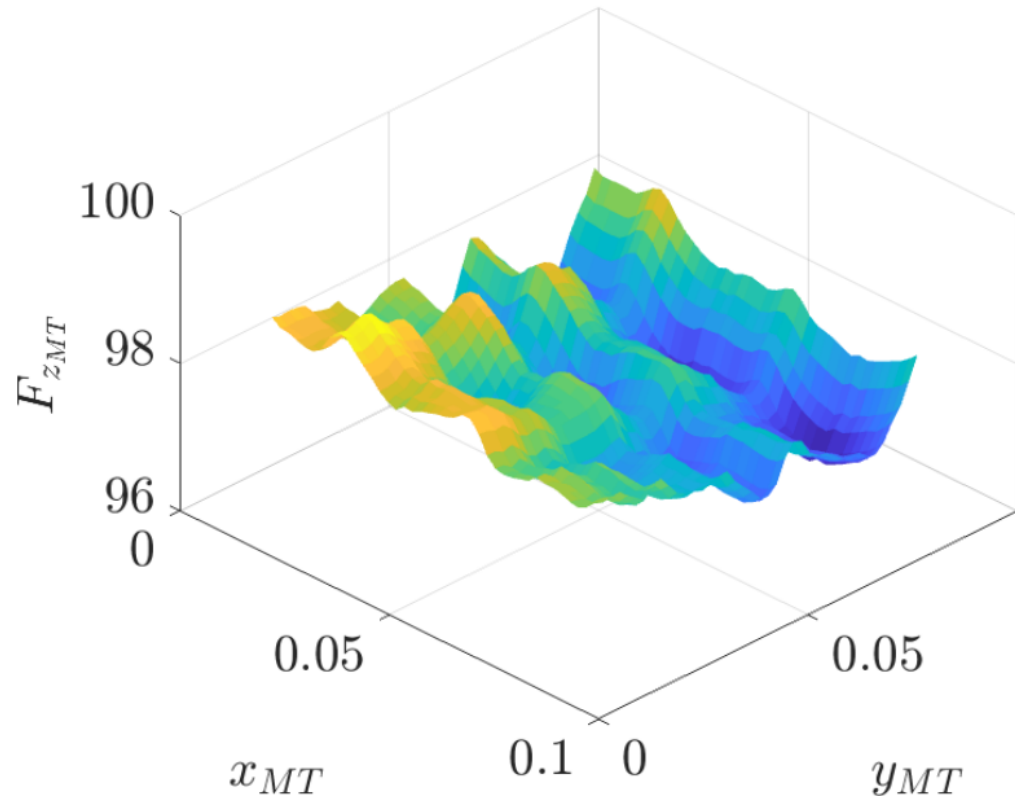
Experiment 2



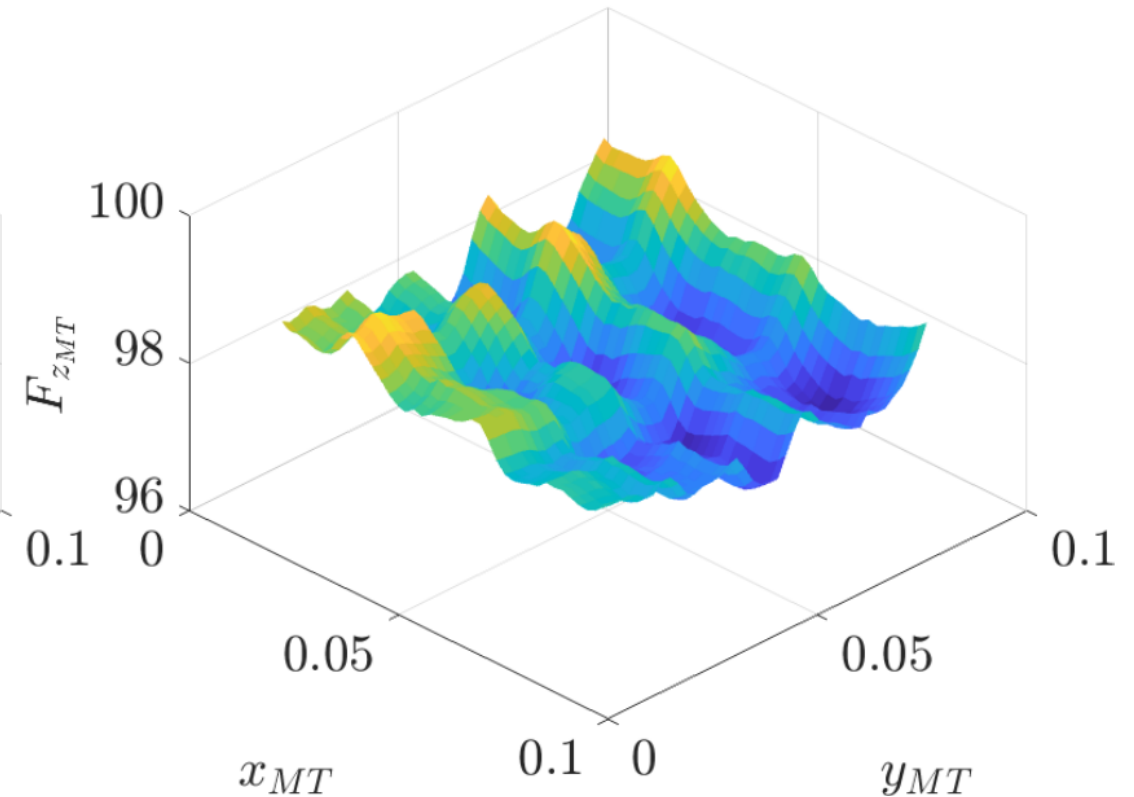
Reasons: Coil Pitch, Metrology frame / Stator Alignment, Manufacturing errors, ...

6-DOF High-Precision Motion System

Experiment 1



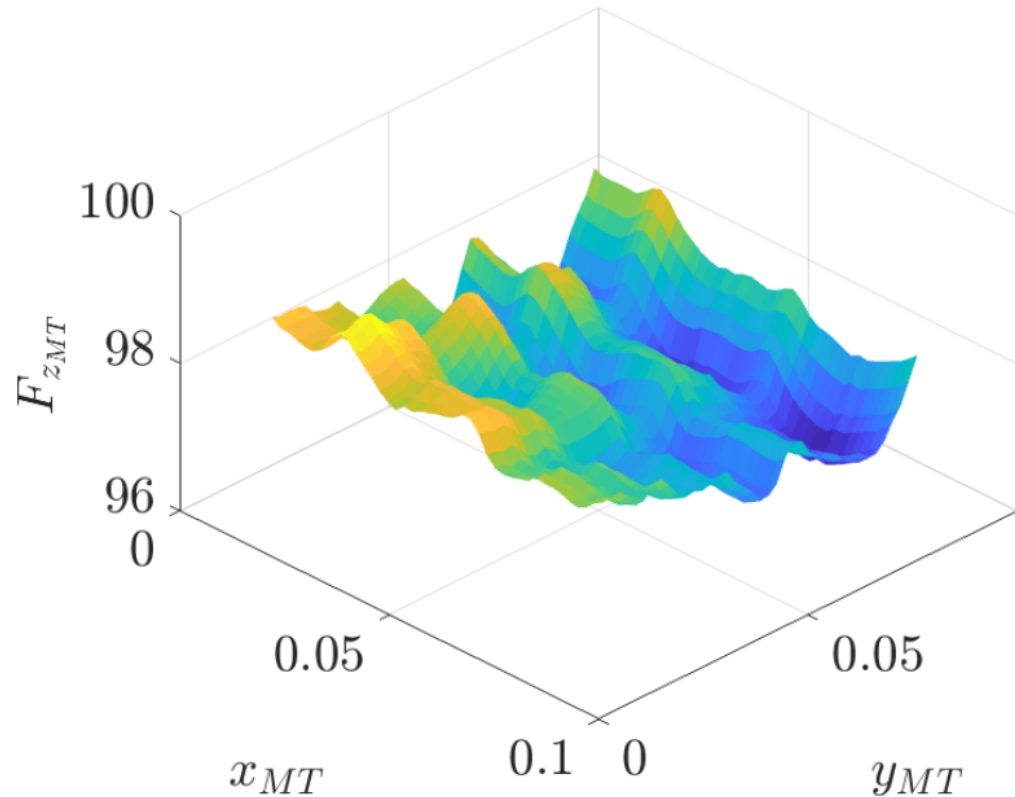
Experiment 2



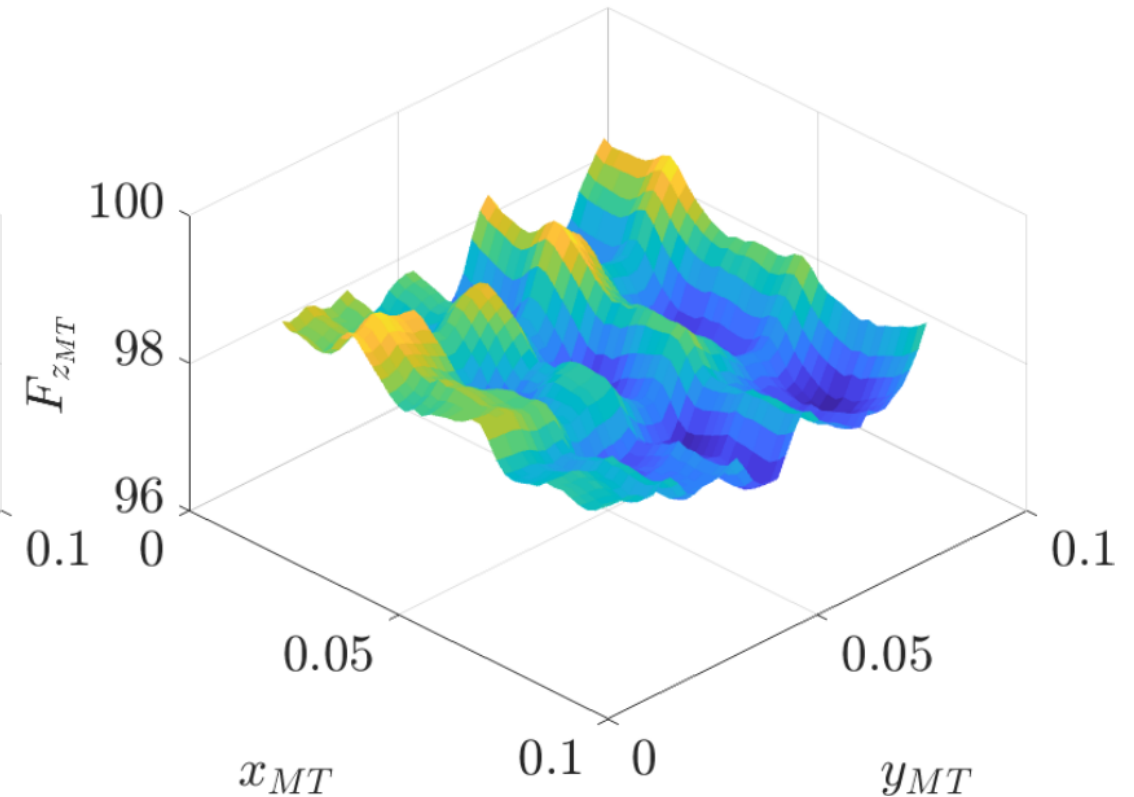
Question: How can we learn this offset for feed-forward compensation?

6-DOF High-Precision Motion System

Experiment 1



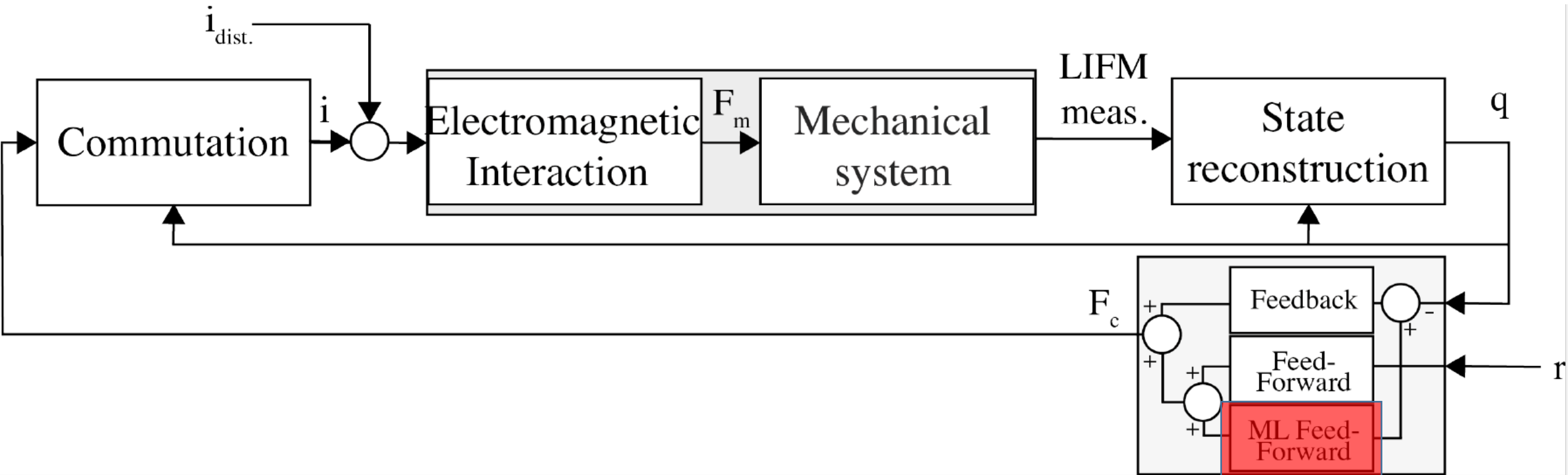
Experiment 2



Question: How can we learn this offset for feed-forward compensation?

Machine Learning

6-DOF High-Precision Motion System

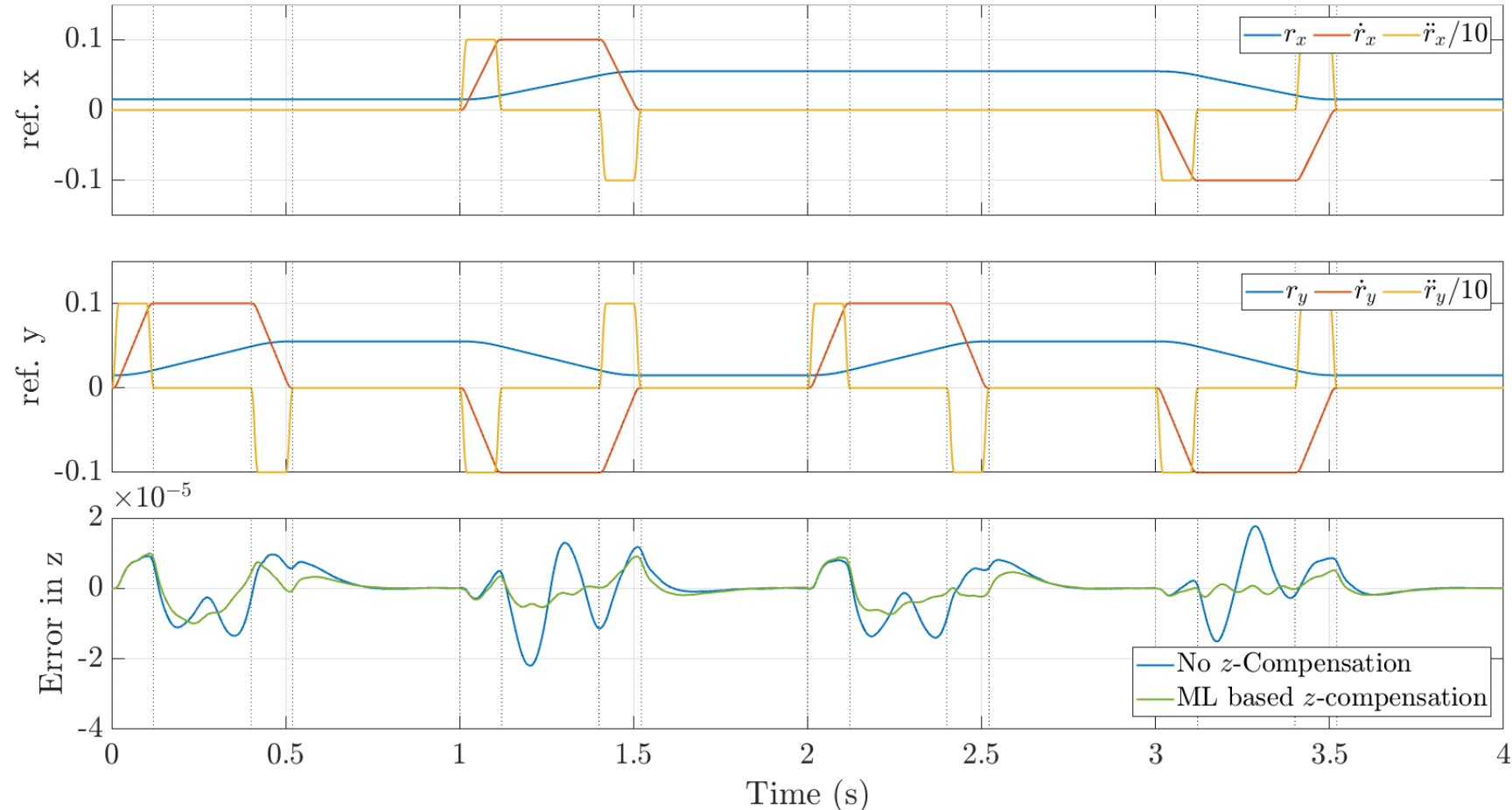


Question: How can we learn this offset for feed-forward compensation?

Machine Learning

6-DOF High-Precision Motion System

Measurement results with GP-based offset compensation (low-bandwidth controller)



Drastic error reduction!

Contents

Data-Driven Modelling Motivation

Data-Generating Systems / Models of Dynamical Systems

- Linear Time-Invariant

- Linear Parameter Varying

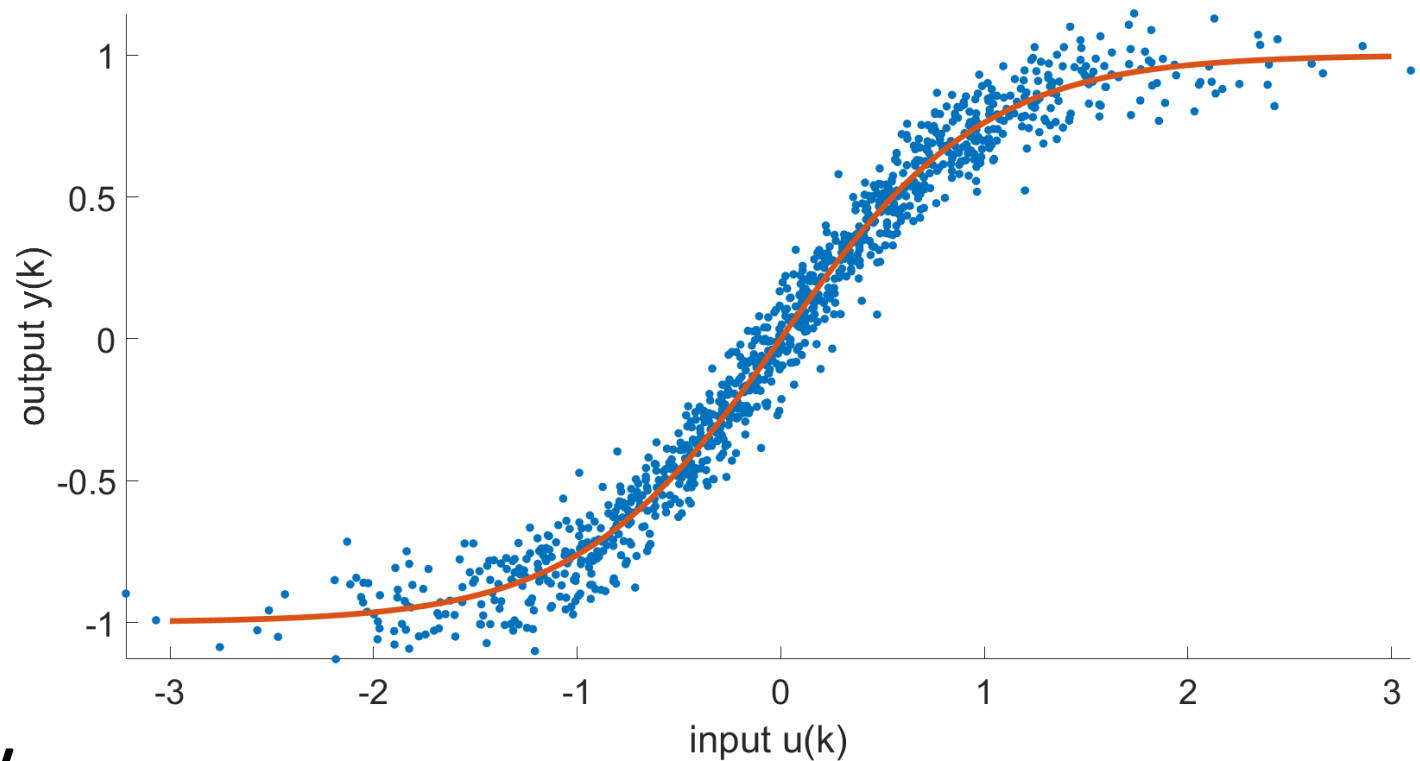
- Nonlinear

Estimators

Data-Driven Modelling Process

Static vs. Dynamic

$$y_k = f(u_k) + e_k$$



Static Nonlinearity

Output only depends on the instantaneous input(s)

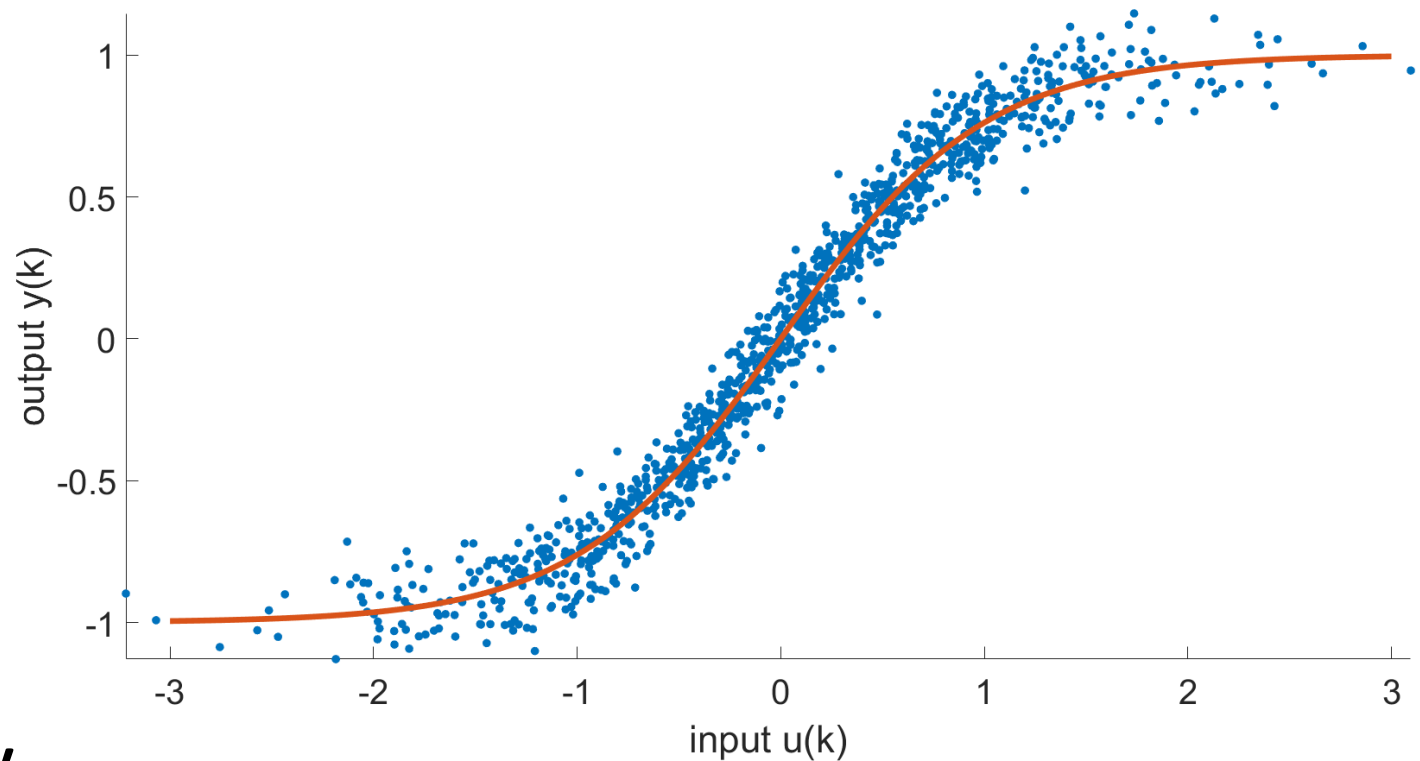
Easy to analyze (for low-dimensional inputs)

Easy to estimate

Static vs. Dynamic

$$y_k = f(u_k) + e_k$$

Example: polynomial nonlinearity
(see exercise session)



Static Nonlinearity

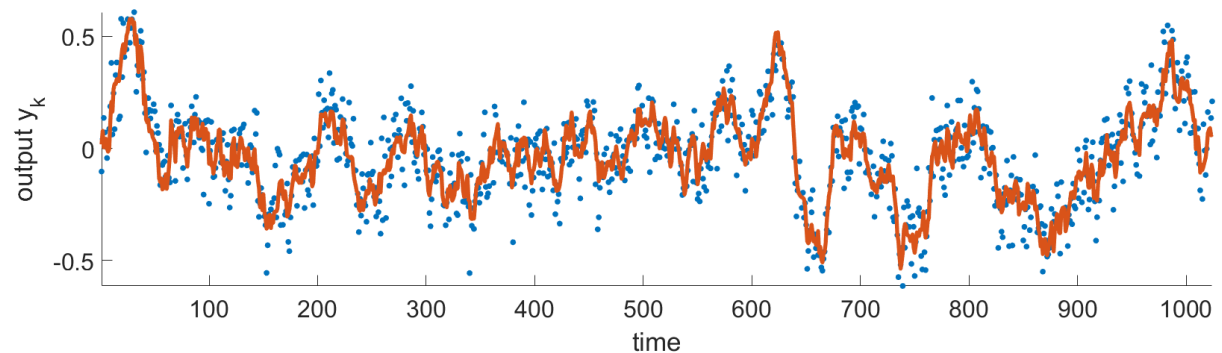
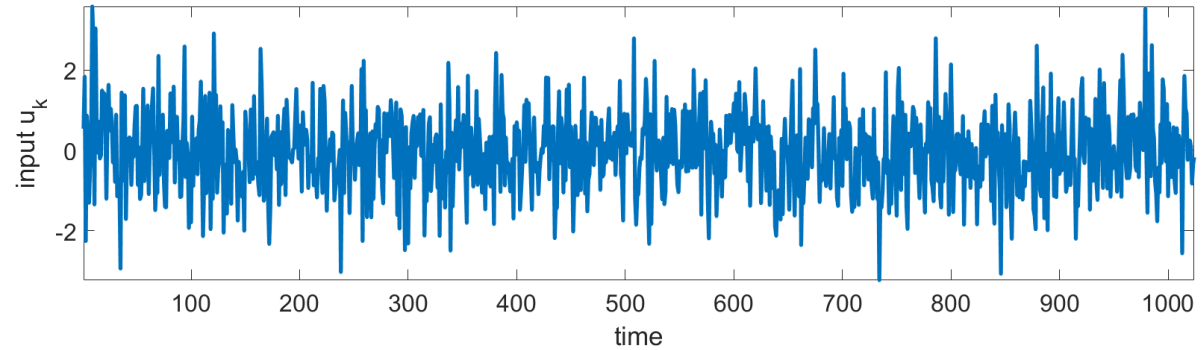
Output only depends on the instantaneous input(s)

Easy to analyze (for low-dimensional inputs)

Easy to estimate

Static vs. Dynamic

$$y_k = \sum_{i=1}^n g_i u_{k-i} + v_k$$



Dynamic Relation

Output depends past input, past output and/or system states
Analysis possible for linear dynamics (harder for nonlinear dynamics)
More difficult to estimate

Continuous vs Discrete Time

Continuous-Time Physics:

$$m\ddot{y}(t) + k_s y(t) = u(t)$$

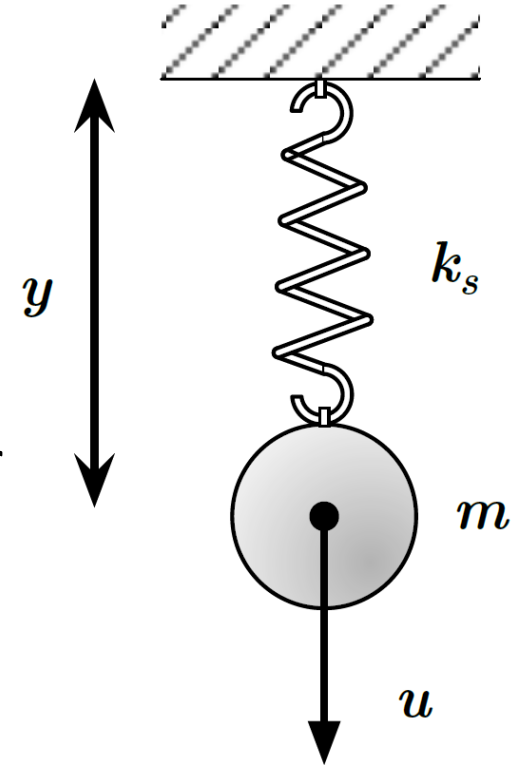
Data often gathered using fixed sampling intervals

→ Estimating discrete-time difference equation models is easier

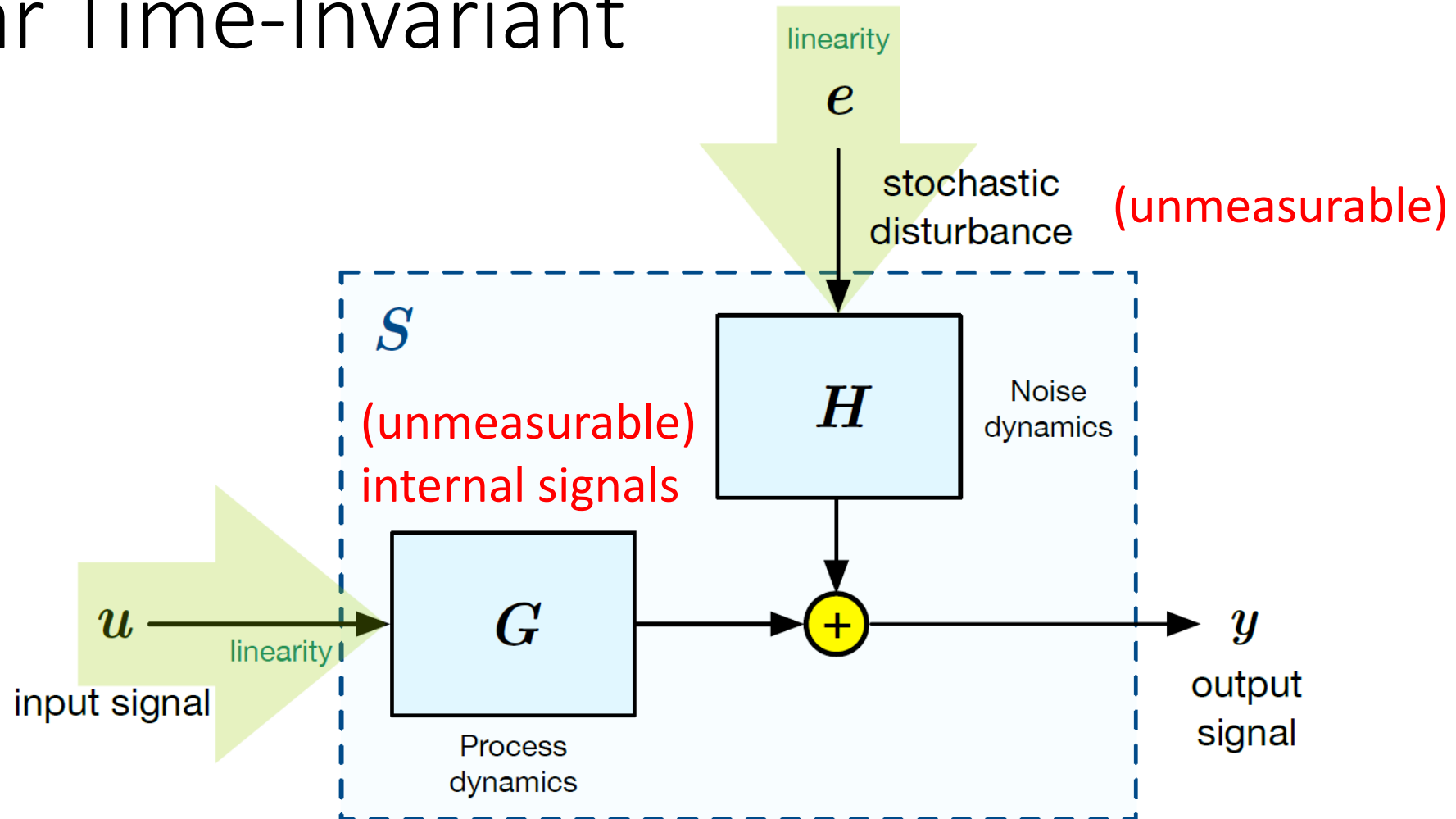
E.g. linear difference equation models:

$$\begin{aligned} y_k + a_1 y_{k-1} + \dots + a_{n_a} y_{k-n_a} \\ = u_k + b_1 u_{k-1} + \dots + b_{n_b} u_{k-n_b} \end{aligned}$$

Example: Linear Spring



Linear Time-Invariant



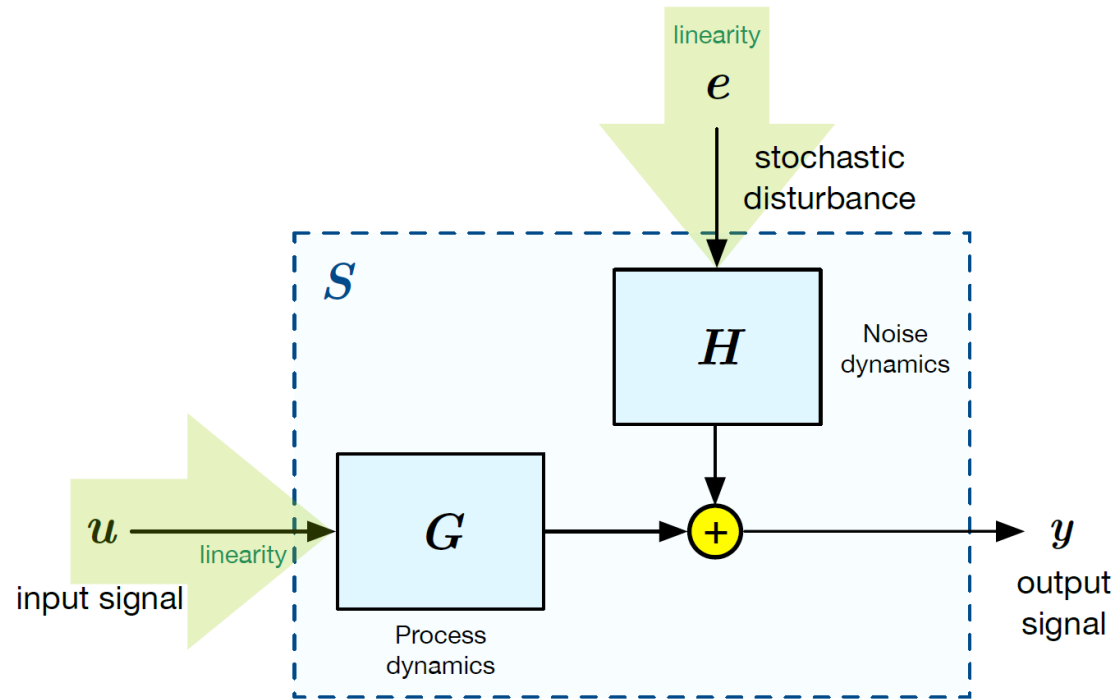
$$u : \mathbb{Z} \rightarrow \mathcal{U} \subseteq \mathbb{R}$$

(measurable) input signal

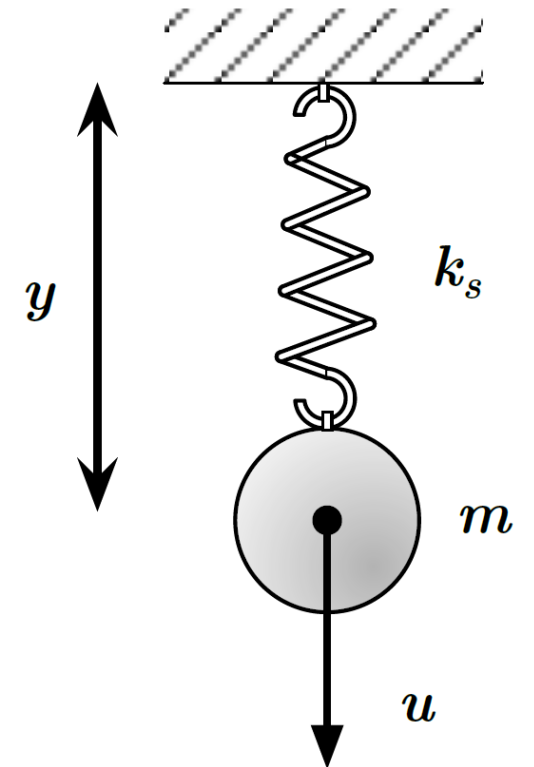
$$y : \mathbb{Z} \rightarrow \mathcal{Y} \subseteq \mathbb{R}$$

(measurable) output signal

Linear Time-Invariant



Example: Linear Spring



Choice of input / output partition might not be unique

Most often, causal relationships are considered

Stationary signals: statistical properties do not change over time

Representing LTI Dynamics

Impulse Response:

$$y_k = \sum_{i=1}^n g_i u_{k-i} + v_k$$

v_k : colored zero-mean noise process (induced by white noise e_k)

ARX:

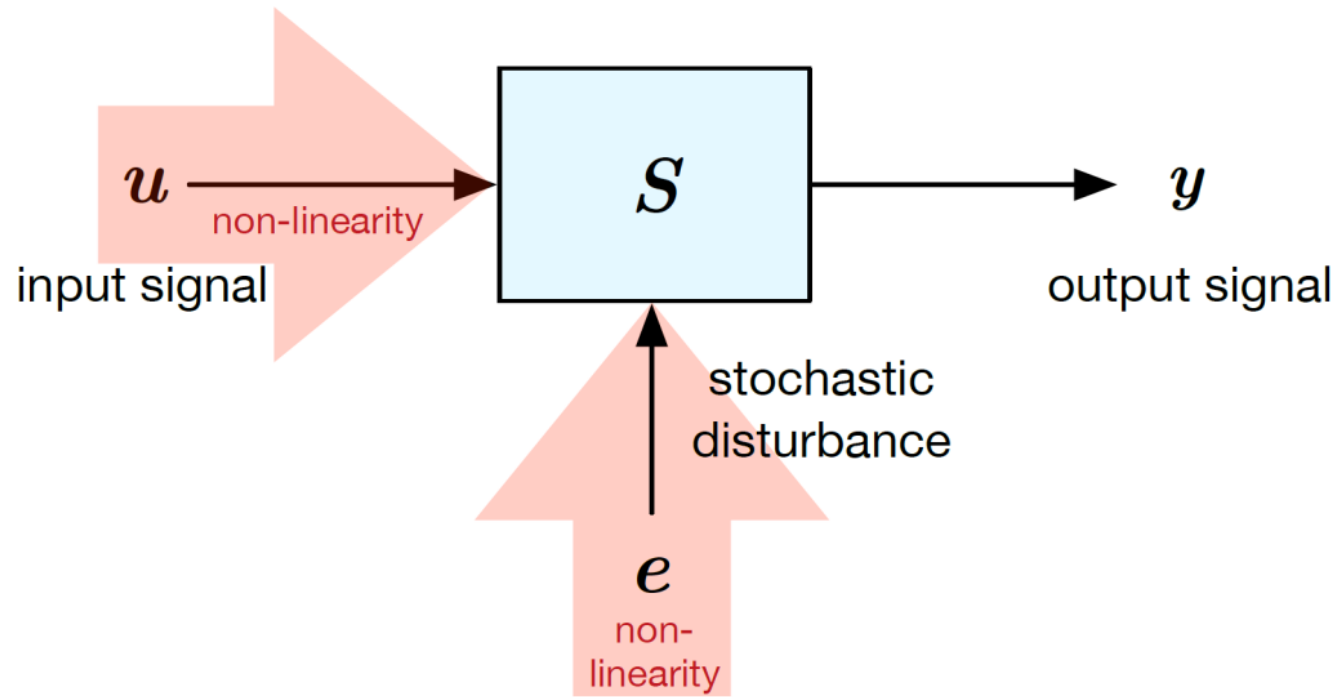
$$y_k = \sum_{i=1}^{n_a} a_i y_{k-i} + \sum_{j=0}^{n_b} b_j u_{k-j} + e_k$$

State-Space:

$$x_{k+1} = Ax_k + Bu_k + w_k \quad w_k: \text{process noise}$$
$$y_k = Cx_k + Du_k + e_k$$

Many more: ARMAX, Output Error, Box Jenkins, Frequency Domain,
also Continuous-Time counterparts.

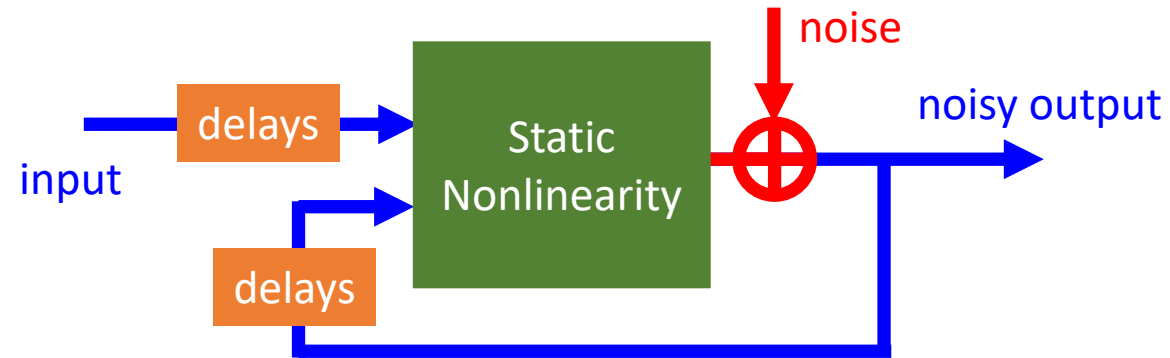
Nonlinear



Example: Flexible Aerospace Structures

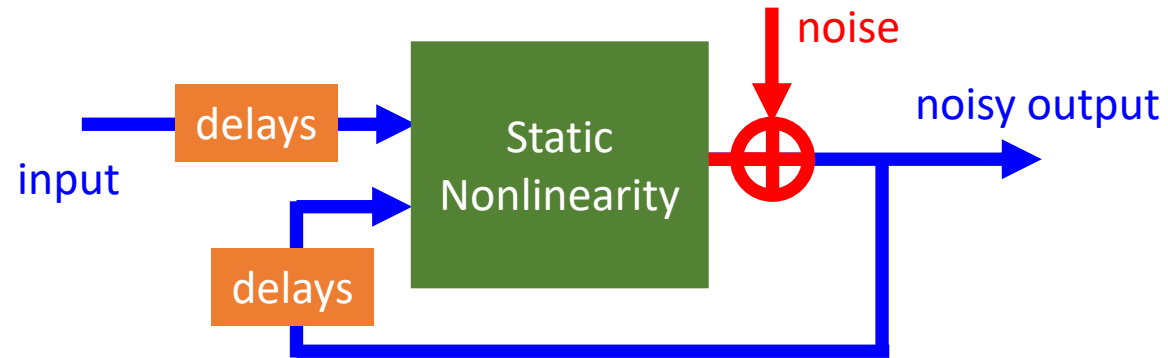


Nonlinear: NARX



$$y_k = f(u_k, u_{k-1}, \dots, u_{k-n_b}, y_{k-1}, \dots, y_{k-n_a}) + e_k$$

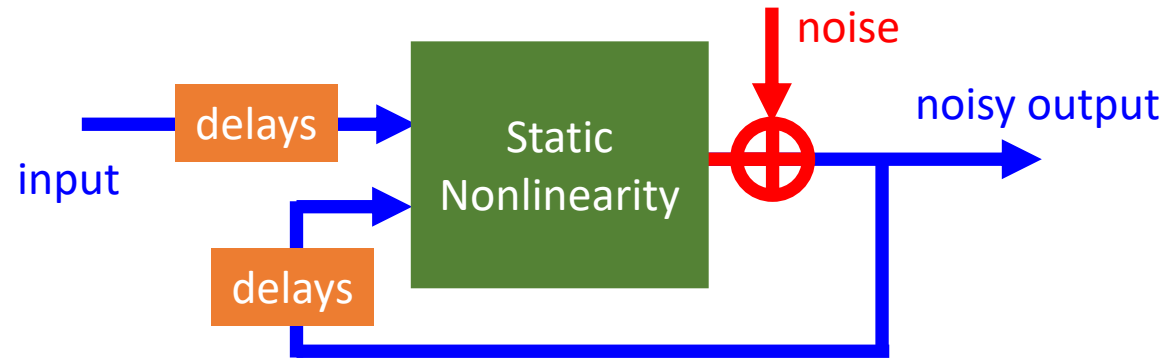
Nonlinear: NARX



$$\boxed{y_k} = \boxed{f}(\underbrace{u_k, u_{k-1}, \dots, u_{k-n_b}, y_{k-1}, \dots, y_{k-n_a}}_{\text{known}}) \boxed{+ e_k}_{\text{unknown}}$$

To be estimated

Nonlinear: NARX



$$y_k = f(u_k, u_{k-1}, \dots, u_{k-n_b}, y_{k-1}, \dots, y_{k-n_a}) + e_k$$

To be estimated
known
unknown

NARX (Nonlinear AutoRegressive with eXogenous input)

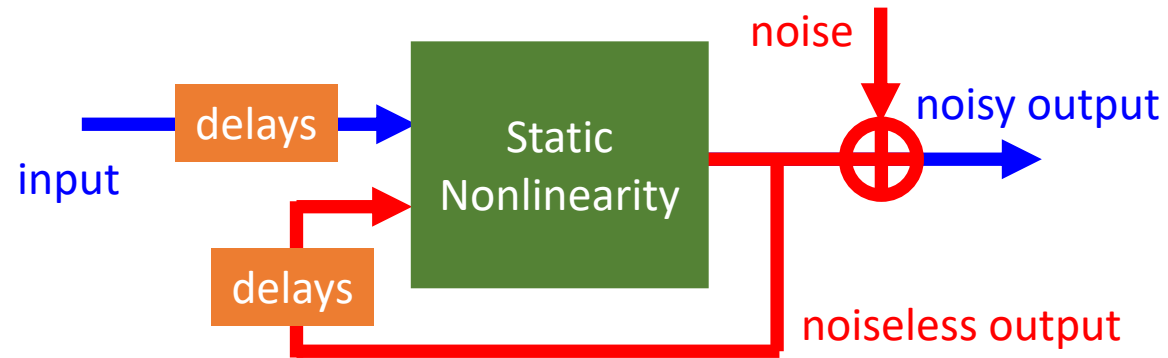
Output depends on past inputs and past measured noisy outputs

Difficult to analyze

Easy to estimate

Noise part of the process dynamics

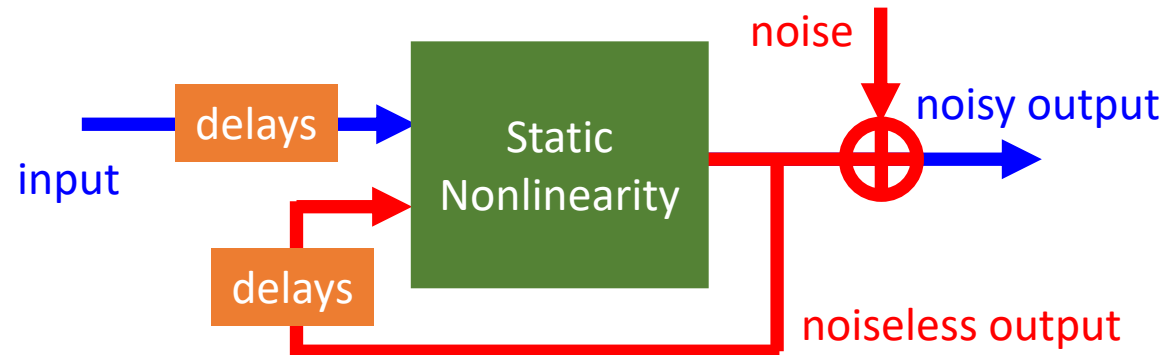
Nonlinear: NOE



$$s_k = f(u_k, u_{k-1}, \dots, u_{k-n_b}, s_{k-1}, \dots, s_{k-n_a})$$

$$y_k = s_k + e_k$$

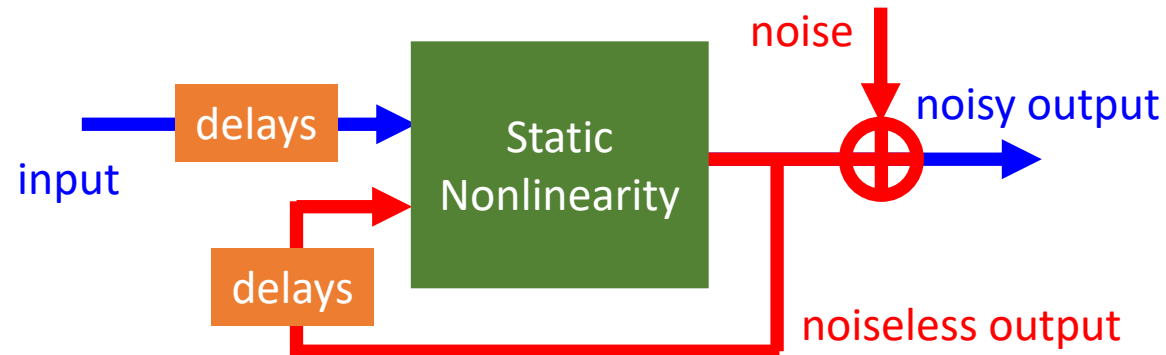
Nonlinear: NOE



To be estimated

$$\begin{aligned}
 \text{unknown } s_k &= f(u_k, u_{k-1}, \dots, u_{k-n_b} \mid s_{k-1}, \dots, s_{k-n_a}) \\
 y_k &= s_k + e_k \quad \text{known}
 \end{aligned}$$

Nonlinear: NOE



To be estimated

$$\begin{aligned}
 \text{unknown } s_k &= f(u_k, u_{k-1}, \dots, u_{k-n_b}, s_{k-1}, \dots, s_{k-n_a}) \\
 y_k &= s_k + e_k
 \end{aligned}$$

known

NOE (Nonlinear Output Error)

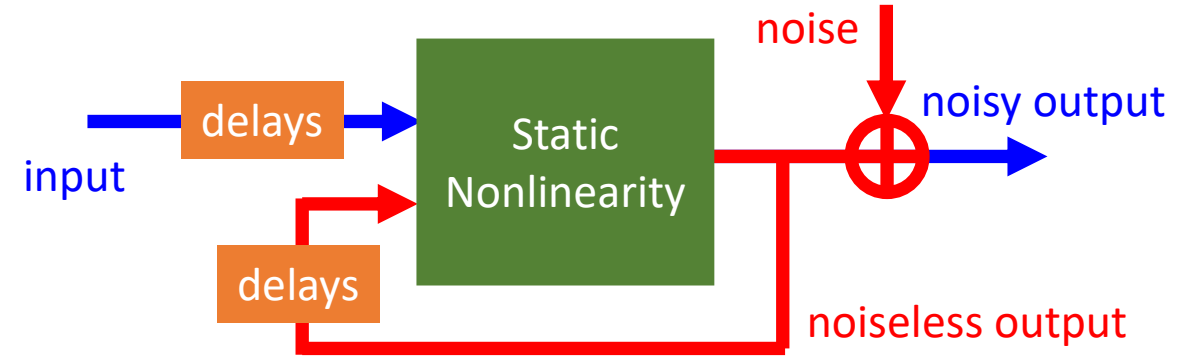
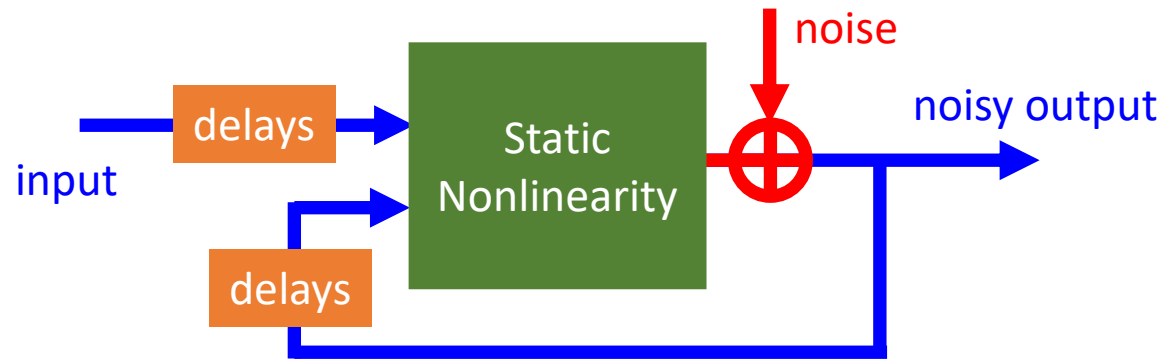
Output depends on past inputs and past unknown noiseless outputs

Difficult to analyze

Difficult to estimate

Noise only on the measurements

Nonlinear: NARX vs NOE



$$y_k = f(u_k, u_{k-1}, \dots, u_{k-n_b}, y_{k-1}, \dots, y_{k-n_a}) + e_k$$

$$s_k = f(u_k, u_{k-1}, \dots, u_{k-n_b}, s_{k-1}, \dots, s_{k-n_a})$$

$$y_k = s_k + e_k$$

Different Noise Model

Nonlinear: State-Space

$$x_{k+1} = f(x_k, u_k) + w_k$$

$$y_k = h(x_k, u_k) + e_k$$

Nonlinear: State-Space

unknown states

known input

$$x_{k+1} = f(x_k, u_k) + w_k$$

process noise

$$y_k = h(x_k, u_k) + e_k$$

output noise

known output

multivariate static
nonlinearities

Nonlinear: State-Space

unknown states

known input

$$x_{k+1} = f(x_k, u_k) + w_k$$

process noise

$$y_k = h(x_k, u_k) + e_k$$

output noise

known output

multivariate static
nonlinearities

Nonlinear State-Space

Output depends on the instantaneous inputs and (unknown) states

Some analysis possibilities

Difficult to estimate (due to unmeasured states)

Multiple noise structures possible (output and process noise)

Learning Challenge

How to estimate the dynamics starting from (noisy) input-output data?

How to select *model order* and *function complexity*?

How to *estimate the functions* (f, h) directly from data $\{(y_k, u_k)\}_{k=1}^N$?

In fact, structure free (non-parametric) system identification always reduces to function estimation under various system classes, information sources and stochastic settings!

Learning Challenge

How to estimate functions efficiently?

Artificial Neural Networks (ANNs)

Pros: high flexibility and approximation capability

Cons: immature theory (stochastic prop., function space, representer concept), no reliable uncertainty bounds

Gaussian Processes (GPs)

Pros: mature theory: stochastic guarantees and uncertainty bounds (very useful for control and also for model certification)

Cons: can be seen as a 1-layer ANN (lower approximation capabilities)

Contents

Data-Driven Modelling Motivation

Data-Generating Systems / Models of Dynamical Systems

Estimators

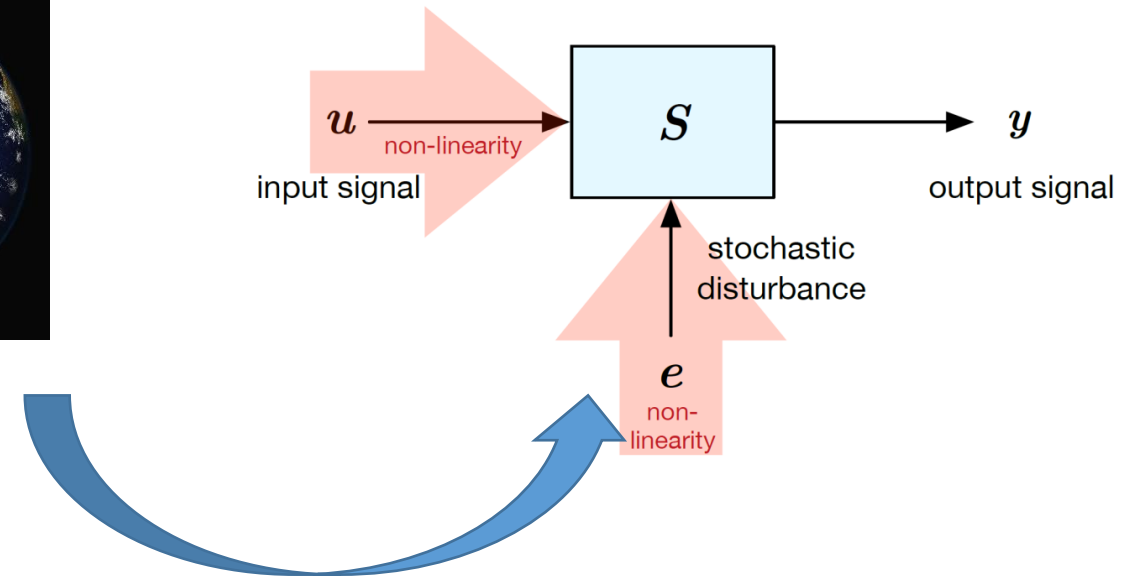
Data-Driven Modelling Process

Estimation Criterion

The cost function describes the objective we want to minimize or maximize

Describes how well the model matches the data

Can be extended with extra penalty terms (regularization)



Cost Function
Objective Function
Loss Function

Typical Cost Function

Squared L^2 Norm

$$\frac{1}{N} \sum_{k=0}^{N-1} (y_k - \hat{y}_k)^2$$

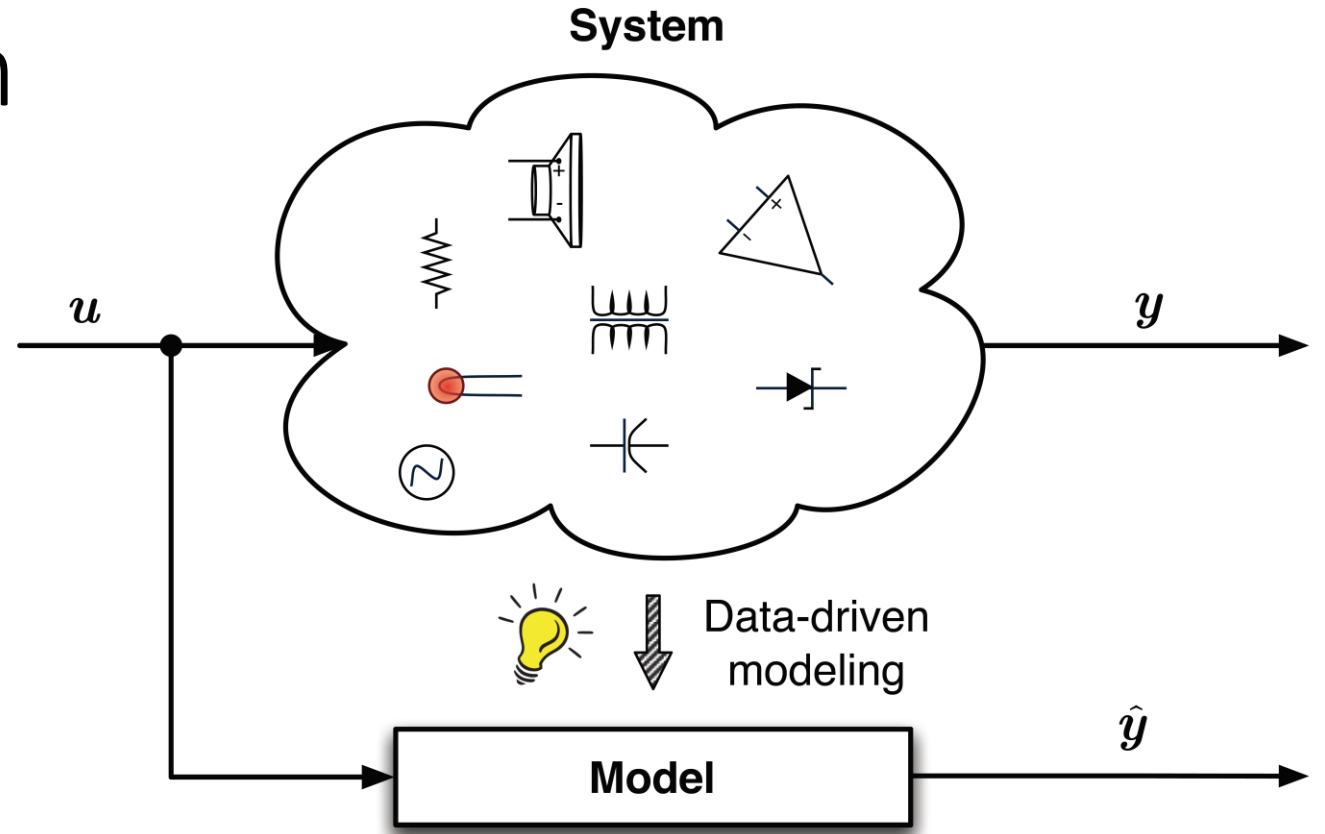
normalize with
samples

simulation error
or
prediction error

Other cost functions:
Maximum Likelihood

Extensions:

Regularization Terms (e.g. l^1 or l^2 Norm of Parameter Vector)



1-Step Ahead Prediction

$$\hat{y}_k = \mathbb{E} \{ y_k | \mathcal{D}_{k-1}, u_k \}$$

data up to timestep $k-1$

predicted output
at timestep k

input at timestep k

Expectation taken over the stochastic disturbances

Simulation

$$\hat{y}_k = \mathbb{E} \left\{ y_k \mid \{u_j\}_{j=1}^k \right\}$$

↓
simulated output
at timestep k

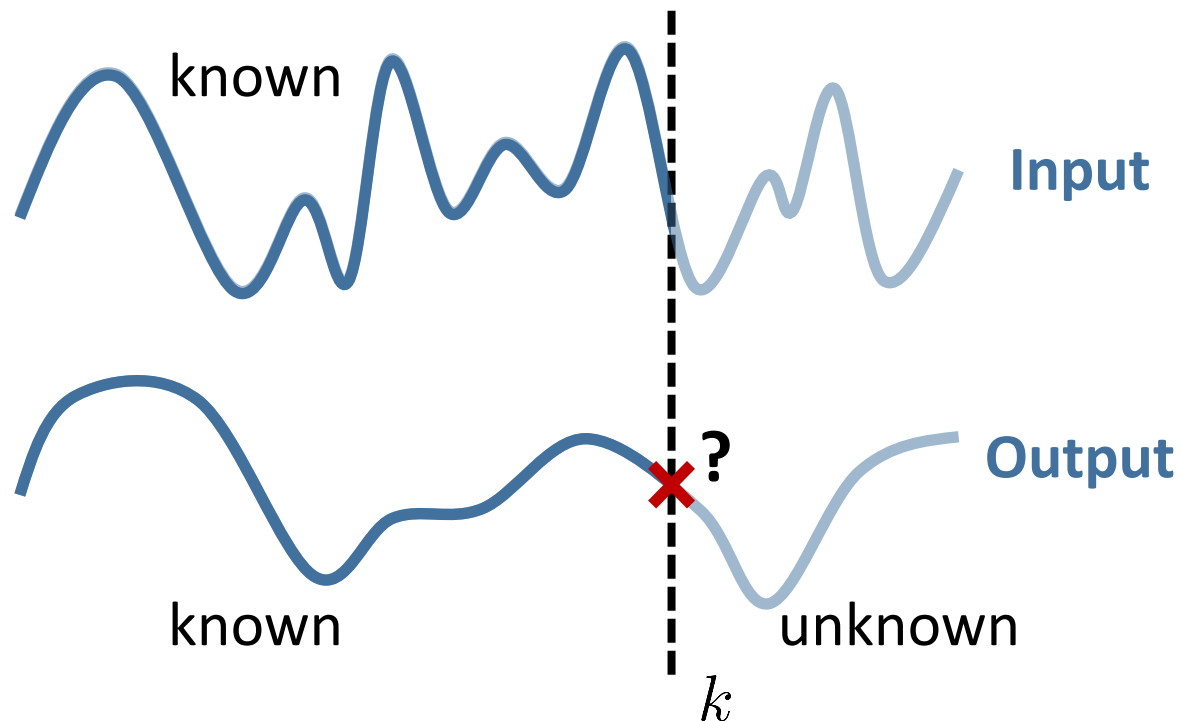
↓
inputs from start
up to timestep k

Expectation taken over the stochastic disturbances

Prediction vs Simulation

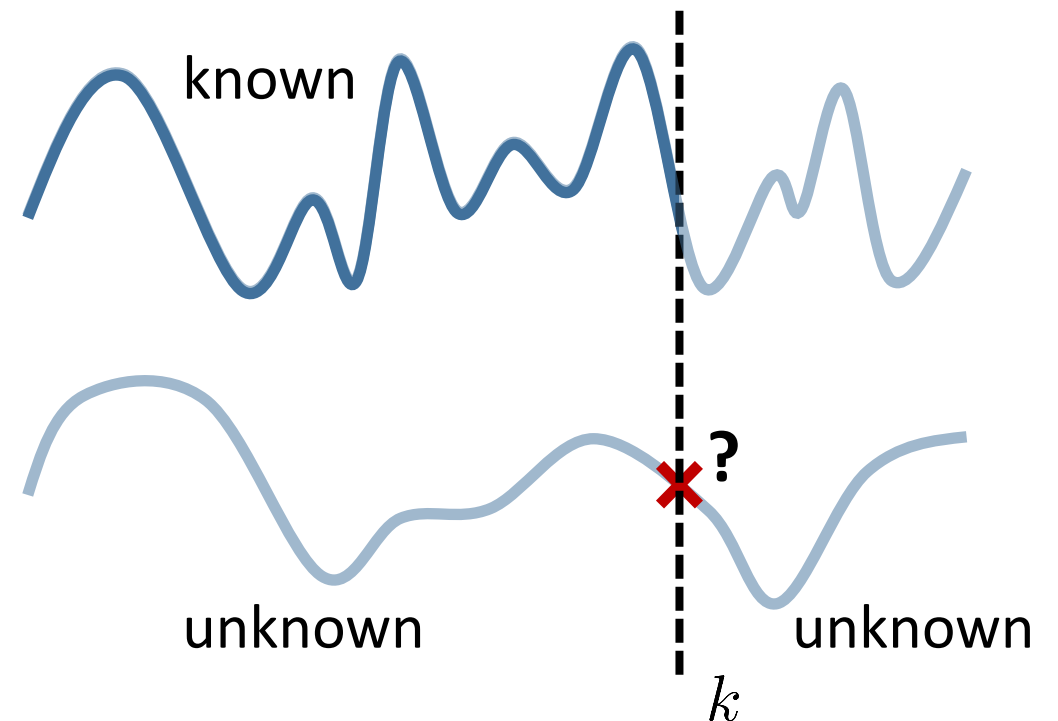
Prediction

$$\hat{y}_k = \mathbb{E} \{y_k | \mathcal{D}_{k-1}, u_k\}$$



Simulation

$$\hat{y}_k = \mathbb{E} \{y_k | \{u_j\}_{j=1}^k\}$$



Estimator

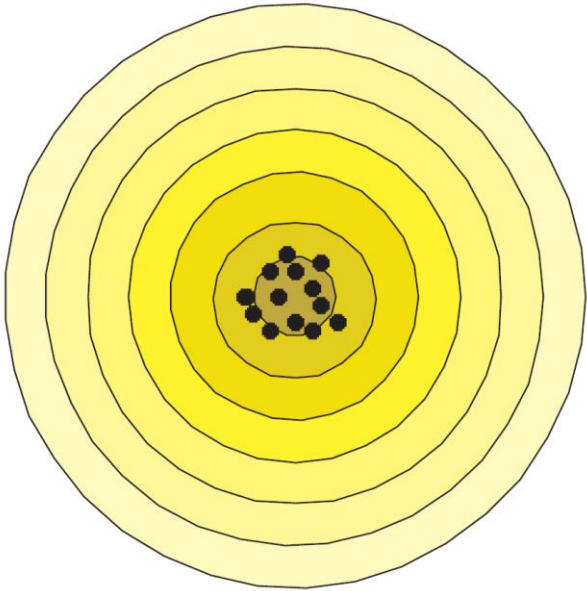
An **estimator** $\hat{\theta}_N$ of θ_o is a mapping from the (measured) data $\mathcal{D}_N$ to $\hat{\theta}_N$. If the data contains random variables, then $\hat{\theta}_N$ is a random variable.

Estimator

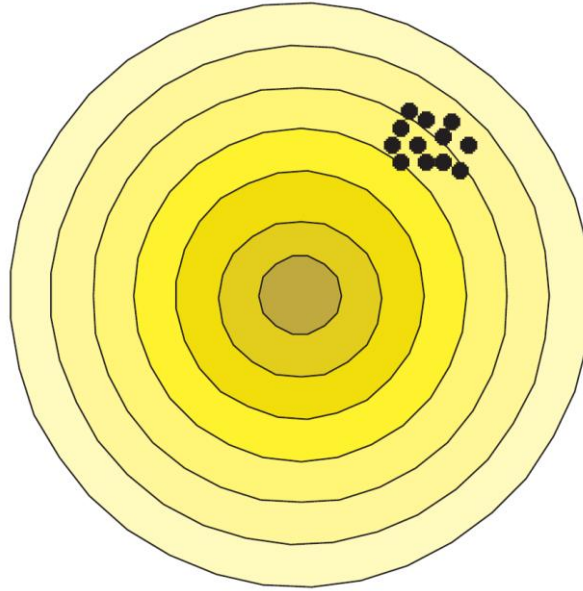
An **estimator** $\hat{\theta}_N$ of θ_o is a mapping from the (measured) data $\mathcal{D}_N$ to $\hat{\theta}_N$. If the data contains random variables, then $\hat{\theta}_N$ is a random variable.

- Unbiased: $\mathbb{E}\{\hat{\theta}_N\} = \theta_* = \theta_o$ $\text{bias} := \theta_* - \theta_o$
- Variance: $P_\theta = \mathbb{E}\{(\hat{\theta}_N - \theta_*)(\hat{\theta}_N - \theta_*)^\top\}$
- Consistent: $\hat{\theta}_N$ is consistent if:
 - $P\{\lim_{N \rightarrow \infty} \hat{\theta}_N = \theta_o\} = 1$
 - $\hat{\theta}_N \rightarrow \theta_o$ with probability 1 for $N \rightarrow \infty$.

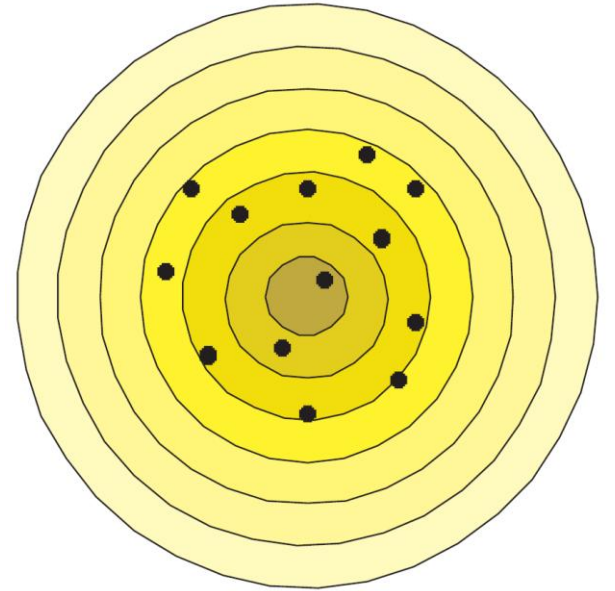
Estimator



Unbiased
Small variance



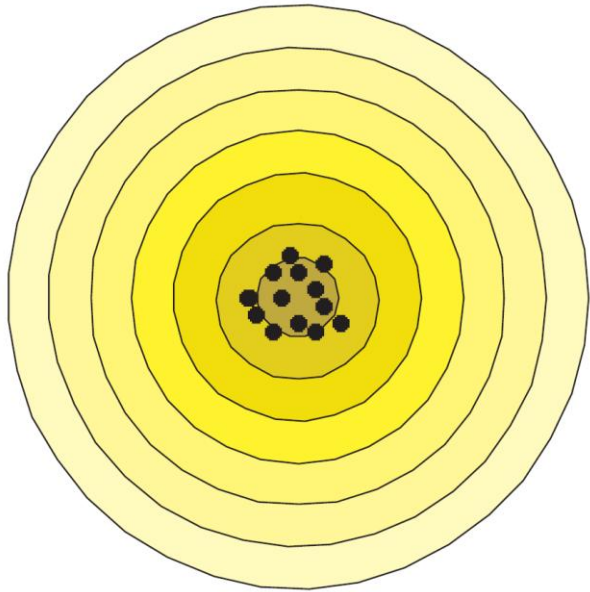
Biased
Small variance



Unbiased
Large variance

Consistency = Asymptotically unbiased and variance tends to 0

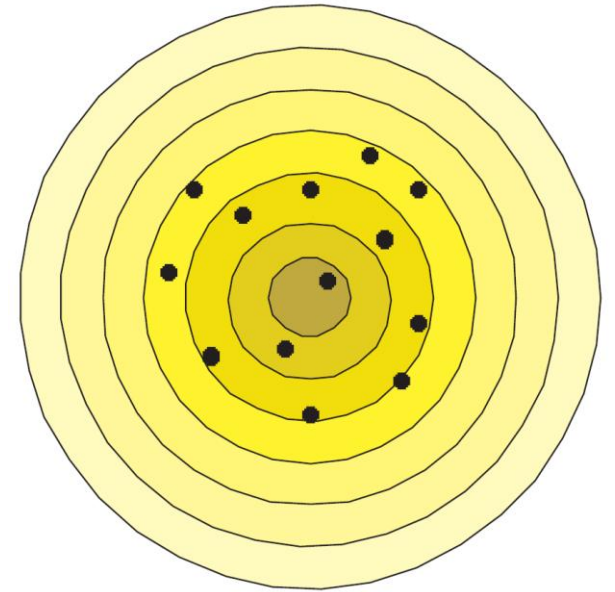
Estimator



Unbiased
Small variance



Biased
Small variance



Unbiased
Large variance

Bias – Variance Trade-Off

Consistency = Asymptotically unbiased and variance tends to 0

Contents

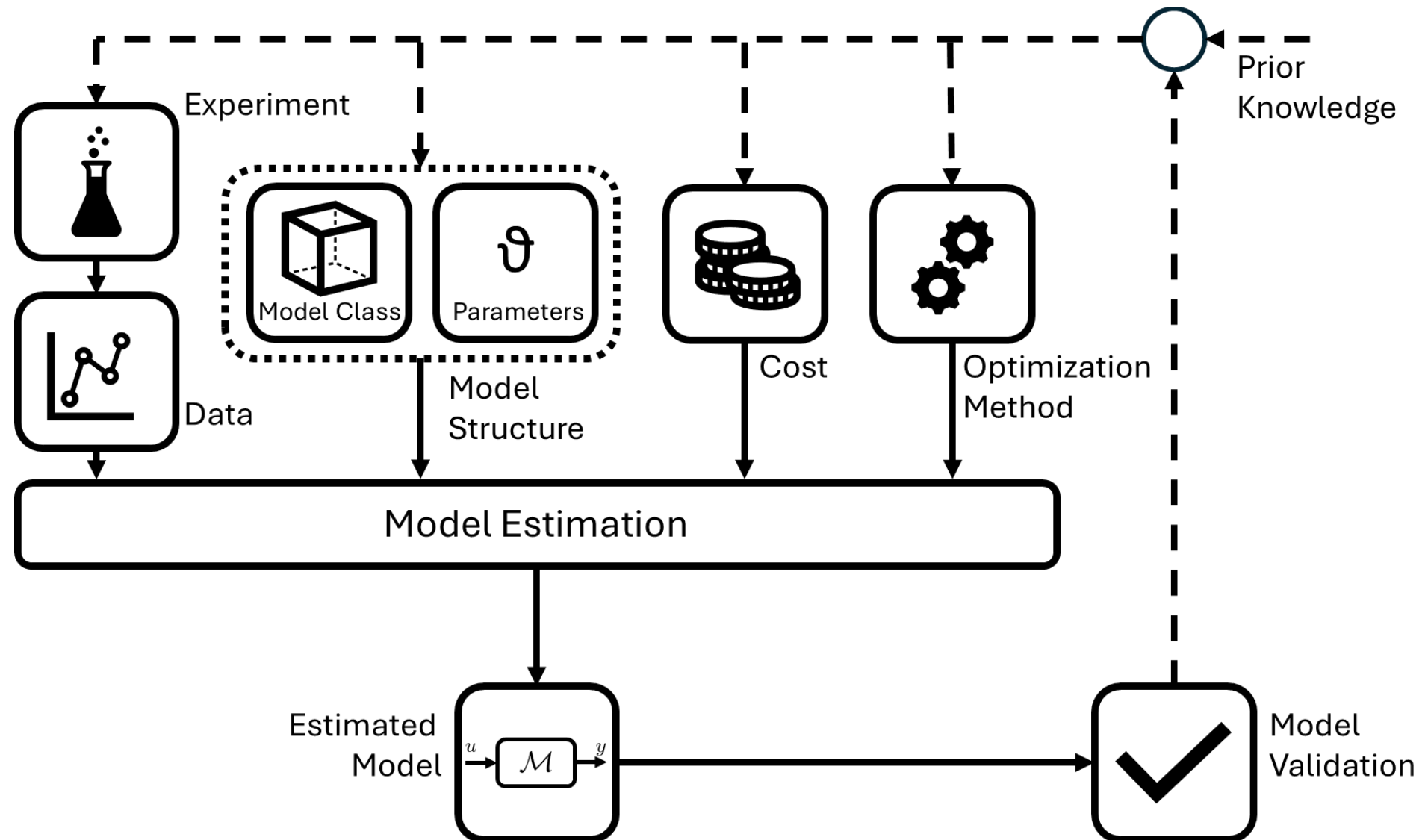
Data-Driven Modelling Motivation

Data-Generating Systems / Models of Dynamical Systems

Estimators

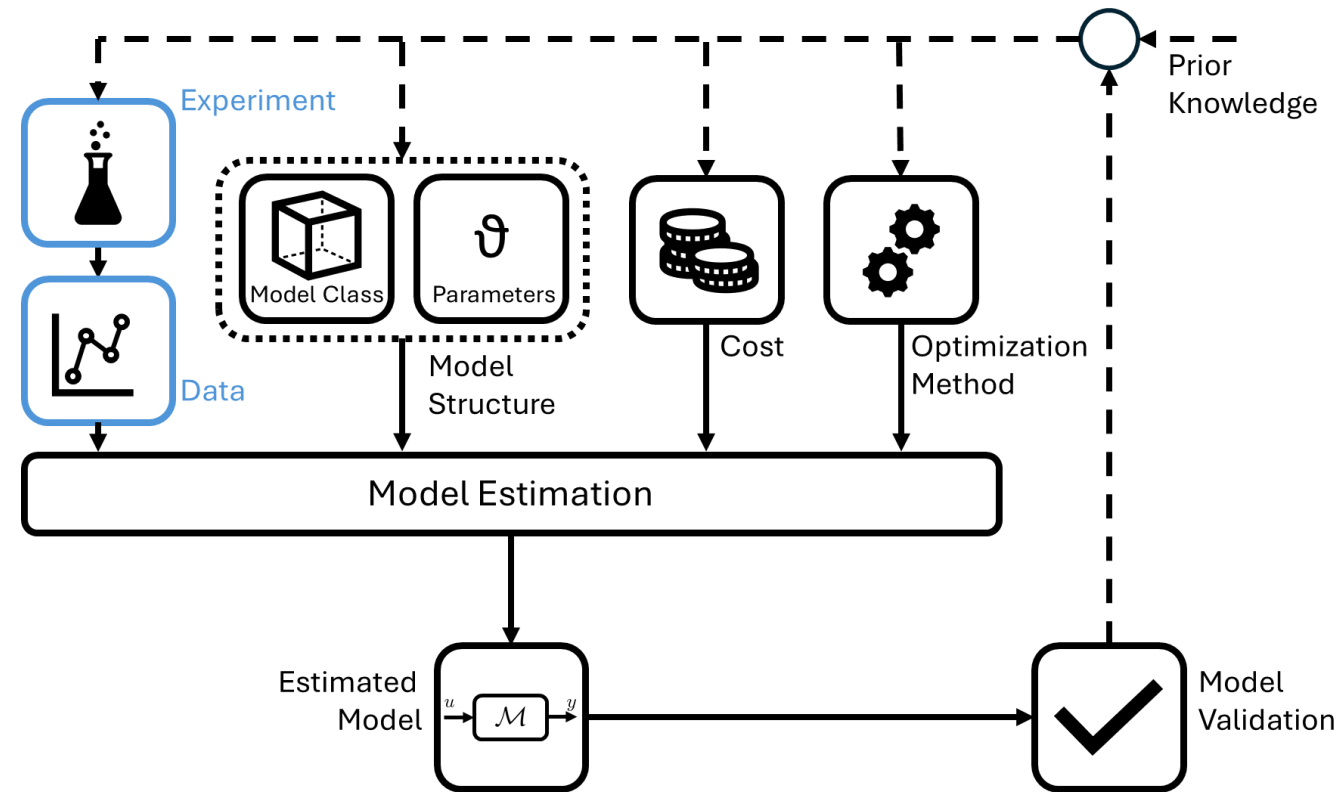
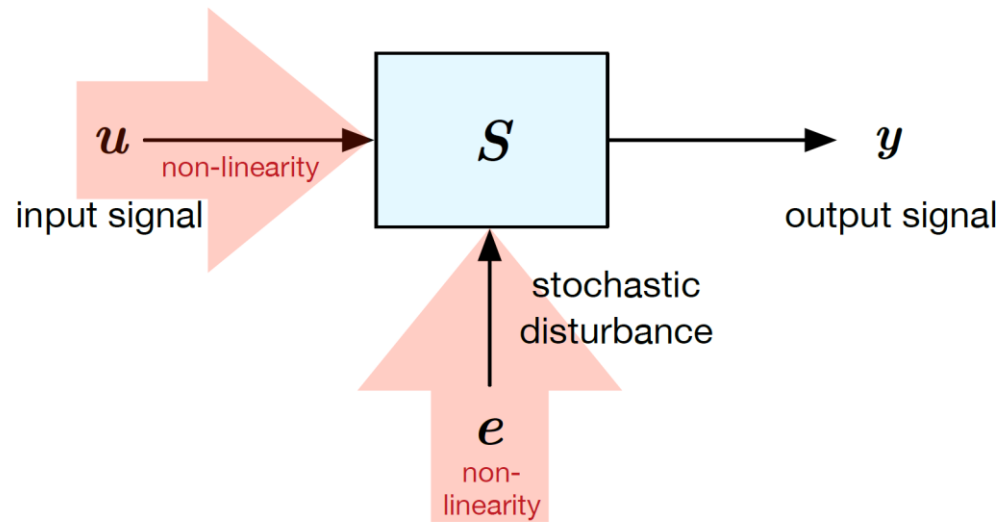
Data-Driven Modelling Process

Data-Driven Modelling Process



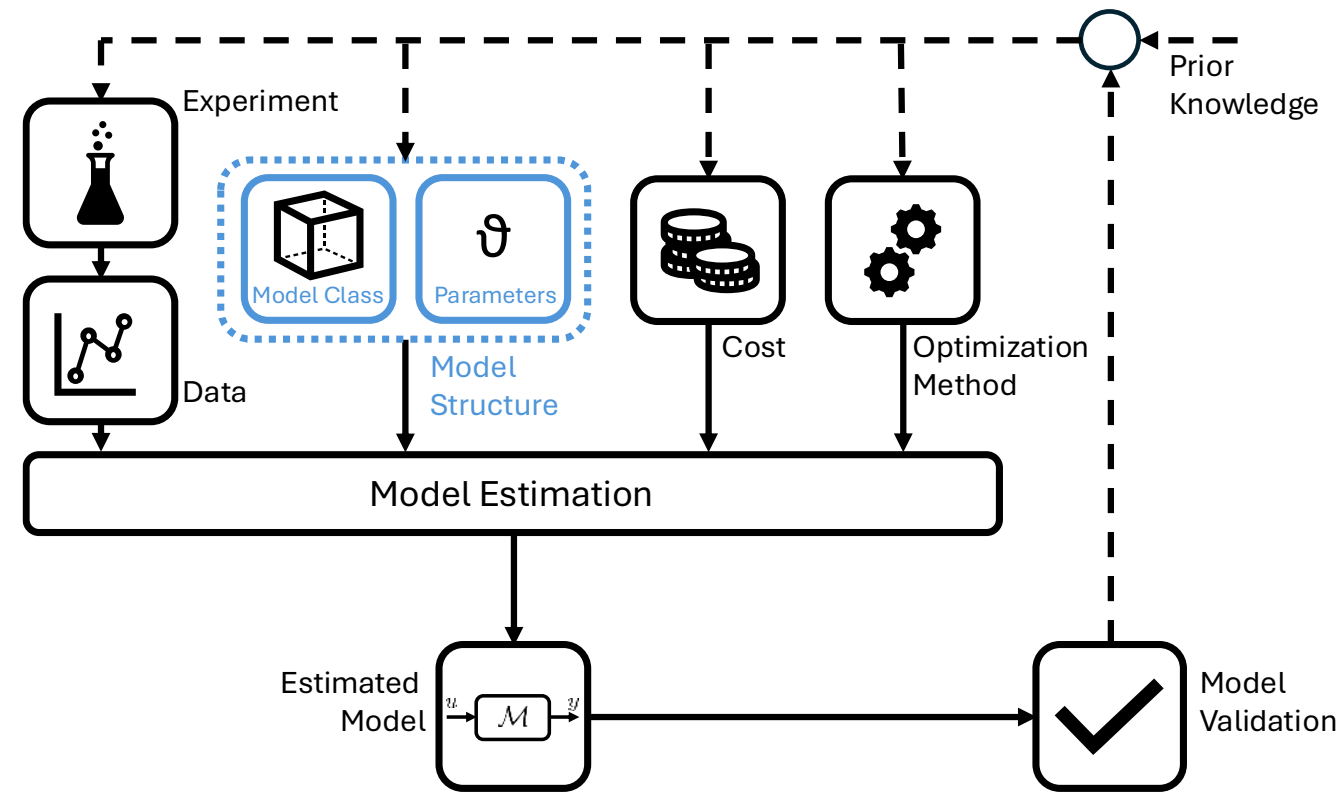
Data-Driven Modelling Process: Data

Measured data can be given to you
or can be obtained after
performing carefully designed
experiments.



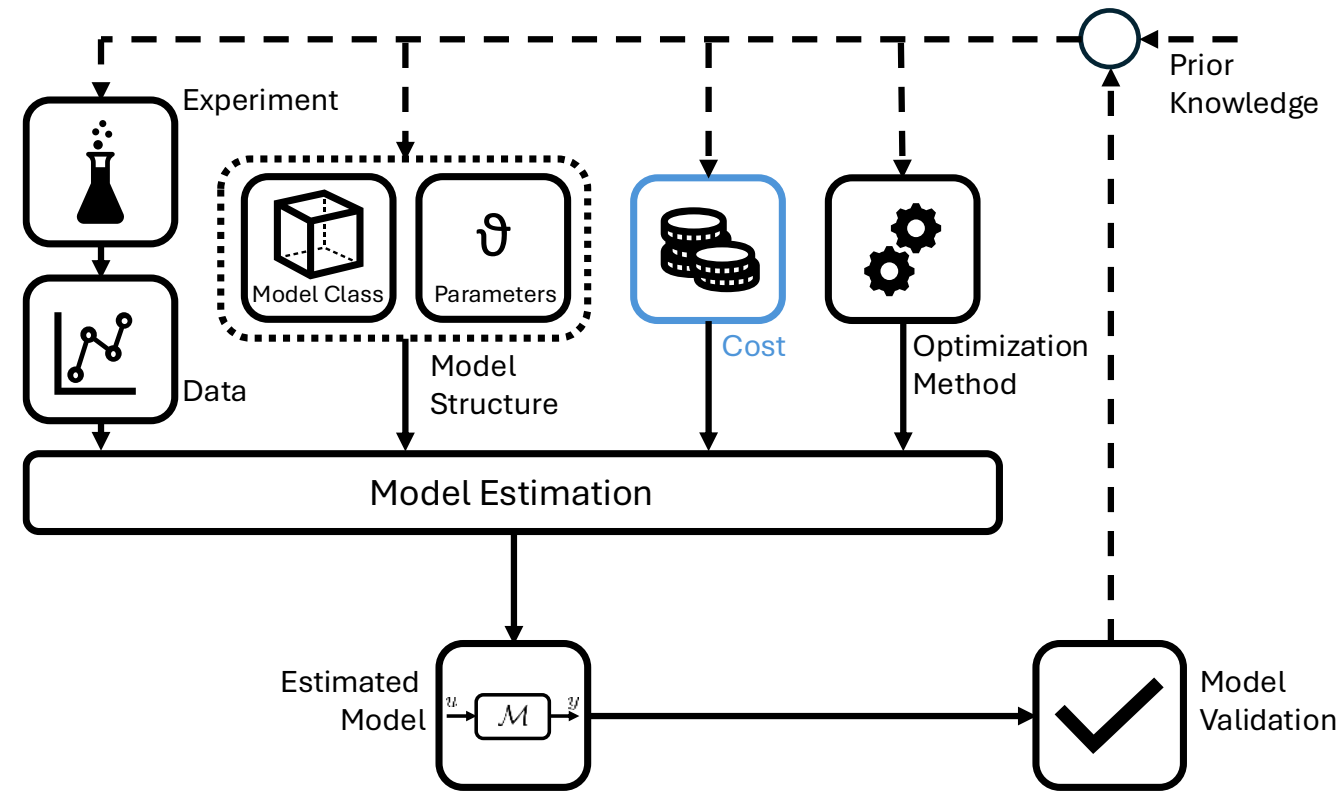
Data-Driven Modelling Process: Model Structure

Type of model you want to estimate with a chosen parametrization, also includes a noise model.



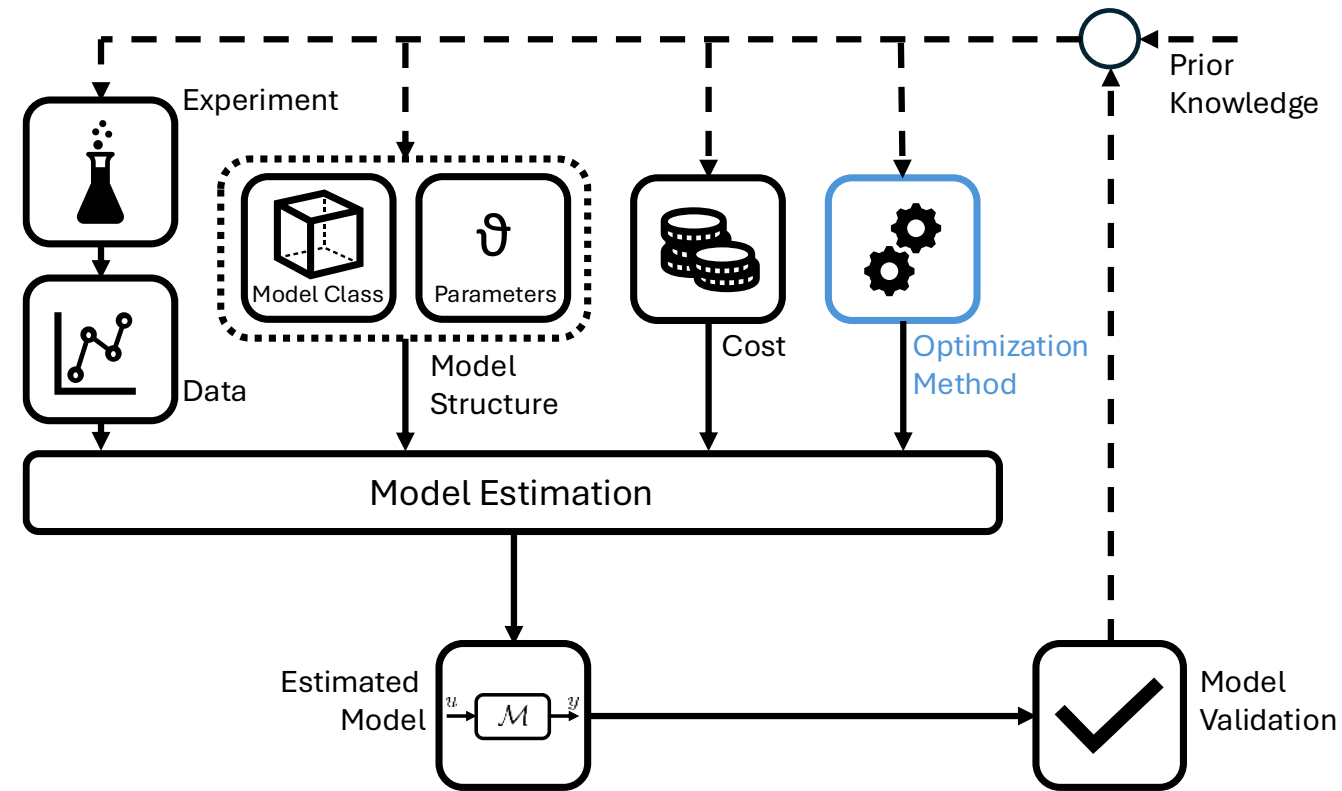
Data-Driven Modelling Process: Cost

Expresses how 'far' the model is from the data.



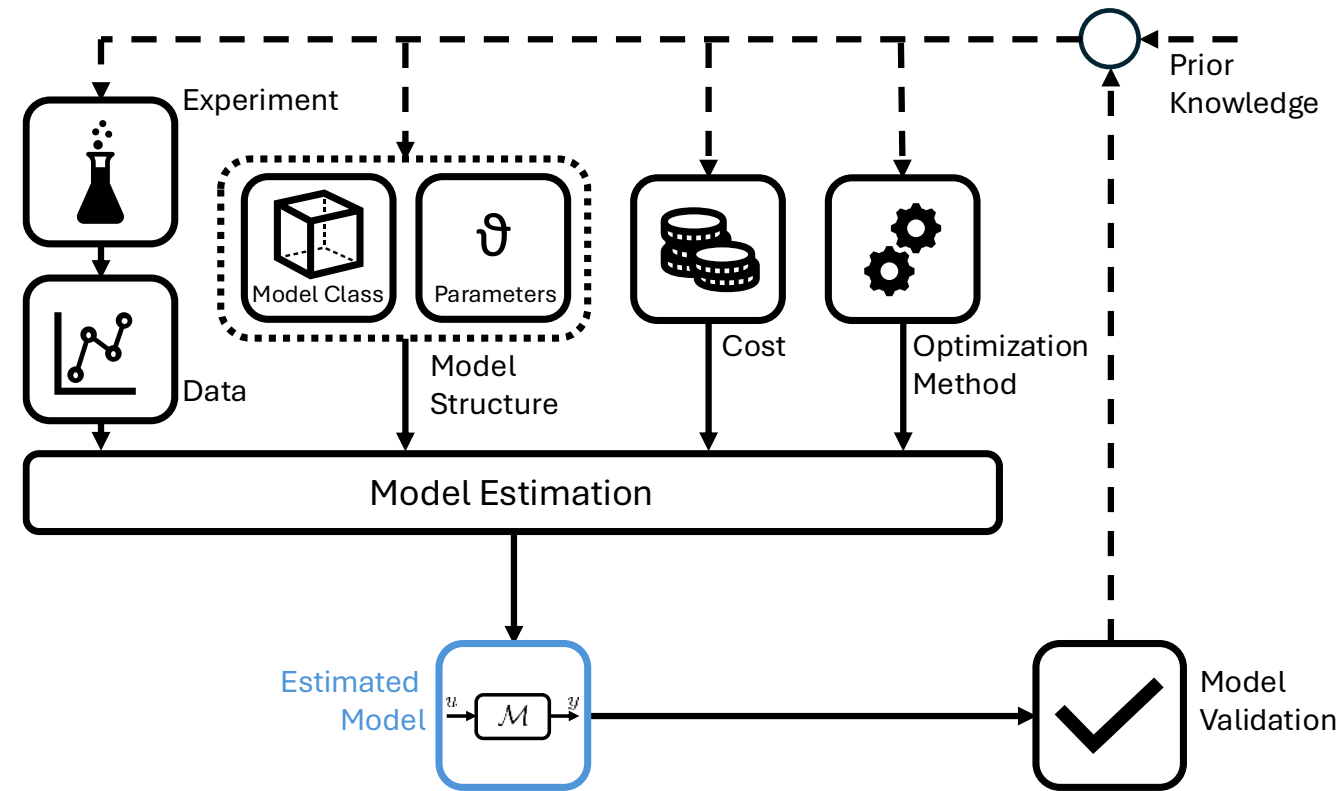
Data-Driven Modelling Process: Optimization

Minimizing the cost by modifying the model parameters results in the estimated model.



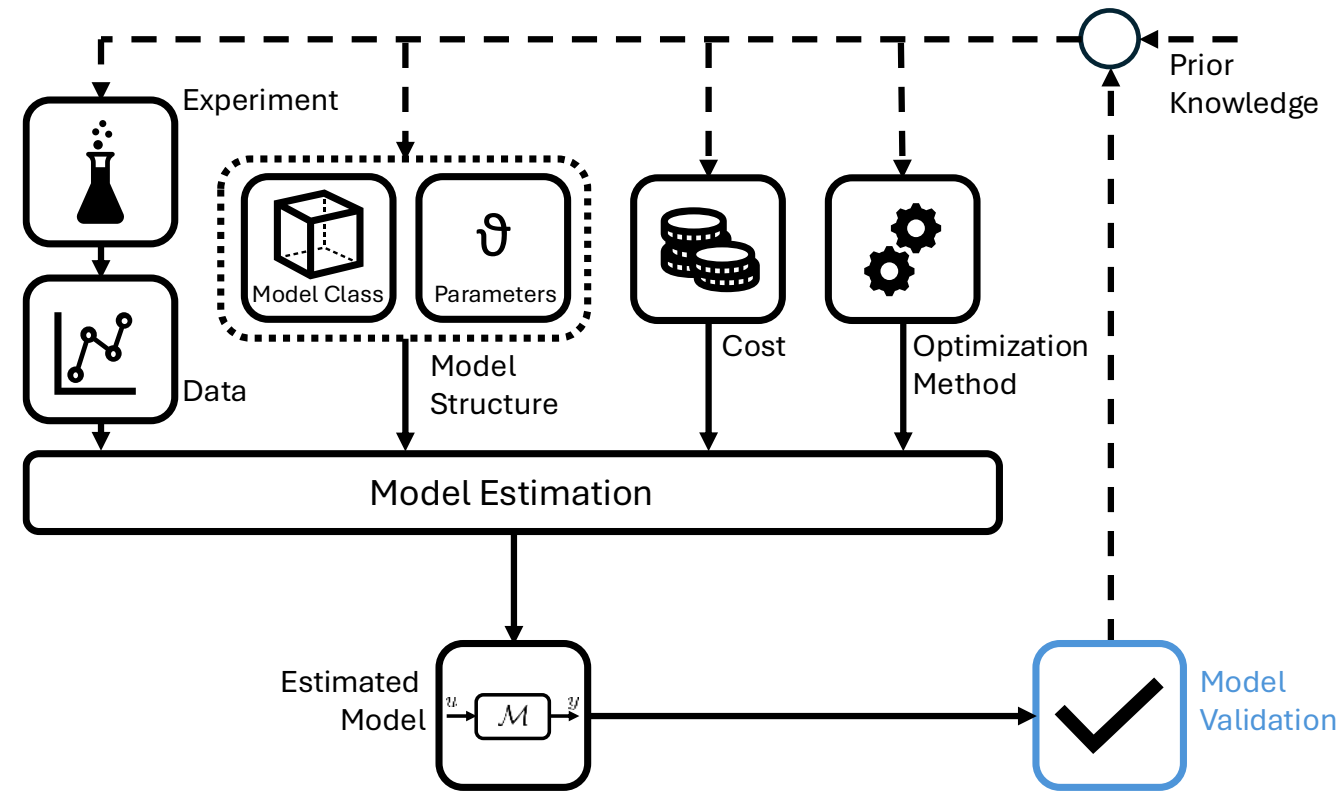
Data-Driven Modelling Process: Estimate

The estimated parameters combined with the previously chosen model structure result in the estimated model.

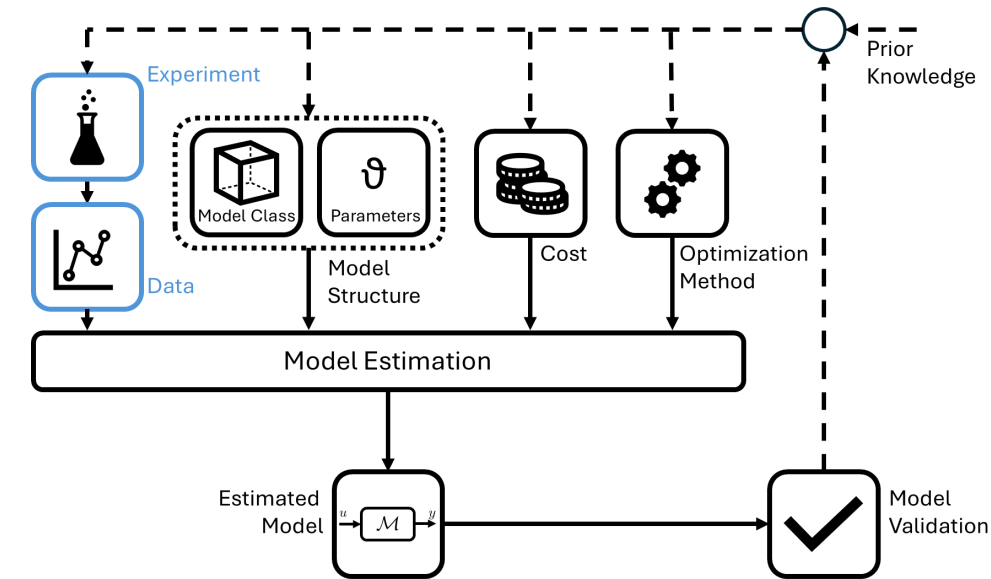
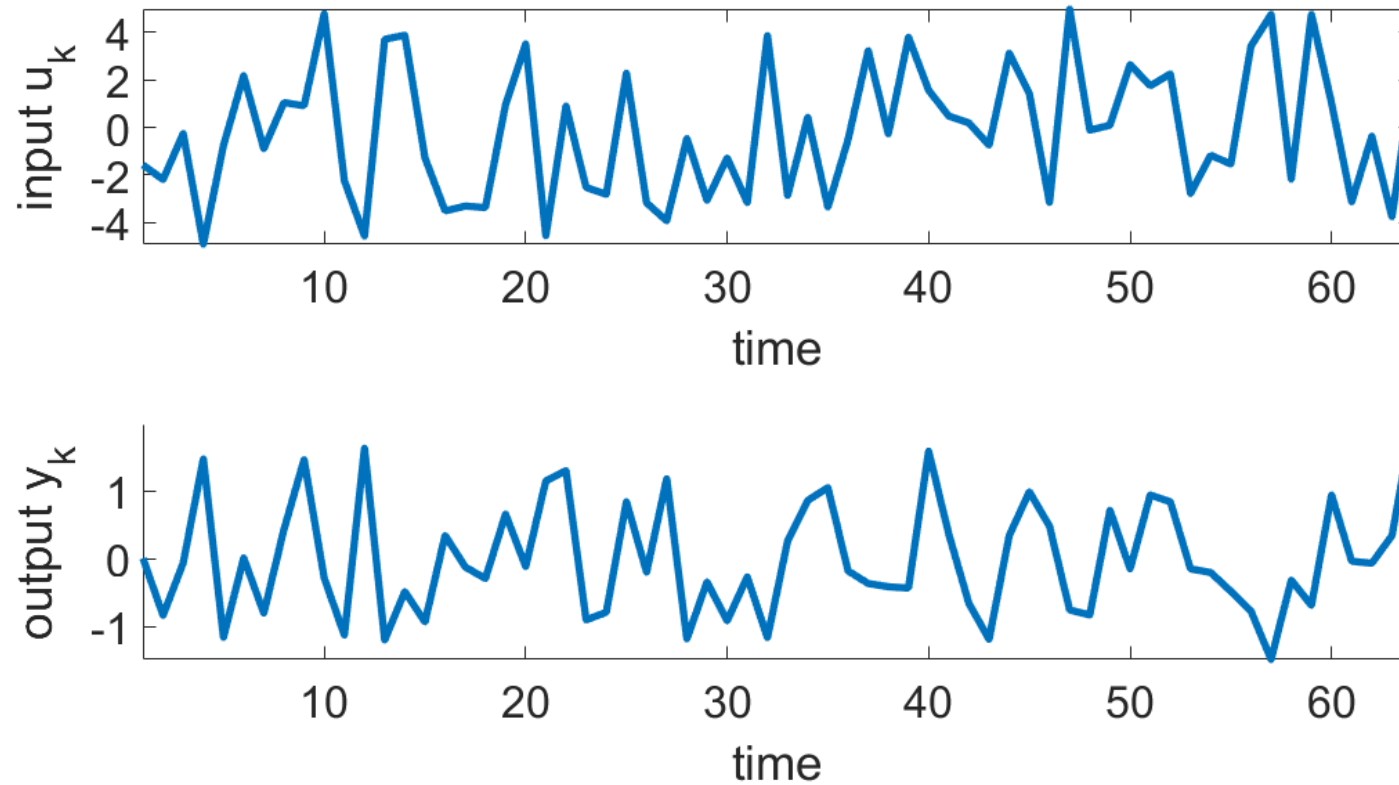


Data-Driven Modelling Process: Validation

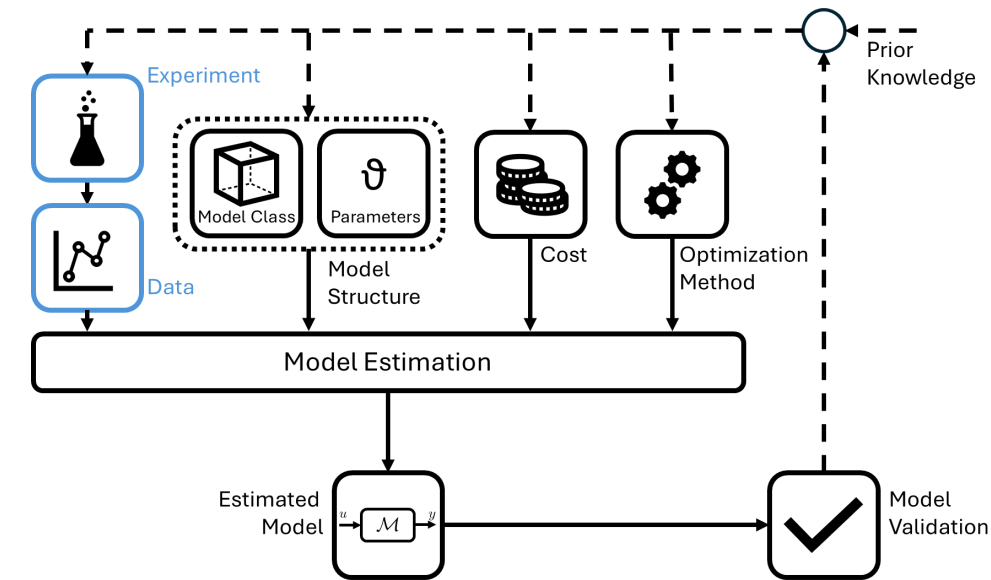
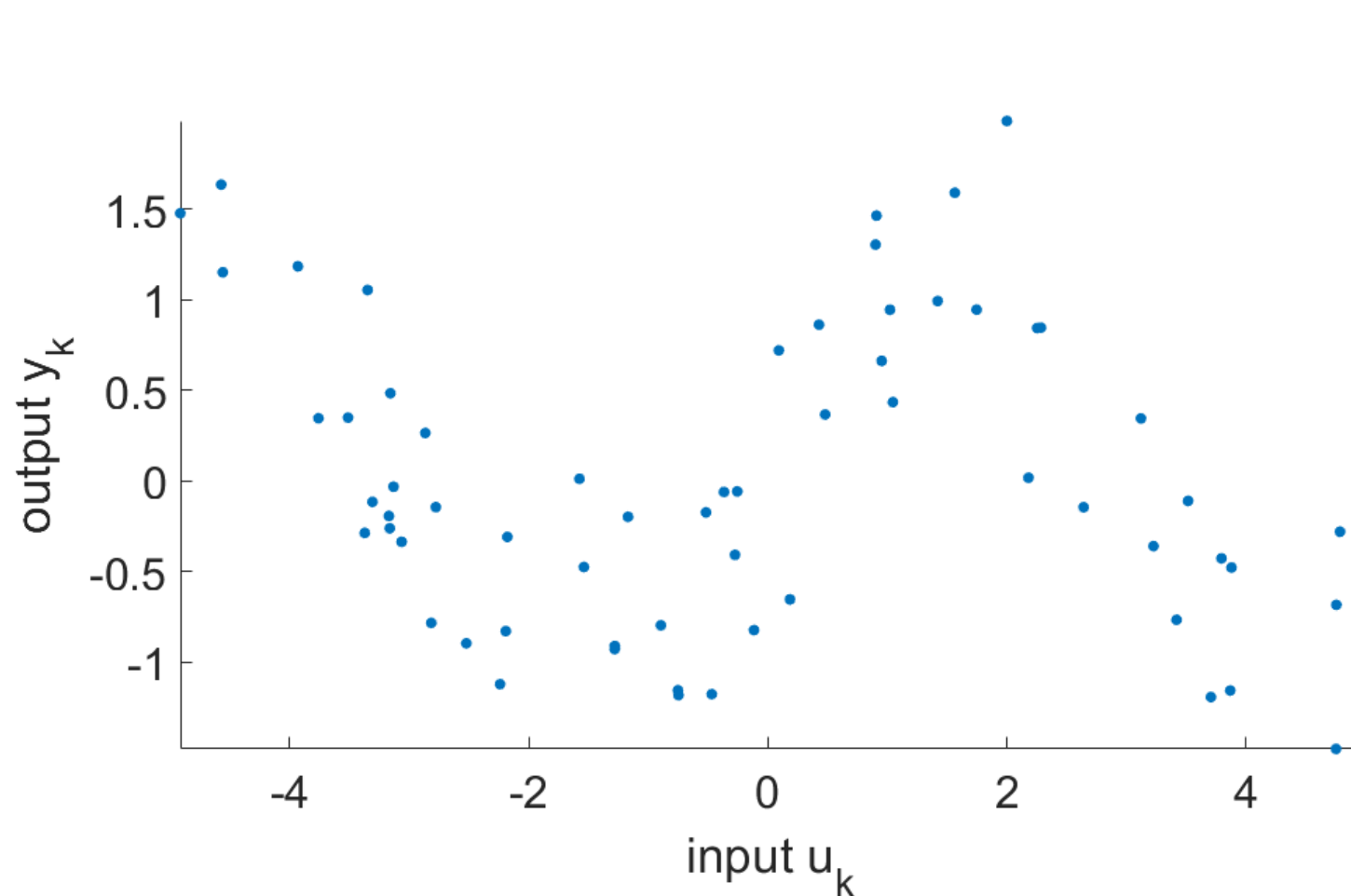
Validate the model, e.g. on a validation dataset. Decide if the model is good, or if new data or a different model structure is needed.



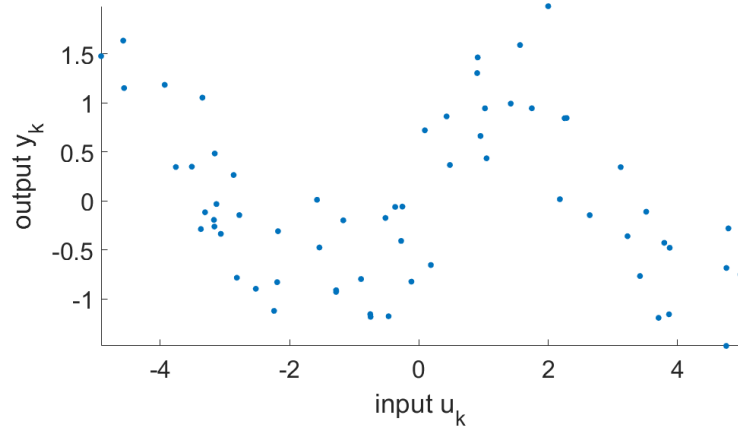
Data-Driven Modelling Process: Example



Data-Driven Modelling Process: Example

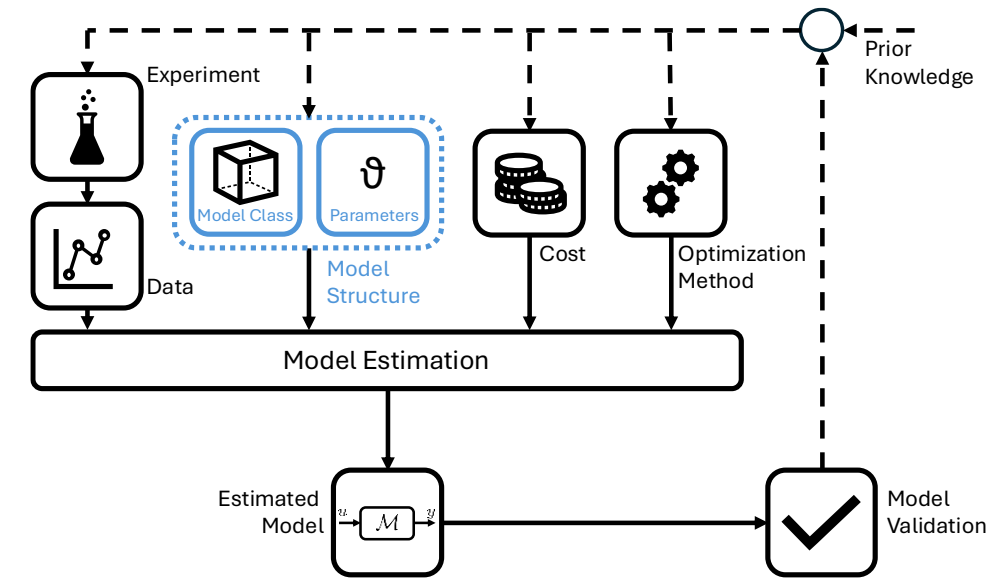


Data-Driven Modelling Process: Example

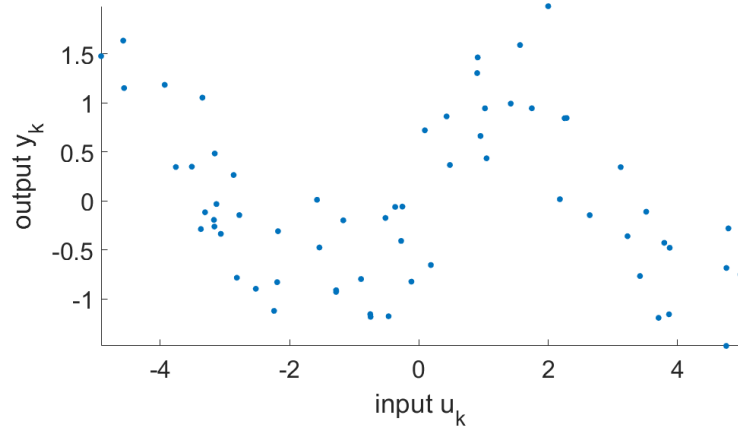


Polynomial Nonlinearity:

$$\hat{y}_k = \sum_{i=0}^{n_f} \theta_i u_k^i$$

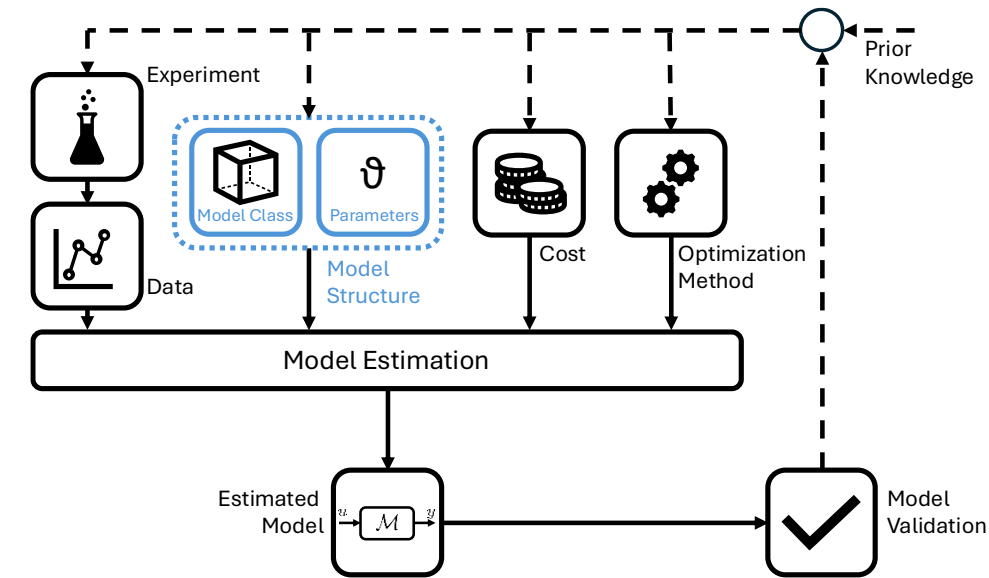


Data-Driven Modelling Process: Example

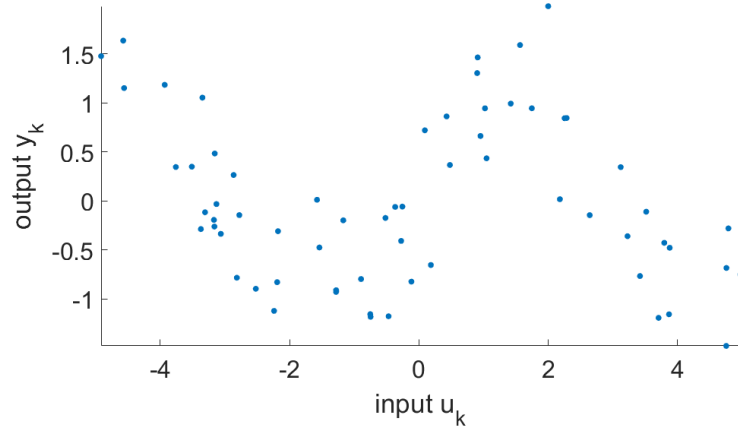


Polynomial Nonlinearity:

$$\hat{y}_k = \sum_{i=0}^{n_f} \theta_i u_k^i \longrightarrow \text{Choose the order}$$



Data-Driven Modelling Process: Example

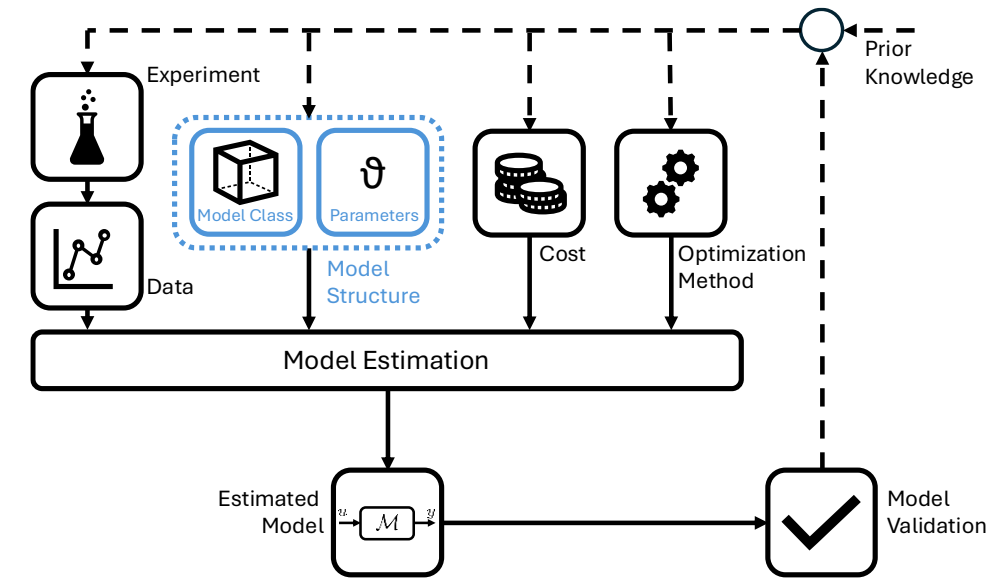


Polynomial Nonlinearity:

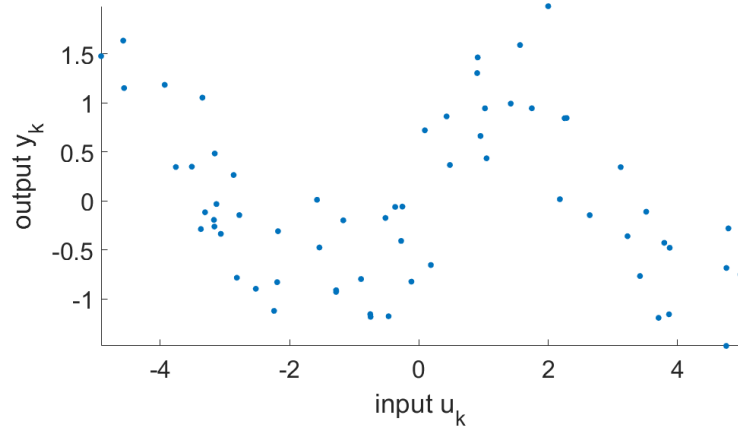
$$\hat{y}_k = \sum_{i=0}^{n_f} \theta_i u_k^i$$

Choose the order

Let's start with order 3

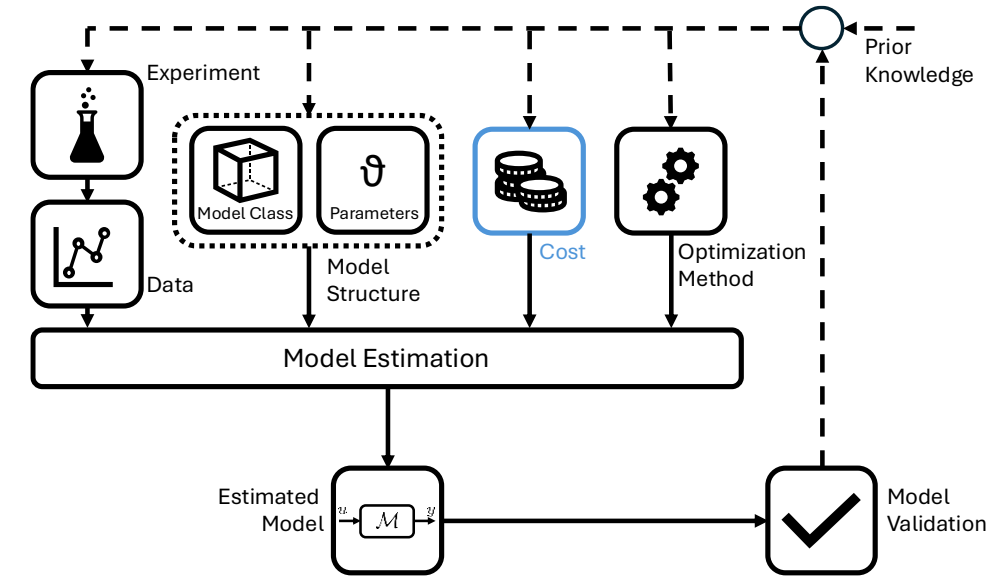


Data-Driven Modelling Process: Example

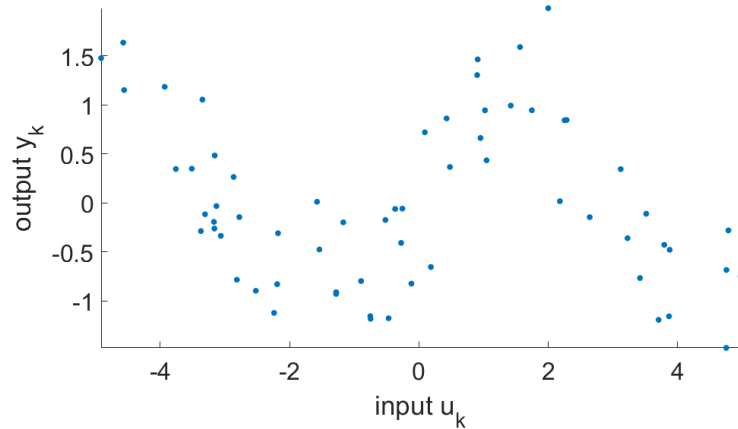


Polynomial Nonlinearity:
$$\hat{y}_k = \sum_{i=0}^{n_f} \theta_i u_k^i$$

Cost Function:
$$\frac{1}{N} \sum_{k=0}^{N-1} (y_k - \hat{y}_k)^2$$

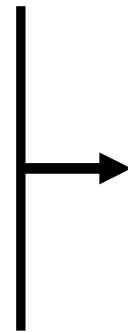


Data-Driven Modelling Process: Example



Polynomial Nonlinearity: $\hat{y}_k = \sum_{i=0}^{n_f} \theta_i u_k^i$

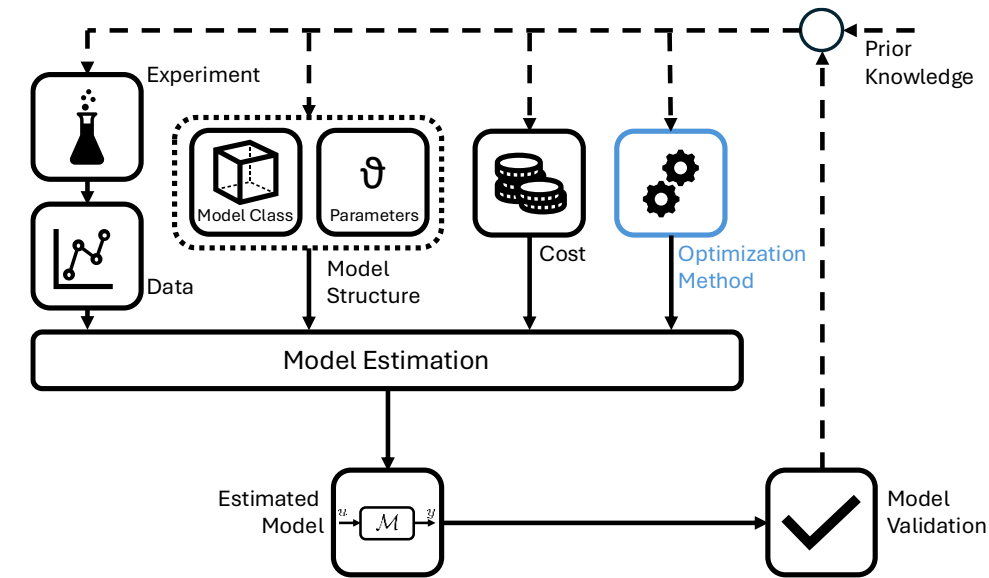
Cost Function: $\frac{1}{N} \sum_{k=0}^{N-1} (y_k - \hat{y}_k)^2$



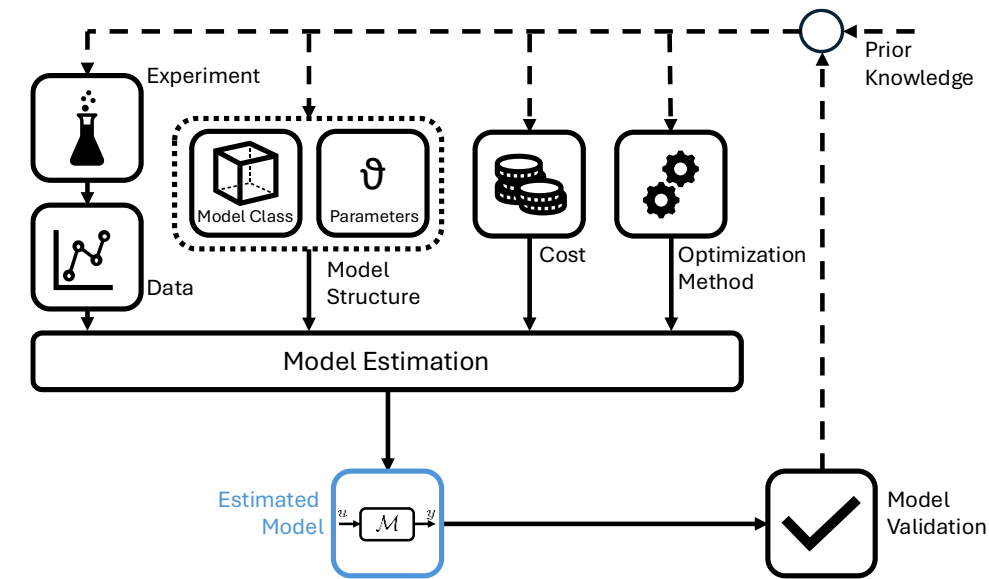
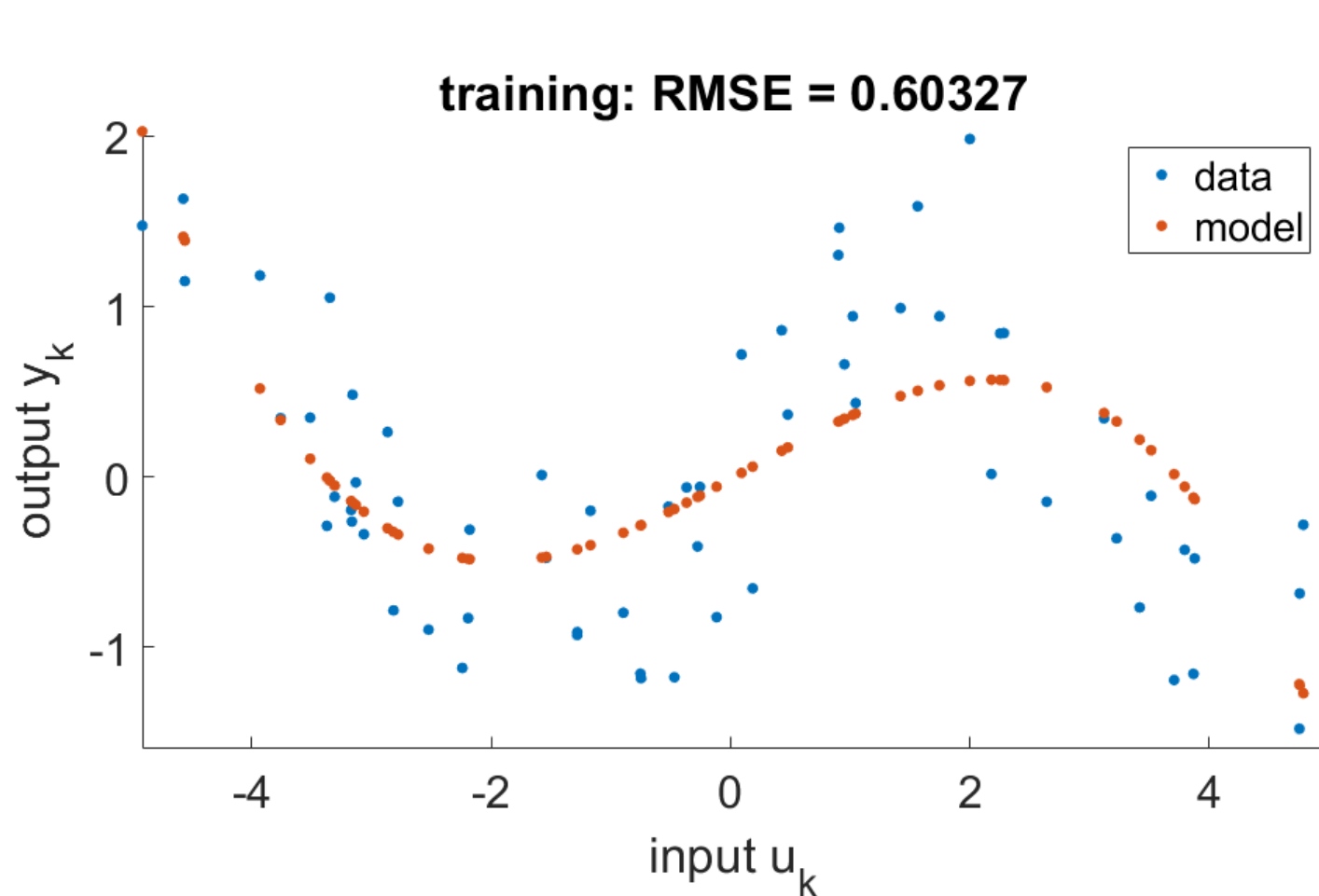
$Y = K\theta$
Linear in the parameters
Quadratic cost function



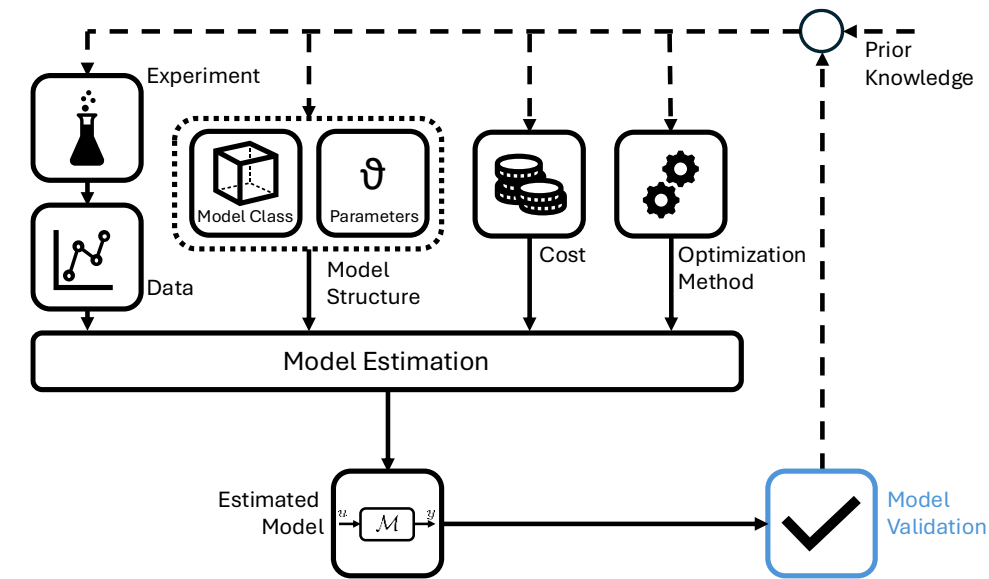
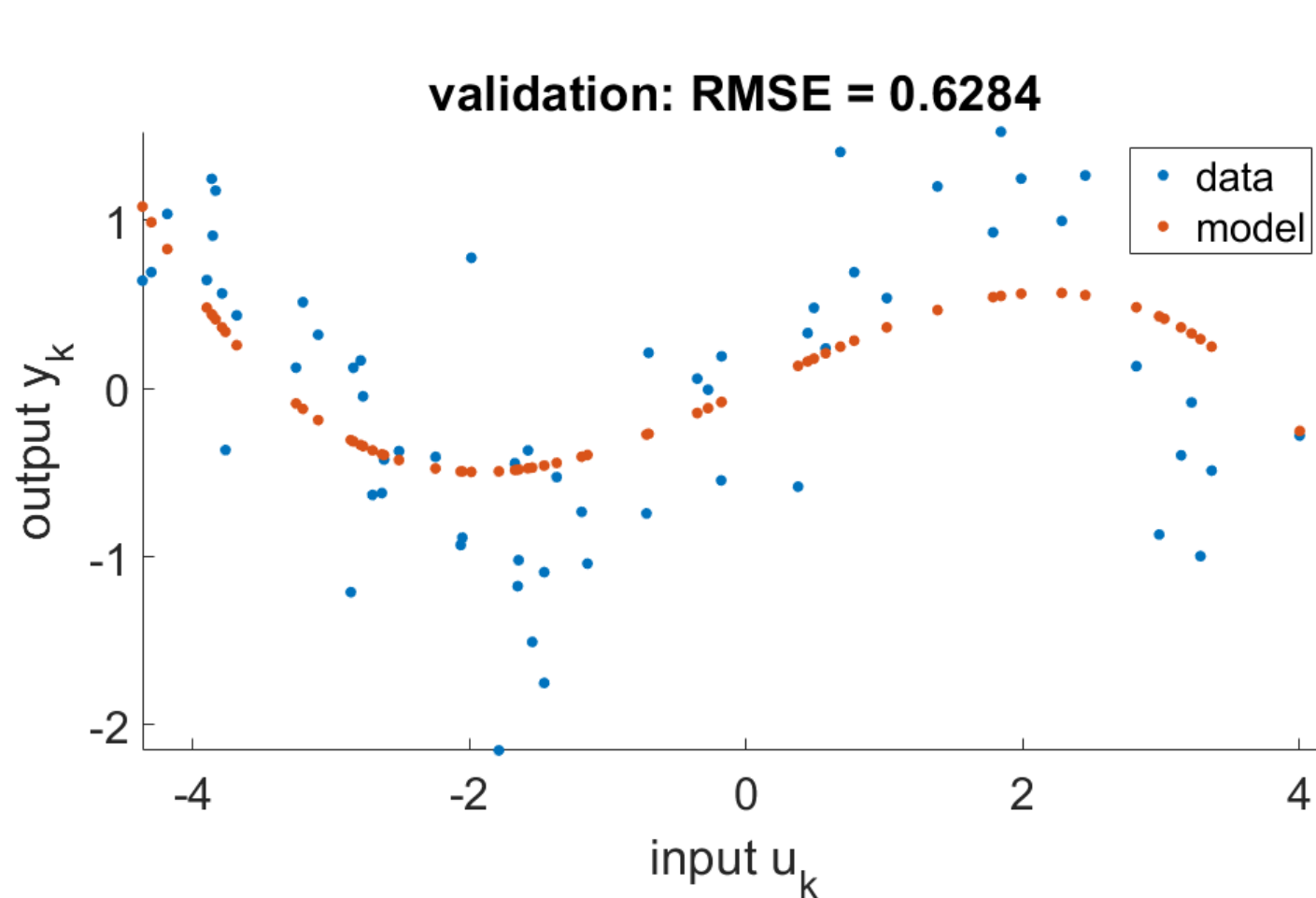
Linear Least Squares
 $\hat{\theta} = (K^T K)^{-1} K^T Y$



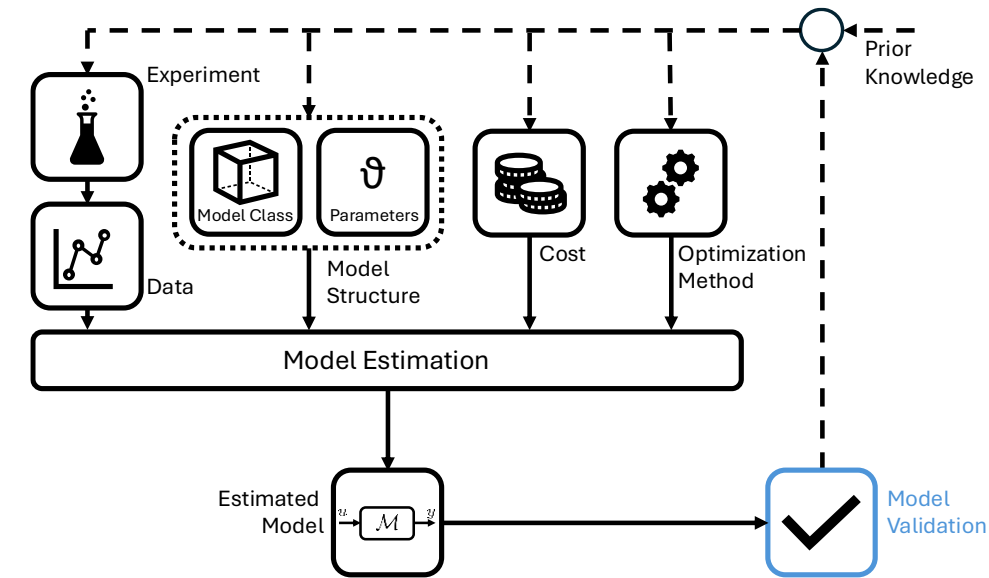
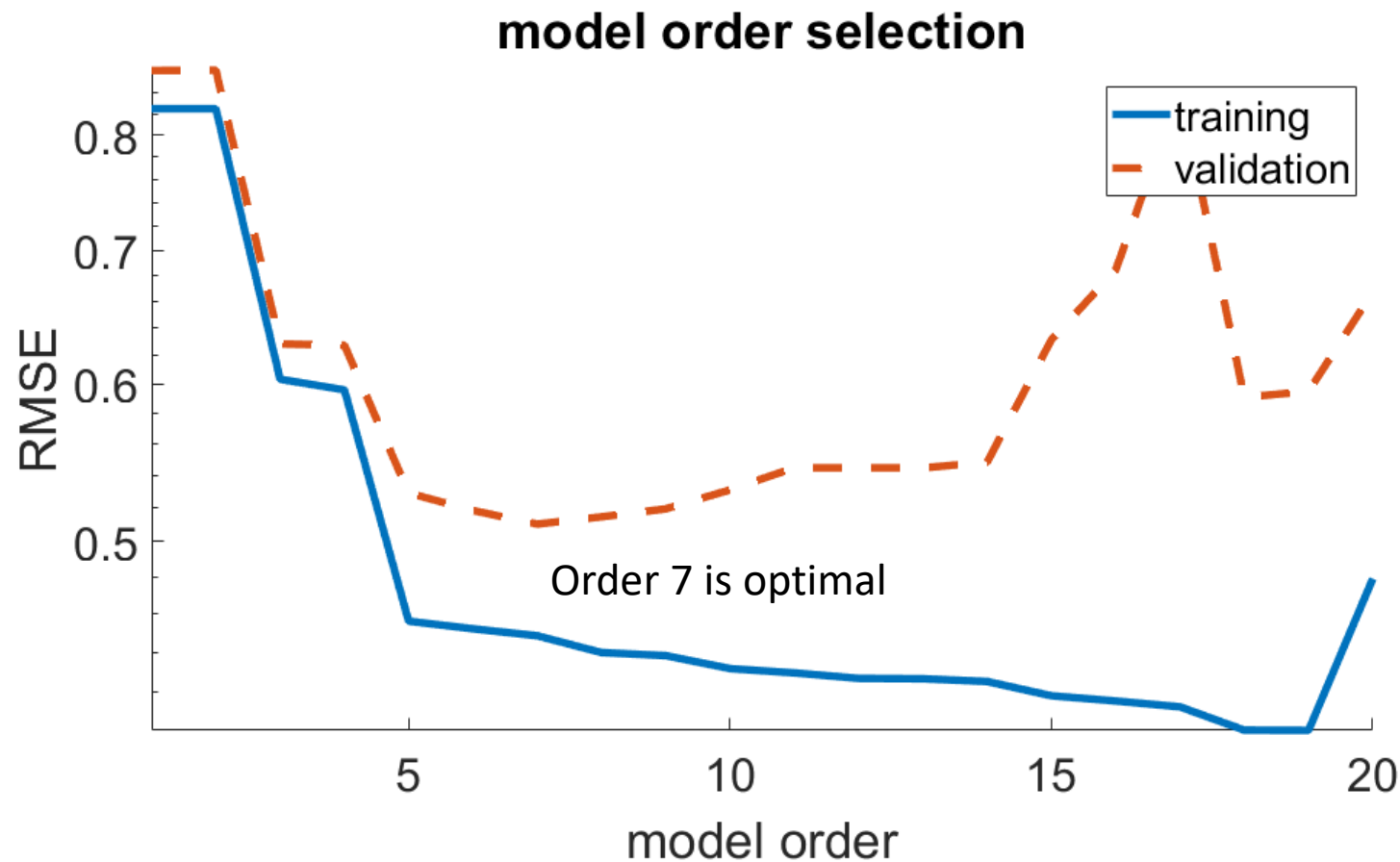
Data-Driven Modelling Process: Example



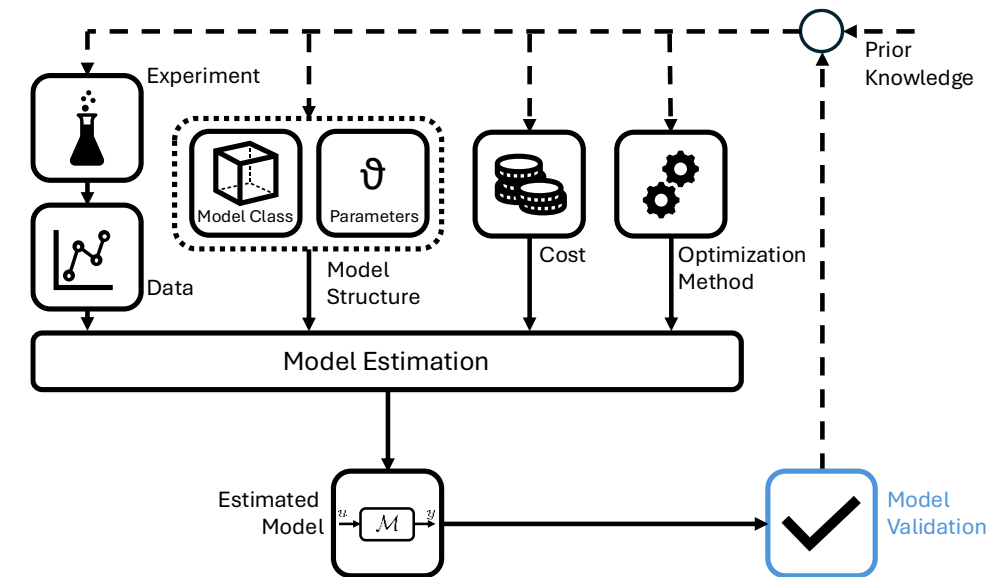
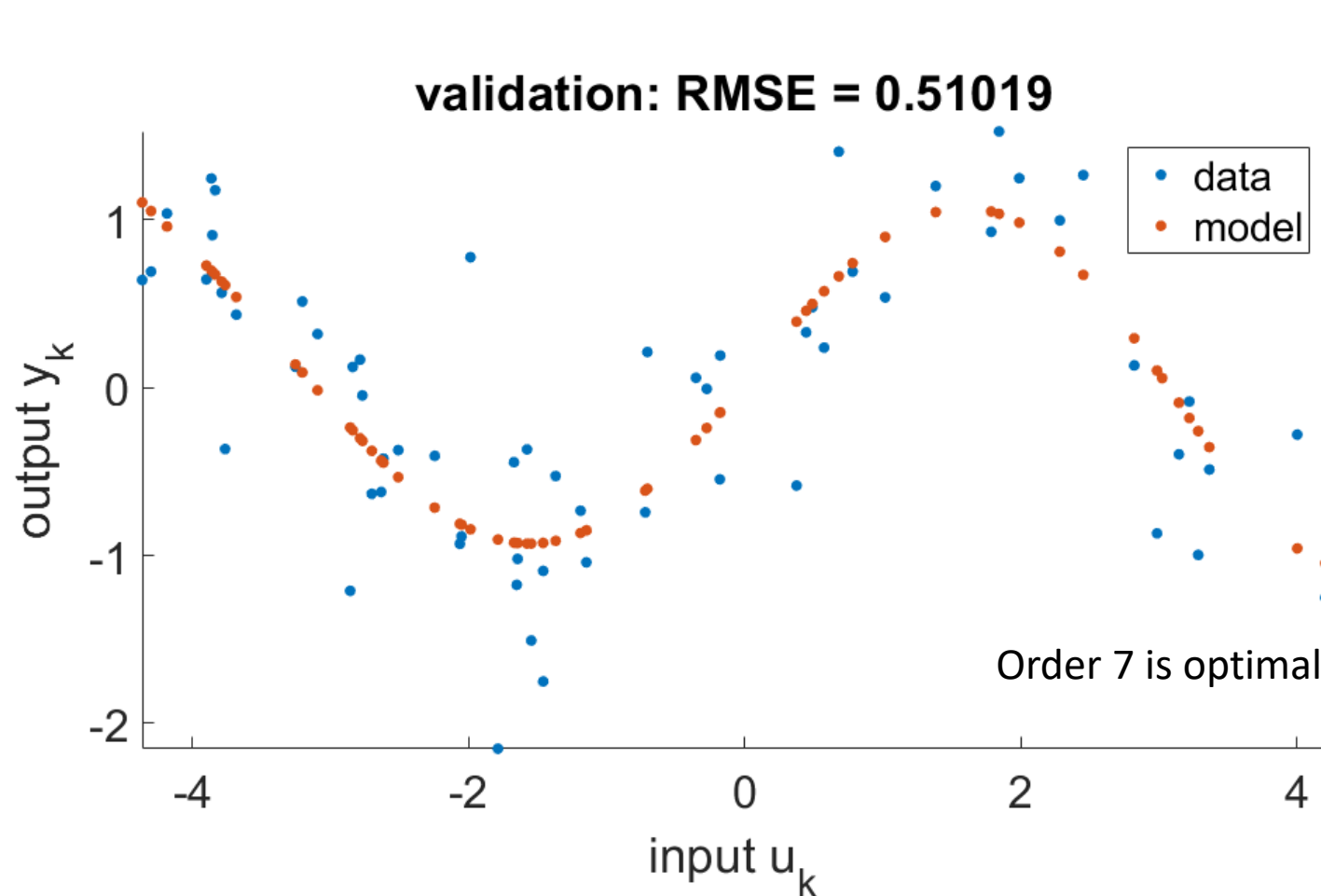
Data-Driven Modelling Process: Example



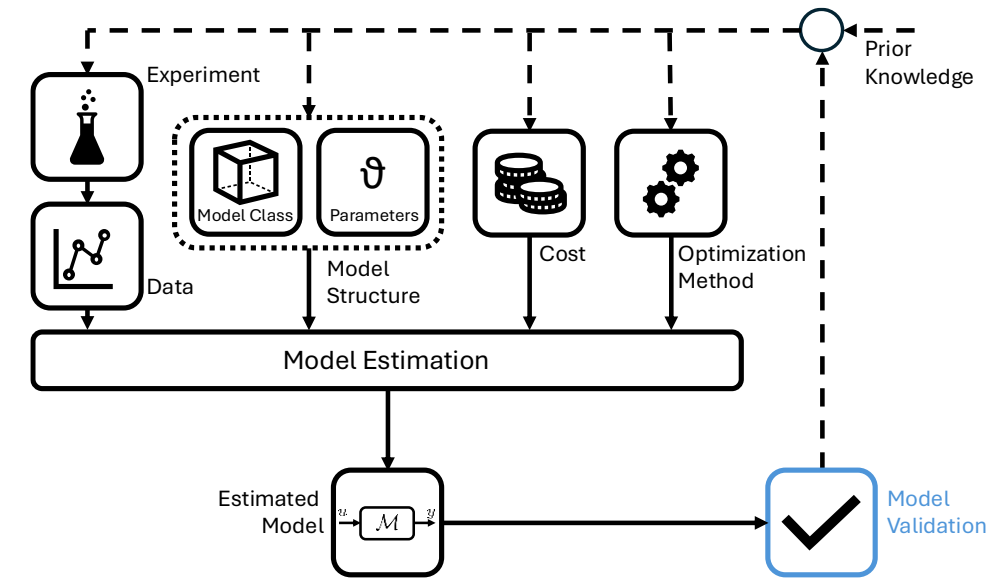
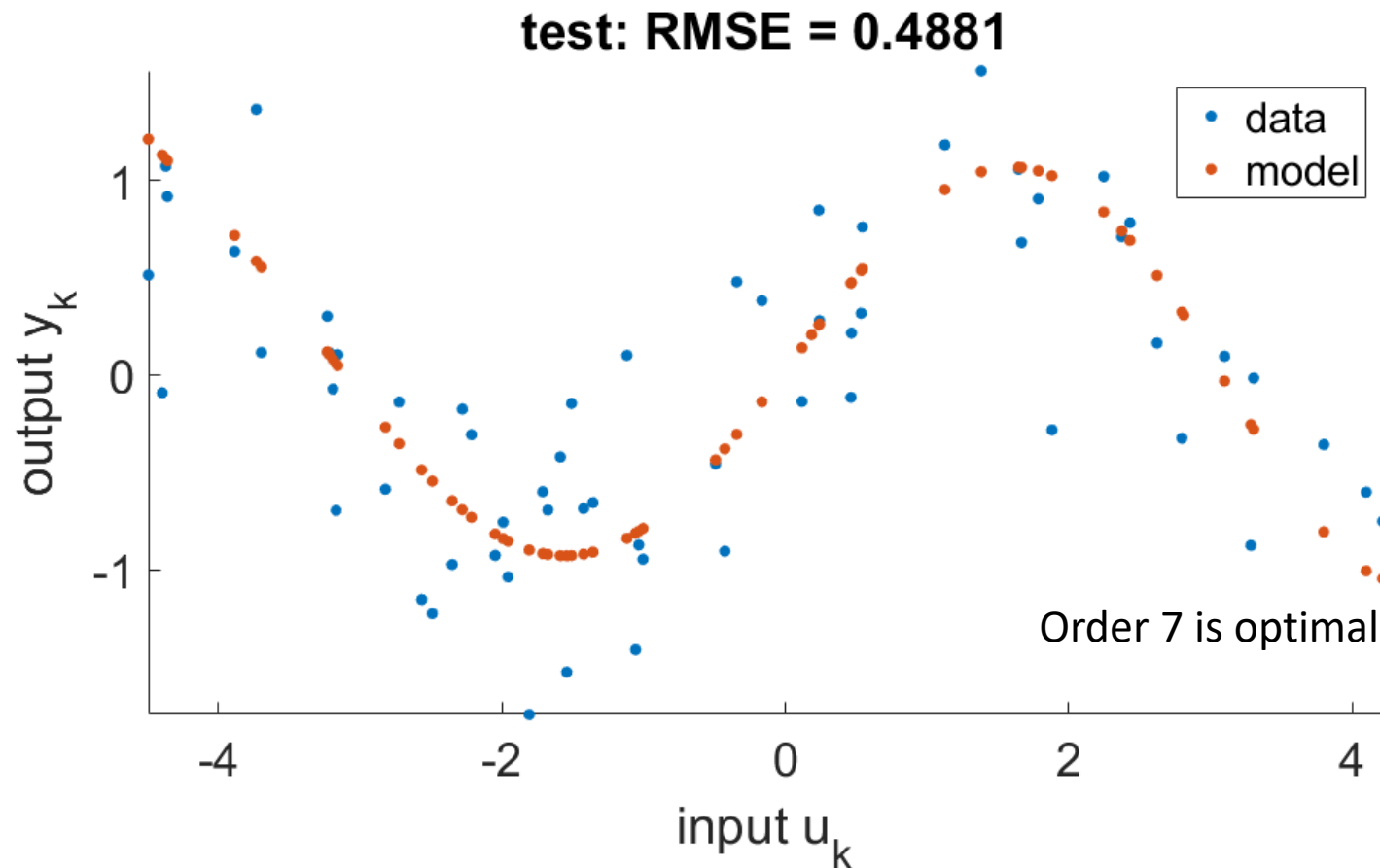
Data-Driven Modelling Process: Example



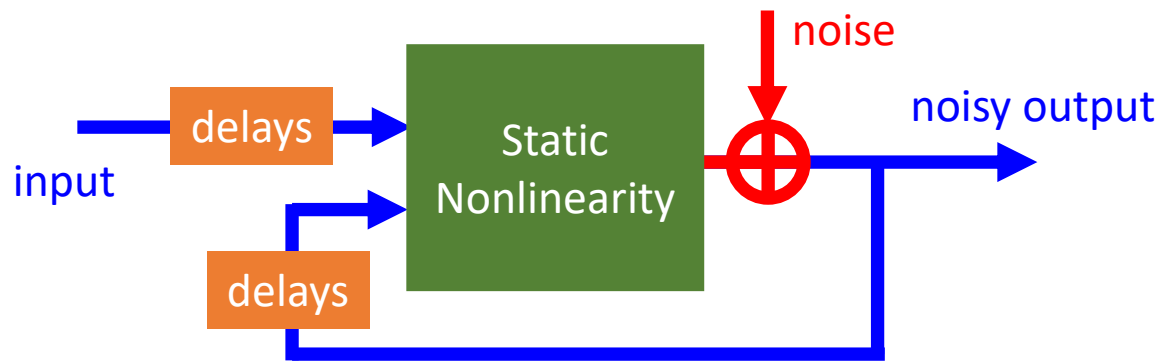
Data-Driven Modelling Process: Example



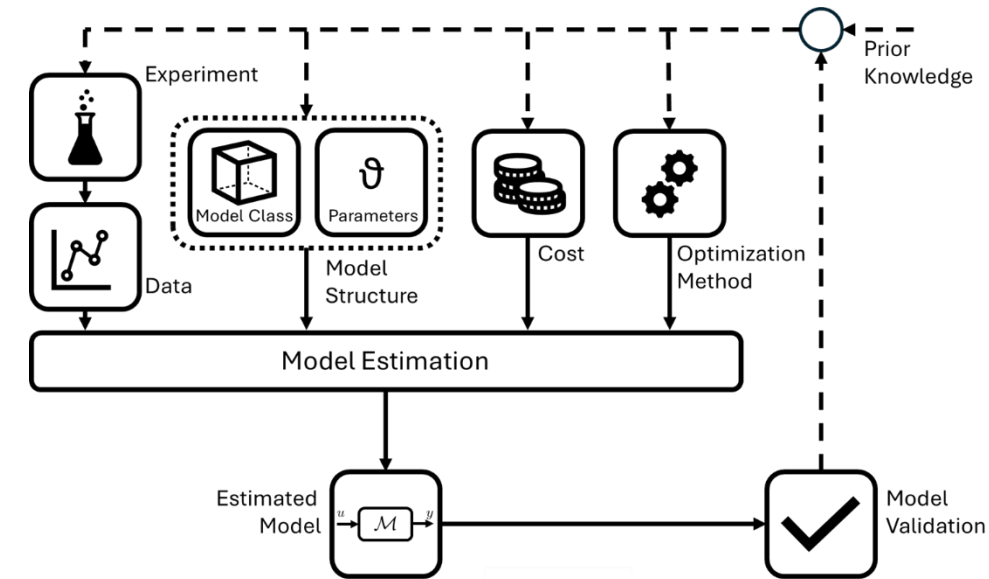
Data-Driven Modelling Process: Example



Data-Driven Modelling Process: Example



See exercises!



How Can Machine Learning Contribute?

Model Completion (see motivating example)

Encoding Prior Knowledge

Function Representation

Automate the Data-Driven Modelling Process

Alternative Learning Methods

How Can Machine Learning Contribute?

Level 1: Function Estimation

(supervised learning)

$$y_k = f(u_k, u_{k-1}, \dots, u_{k-n_b}, y_k, \dots, y_{k-n_a}) + e_k$$

Methods: GP, LS-SVM, IV-SVM, NN, ...

Level 2: Estimate Dynamical Relation

(self-supervised learning)

$$x_{k+1} = f(x_k, u_k) + w_k$$

$$y_k = h(x_k, u_k) + e_k$$

Methods: KCCA, KPCA, Deep NN, Bayesian Learning, MCMC, ...

Level 3: Automated Structure-Free Modeling

(self-supervised learning)

Level 4: Active-Interaction Based Learning

(unsupervised learning)

Towards a “logic engine” to solve dynamic modeling problems

Contents

Data-Driven Modelling Motivation

Data-Generating Systems / Models of Dynamical Systems

Estimators

Data-Driven Modelling Process

What did we not discuss?

How to represent the model (static nonlinearity)?

How to learn the model (minimize the cost)?

How can we estimate a policy instead of a model?

Background material on **Norms** and **Probabilities** (Canvas)